

学校代码: 10284
分 类 号: O643.12
密 级: 公开
U D C: 544.4
学 号:



南京大學

硕 士 学 位 论 文

论 文 题 目	基于历史日志的 REST API 模糊测试技术
作 者 姓 名	×××
专业学位类别(领域)	软件工程
研 究 方 向	模糊测试
导 师 姓 名	××× ×××

2025 年 5 月 20 日

答辩委员会主席_____

评 阅 人_____

论文答辩日期 2025 年 5 月 21 日

研究生签名:

导师签名:

Log-based REST API Fuzzing

by

× × ×

Supervised by

× × × × × ×

A dissertation submitted to
the graduate school of Nanjing University
in partial fulfilment of the requirements for the degree of

MASTER OF ENGINEERING

in

Software Engineering



Software Institute
Nanjing University

May 20, 2025

南京大学研究生毕业论文中文摘要首页用纸

毕业论文题目： 基于历史日志的 REST API 模糊测试技术

软件工程 专业 2023 级硕士生姓名： ×××

指导教师（姓名、职称）： ××× ×××

摘 要

在信息高度互联的时代，网络接口已成为支撑企业运营与个人生活的关键基础设施。REST API 因其结构简洁、通信高效等特点，逐渐发展为当下最流行的接口设计风格。因此，REST API 服务的质量至关重要，国内外如腾讯、亚马逊、谷歌等企业普遍存在对接口进行全面测试，以保障服务鲁棒性与可靠性的迫切需求。目前，主流的 REST API 测试方法大多通过服务的规格说明文档来识别接口间的依赖关系，并据此构造有效的请求序列进行测试。然而，规格说明文档本身所提供的信息较为有限，且常常存在内容缺失或描述错误的问题，导致现有方法在实际应用中效果不佳，甚至不可用。

为应对上述挑战，本文提出了一种基于历史日志的 REST API 模糊测试技术，创新性地引入服务端历史日志作为补充信息，能够有效弥补现有 REST API 测试技术的短板。REST API 服务的历史日志天然记录了用户真实的接口调用轨迹，这些轨迹不仅反映了接口的实际使用模式，还蕴含其背后的业务逻辑，有助于构造具备真实依赖关系的有效请求序列。为高效利用日志信息，本技术设计了包含规格解析、日志解析与模糊测试在内的三阶段流程。首先，提取规格文档中的基本接口定义信息，为后续请求构造与变异提供蓝本；随后，提出多种日志切分策略——包括基于 HTTP 请求语义、时间窗口与固定序列长度的方法——以将具有关联性的请求记录有序聚合，最大程度保留其业务上下文与调用逻辑；最后，基于生成的日志切片集，执行包含种子选择、种子调度、种子变异与执行反馈在内的完整模糊测试流程，以实现目标服务的全面探索与有效测试。

本文围绕所提出的技术方案实现了一套基于历史日志的 REST API 模糊测试系统。系统以命令行自动化测试工具的形式提供服务，由规格解析模块、日志解析模块、模糊测试模块三大核心模块组成，旨在充分利用日志信息对待测服务

进行全面且高效的测试；同时，系统配备命令行界面，便于用户实时监控测试进度与状态。为验证系统的实际效果，本文将其与当前主流的 REST API 模糊测试工具——RESTler、EvoMaster（黑盒模式）和 RESTCT——在相同服务环境下进行了对比实验。实验结果表明，相较于上述工具，本系统在代码覆盖率方面提升了 22%-43%，并成功发现了 2 个其他工具未能检测出的接口缺陷，有效验证了该技术在真实场景中的可行性与优越性。

关键词：REST API 测试；模糊测试；日志；大语言模型

南京大学研究生毕业论文英文摘要首页用纸

THESIS: Log-based REST API Fuzzing

SPECIALIZATION: Software Engineering

POSTGRADUATE: × × ×

MENTOR: × × × × × ×

ABSTRACT

In today's highly interconnected digital era, web APIs have become a critical infrastructure supporting both enterprise operations and daily life. Among various API styles, REST has emerged as the most popular due to its simplicity and communication efficiency. Therefore, the quality of REST API services is crucial, and there is an urgent need for companies such as Tencent, Amazon, Google, etc. to conduct comprehensive testing of REST APIs to ensure service robustness and reliability. Currently, mainstream REST API testing methods rely heavily on REST API specifications to identify dependencies among APIs and generate effective request sequences accordingly. However, the information provided by the specification is limited, and there are often problems with missing content or incorrect descriptions, resulting in existing methods being ineffective or even unusable.

To address these challenges, this thesis introduces a novel REST API fuzzing technique that leverages historical server-side logs as an additional information source. Historical logs inherently capture real user interaction traces with REST APIs, reflecting actual usage patterns and underlying business logic. These traces are valuable for constructing realistic and dependency-aware request sequences. The proposed approach adopts a three-stage framework consisting of specification parsing, log parsing, and fuzzing. First, API definitions are extracted from the specification to serve as the blueprint for building a concrete request. Then, various log slicing strategies, namely slicing by HTTP method semantics, time windows, and fixed-length sequences, are introduced to aggregate related requests and preserve their contextual and logical re-

relationships. Finally, a full fuzzing workflow is applied to the generated log slices, including seed selection, scheduling, mutation, and feedback-driven execution, to enable thorough and effective service exploration.

A prototype system implementing the proposed log-based REST API fuzzing technique has been developed. The system is implemented as a command-line automated testing tool, consisting of three core modules: specification parsing, log parsing, and fuzzing. It is designed to fully leverage log information to conduct comprehensive and efficient testing of the target service. Additionally, a command-line user interface is provided to enable users to monitor the testing progress and status in real time. To evaluate the proposed technique, the system was compared against three mainstream REST API fuzzing tools—RESTler, EvoMaster (in black-box mode), and RESTCT—through well-designed experiments. The experimental results demonstrate that, compared to the baselines, the proposed log-based technique can improve code coverage by 22%–43%; notably, it also additionally triggers two unique bugs that the baselines fail to expose, revealing the practicality and superiority of the proposed approach in real-world scenarios.

KEYWORDS: REST API Testing; Fuzzing; Log; Large Language Model

目 录

目 录	V
插图目录	IX
表格目录	XI
第一章 引言	1
1.1 背景与研究意义	1
1.2 国内（外）相关技术研究现状	2
1.3 本文的主要工作	6
1.4 本文的组织结构	7
第二章 相关技术与概念	9
2.1 HTTP 与 REST API	9
2.2 REST API 测试与 Swagger/OpenAPI 文档	10
2.3 模糊测试（Fuzzing）	12
2.4 大语言模型	13
2.5 本章小结	14
第三章 REST API 模糊测试系统的需求分析与概要设计	15
3.1 系统整体概述	15
3.2 需求分析	17
3.2.1 涉众分析	17
3.2.2 功能性需求分析	18
3.2.3 非功能性需求分析	20
3.2.4 系统用例图	21

3.2.5	系统用例描述	22
3.3	系统概要设计	26
3.3.1	系统整体架构设计	26
3.3.2	系统模块划分	28
3.3.3	系统逻辑视图	30
3.3.4	系统开发视图	32
3.3.5	系统进程视图	34
3.3.6	系统物理视图	36
3.4	本章小结	37
第四章	REST API 模糊测试系统的详细设计与实现	39
4.1	规格解析模块详细设计与实现	39
4.1.1	规格解析模块详细设计	39
4.1.2	基本信息提取功能具体实现	42
4.1.3	依赖关系推理功能具体实现	46
4.2	日志解析模块详细设计与实现	48
4.2.1	日志解析模块详细设计	49
4.2.2	多用户日志队列提取具体实现	51
4.2.3	日志切片具体实现	53
4.3	模糊测试模块详细设计与实现	57
4.3.1	模糊测试模块详细设计	57
4.3.2	种子库增强具体实现	59
4.3.3	模糊测试具体实现	64
4.4	运行实例展示	66
4.4.1	运行准备: REST 博客服务及其 Nginx 网关日志	67
4.4.2	关键功能: 接口依赖推理与日志解析	68
4.4.3	开始测试: 系统配置与模糊测试执行	69
4.5	本章小结	72
第五章	REST API 模糊测试系统的测试与评估	73
5.1	测试设计	73

5.1.1	测试准备	73
5.1.2	单元测试	74
5.1.3	集成测试	75
5.1.4	系统测试	77
5.2	实验设计	78
5.2.1	有效性分析：代码覆盖、缺陷以及响应状态码	78
5.2.2	消融实验：日志信息的作用	81
5.2.3	策略选择：日志切分策略的差异	83
5.2.4	组件验证：大模型任务准确性	84
5.3	本章小结	85
第六章 总结与展望		87
6.1	总结	87
6.2	展望	88
参考文献		89
致谢（盲审阶段，暂时隐去）		93

插图目录

2-1	一个 REST 风格的博客服务实例	10
2-2	REST 风格的博客服务的 Swagger 文档	11
3-1	基于历史日志的 REST API 模糊测试系统概览图	16
3-2	基于历史日志的 REST API 模糊测试系统用例图	21
3-3	基于历史日志的 REST API 模糊测试系统整体架构图	27
3-4	基于历史日志的 REST API 模糊测试系统逻辑视图	30
3-5	基于历史日志的 REST API 模糊测试系统开发视图	32
3-6	基于历史日志的 REST API 模糊测试系统进程视图	34
3-7	基于历史日志的 REST API 模糊测试系统物理视图	36
4-1	规格解析模块详细设计类图	40
4-2	规格解析模块详细设计顺序图	41
4-3	规格版本识别功能代码示例	42
4-4	接口信息提取功能代码示例	43
4-5	数据模型提取功能代码示例	45
4-6	依赖关系推理功能代码示例	46
4-7	依赖关系推理功能提示词模板	47
4-8	请求体参数提取功能代码示例	48
4-9	日志解析模块详细设计类图	49
4-10	日志解析模块详细设计顺序图	50
4-11	日志信息提取功能提示词模板	51
4-12	多用户日志队列生成功能代码示例	52
4-13	HTTP 方法最大前置时间字典示例	53
4-14	模糊测试模块详细设计类图	57

4-15 模糊测试模块详细设计顺序图	58
4-16 种子扩增功能代码示例	60
4-17 具体请求构造功能代码示例	60
4-18 请求体构造功能代码示例	61
4-19 路径参数补全功能代码示例	62
4-20 请求参数依赖字典示例	63
4-21 请求体参数依赖资源补全代码示例	63
4-22 模糊测试流程代码示例	64
4-23 模糊测试反馈代码示例	65
4-24 参数变异算子修改功能代码示例	66
4-25 待测博客服务的 Nginx 网关日志示例	67
4-26 解析得到的待测博客服务局部接口依赖关系示例图	68
4-27 博客服务进行日志解析过程示例	69
4-28 本次运行时的系统配置	70
4-29 基于历史日志的 REST API 模糊测试系统命令行运行界面	71
5-1 博客服务代码行覆盖变化	79
5-2 响应状态码随时间变化	81
5-3 RESTLOM 使用与不使用日志信息的代码覆盖对比	82
5-4 不同日志切分策略的测试效果对比	83

表格目录

1-1	现有 REST API 自动化测试工具对比	5
3-1	涉众分析结果	17
3-2	功能性需求列表	18
3-3	非功能性需求	20
3-4	用例“接口信息提取”详细描述	22
3-5	用例“依赖关系推理”详细描述	23
3-6	用例“日志信息提取”详细描述	23
3-7	用例“日志切片”详细描述	24
3-8	用例“种子扩增”详细描述	24
3-9	用例“切片补全”详细描述	25
3-10	用例“模糊测试”详细描述	25
3-11	用例“测试报告生成”详细描述	26
4-1	待测 REST API 博客服务接口列表	67
5-1	测试服务器硬件及操作系统信息	73
5-2	软件环境依赖	74
5-3	单元测试用例列表	75
5-4	规格说明文档解析测试设计	76
5-5	历史日志解析测试设计	76
5-6	种子库增强测试设计	77
5-7	模糊测试验证测试设计	77
5-8	系统测试用例列表	78
5-9	不同工具发现缺陷数统计	80
5-10	不同大模型在依赖推理任务上的表现（正确/失败）	84

第一章 引言

基于历史日志的 REST API 模糊测试技术是本文提出的一种针对 REST API 服务的新型自动化测试技术，旨在结合真实系统运行过程中产生的历史日志信息，增强测试用例生成质量以及整体测试效果。作为引言部分，本章将首先介绍本文的背景与研究意义，说明当前 REST API 测试所面临的局限与挑战（第1.1节）；接着，回顾 REST API 测试技术以及模糊测试方法发展现状，综述当前相关的测试工具和测试方法（第1.2节）；最后，概述本文的主要工作内容以及结构安排，为后续章节的开展奠定基础（第1.3节、第1.4节）。

1.1 背景与研究意义

在当下高度信息化的社会中，网络已经成为各类系统互联互通的基础。网络接口作为系统与外部网络之间的桥梁，发挥着至关重要的作用^[1]。表述性状态转移（Representational State Transfer, 简称 REST）是一种当前广泛流行的 Web 接口设计规范^[2]，因其简洁、灵活、可扩展等特性，被众多企业广泛采用来构建其 Web 接口，例如谷歌、亚马逊、推特、腾讯和阿里云等^[3]。在实际应用中，REST API 不仅被用于企业内部系统中各微服务之间的通信^[4]，还常作为开放接口，供第三方开发者调用，从而获取企业内部资源并集成到自身的系统中。这种广泛的应用场景使得 REST API 成为现代软件架构中不可或缺的核心组件。

随着 REST API 的广泛应用，开展全面且高效的测试以保障其可靠性与鲁棒性变得愈发重要。当前，对 REST API 的测试主要分为人工测试和自动化测试两种方式。人工测试依赖测试人员手动设计测试用例，并借助 Postman、Apifox、REST Assured 等接口测试工具构造请求并发送至目标服务进行测试。自动化测试工具通常以待测 REST 服务的规格说明文档（如 Swagger^[5] 或 OpenAPI 文档^[6]）为输入，通过解析文档中的接口定义、参数信息、依赖关系乃至自然语言描述等关键内容，自动生成测试用例并执行测试^[7-15]。

然而，现有的 REST API 测试方法在实际应用中仍存在诸多问题和局限。人工测试要求测试人员对待测 REST 服务的功能有较为全面的了解，同时需要手动编写测试用例、构造请求并逐一发送执行，整个过程繁琐耗时且容易操作失误。自动化测试虽然在效率和规模上有优势，但目前的自动化方法主要依赖于待测服务的规格说明文档。这些文档通常由开发人员撰写或者框架自动生成，因此其内容可能存在不完整、不准确甚至与实际服务不一致的情况，导致最终方法应用效果不佳甚至不可用^[16]。更重要的是，规格文档本身包含的信息有限，缺乏更高层业务逻辑、状态依赖或接口调用顺序等关键信息，难以支持测试工具生成真实场景下复杂的测试用例，从而限制了更深层的功能代码覆盖和缺陷探测^[17]。

针对上述问题，本文设计并实现了一种基于历史日志的 REST API 模糊测试技术。在现代企业级系统中，使用日志记录服务的调用过程已成为一种常见且必要的运维实践。REST 服务的历史日志包含了大量真实的用户调用信息，比如接口调用顺序、请求参数值以及请求参数组合约束等，高度贴合服务的实际使用场景，具备天然的时序性。这些特征使日志成为识别接口依赖关系、生成高质量测试用例的重要信息源。本文提出的技术充分挖掘日志中蕴含的高质量信息，设计多种策略对其内容进行解析，自动构造出一系列符合真实用户使用场景的测试用例。在此基础上，该技术进一步开展完整的模糊测试流程，结合测试用例变异生成、执行反馈等机制，实现对待测 REST 服务更加全面和深入的自动化测试，从而显著提升测试的缺陷发现能力和代码覆盖率。

1.2 国内（外）相关技术研究现状

自 2016 年起，学术界和工业界相继提出了多种面向 REST API 的测试方法和自动化工具，旨在提升 REST API 测试的效率与质量。这些研究大致可以按照测试所依赖的信息类型划分为白盒方法与黑盒方法。本节首先围绕这两类方法的代表性研究和主流工具进行介绍与分析。

黑盒方法（Black Box）不依赖于系统内部实现，仅基于对外可见的接口信息进行测试，是 REST API 测试常用的测试方法。此类方法通常以 REST API 的规格说明文档（如 Swagger/OpenAPI）为主要信息源，通过解析其中的接口定义、请求参数类型和响应结构等，自动化地生成测试请求。典型的工作如 RESTler、

RESTCT、MOREST、SCHEMATHESIS 等，均采用此思路。

RESTler^[11] (2019) 是首个具备状态感知能力的 REST API 模糊测试工具。在其诞生之前，无论在工业界还是学术界，尚未出现专门面向 REST API 的独立模糊测试工具，常见的仅是一些针对 HTTP 协议的通用模糊测试工具^[18-19]。这些工具通常依赖预定义的启发式规则或用户手动配置的规则，对 HTTP 请求进行解析和回放，缺乏对接口语义和状态依赖的深入理解。RESTler 的核心创新在于对规格说明文档进行轻量级的静态分析。通过解析 Swagger/OpenAPI 文档，RESTler 能够推断接口之间的“生产者-消费者”关系：例如，当接口 A 创建了资源 x，而接口 B 依赖于该资源 x 作为输入参数时，RESTler 会自动确保接口 A 在接口 B 之前被调用，从而构造出语义上有效的请求序列。此外，RESTler 还具备运行时的动态反馈能力。它在测试执行过程中实时监控接口调用的响应信息，根据服务返回的状态码、错误提示等信息动态调整后续请求生成策略，从而有效规避无效的接口组合，提高测试效率与覆盖率。

SCHEMATHESIS^[10] (2019) 使用基于属性的测试方法 (Property-Based Testing, PBT) 对 REST API 进行系统性测试。与传统的测试方法相比，基于属性的测试关注的是被测程序应当满足的一般性语义属性，而非对某个特定输入产生特定输出的行为^[20]。在 REST API 的测试场景中，SCHEMATHESIS 重点验证服务是否遵循其规格说明文档中定义的语义约束。具体而言，一方面，服务的响应应符合 RFC 7231¹中对 HTTP 状态码的语义定义，比如状态码 200 应返回非空的响应体，状态码 500 表示服务器端发生异常等；另一方面，响应内容还应满足 Swagger/OpenAPI 文档中所定义的结构和约束，例如，返回值的字段应完整且类型匹配，未认证请求不应获取敏感信息，输入输出应严格遵守接口定义等。通过构造大量随机但合规的请求并自动验证其响应是否违反上述属性，SCHEMATHESIS 能够高效发现接口实现中的规范违背与潜在错误。

RESTTESTGEN^[12] (2021) 是一种自动化的 REST API 黑盒测试工具，其主要亮点在于引入了操作依赖图 (Operation Dependency Graph, ODG) 来指导测试用例的生成。该图的设计理念借鉴了程序分析中的程序依赖图 (Program Dependency Graph, PDG)^[21]，并对其进行了针对 REST 场景的定制化改造。在 RESTTESTGEN 中，操作依赖图的每个节点表示一个具体的 REST API 操作，即

1 <https://datatracker.ietf.org/doc/html/rfc7231>

由某一资源的 URI 与对应的 HTTP 方法共同组成的端点（endpoint/operation）；图中的边则表示不同端点之间的依赖关系。当两个端点之间存在共同的属性（如参数、资源标识等）时，若端点 A 的输出包含某个属性 x，且该属性 x 是端点 B 的输入所需，则 RESTTESTGEN 判定端点 B 依赖于端点 A，并在依赖图中添加一条以 x 为标识的有向边。通过构造这样一个操作依赖图，RESTTESTGEN 更有可能生成有效的测试序列；同时，RESTTESTGEN 支持在运行时动态地更新操作依赖图，使其不断适应当前服务的实际行为。

MOREST^[13]（2022）是一种与 RESTTESTGEN 类似的基于图建模的 REST API 黑盒测试工具，同样通过图结构表示接口之间的依赖关系，但在建模能力和信息表达方面更加丰富和灵活。MOREST 引入了一种名为 REST 服务属性图（RESTful-service Property Graph, RPG）的图结构，其灵感来源于属性图（Property Graph）模型^[22]。与 RESTTESTGEN 的操作依赖图不同，REST 服务属性图不仅包括 REST API 操作节点，还引入了来自规格说明文档的数据模型作为图的节点元素，从而增强了图的表达能力。在 REST 服务属性图中，边的类型更加多样：除了表示操作之间依赖关系的边外，还包括从操作到数据模型的有向边（表示某个操作使用或生成某个数据模型）、从数据模型到操作的有向边（表示某个模型被某个操作消费）、以及数据模型之间的无向边（表示模型之间存在字段级别的相似性或嵌套关系）。这种更全面的图结构设计使得 MOREST 能够灵活地应对更加复杂的 REST API 测试场景。

RESTCT^[14]（2022）是一个系统化、全自动的 REST API 测试方法，首次将组合测试（Combinatorial Testing, CT）应用到 REST API 测试领域中。组合测试的核心思想是：大多数软件缺陷是由少量输入参数之间的交互组合引起的^[23]。因此，组合测试通过设计合理的测试集，确保一定数量参数之间的所有可能组合都被覆盖，以在有限成本下最大化缺陷检测能力^[24-25]。RESTCT 通过约束序列覆盖数组（Sequence Covering Array, SCA）和约束覆盖数组（Covering Array, CA），系统性地覆盖不同操作之间的交互顺序以及请求的参数值，从而有效触发在特定操作顺序或参数组合下才会暴露的服务行为异常。

除了以上典型的黑盒测试方法，REST API 测试领域也存在白盒测试工具。白盒方法依赖于被测系统的内部信息，如源代码、控制流图或中间表示等，通过静态分析或符号执行等手段生成具有覆盖目标的测试用例。

EvoMaster^[8] (2019) 是一款面向 REST API 的自动化白盒测试工具，其核心目标是自动生成高覆盖率的测试用例，并借此发现系统中的潜在缺陷。与大多数黑盒测试工具不同，EvoMaster 假设测试者拥有系统的源代码访问权限，因此能够在白盒信息基础上进行更有效的测试生成。EvoMaster 采用基于搜索的软件测试方法^[26]，通过进化算法改善测试用例的生成效果。具体来说，EvoMaster 使用一种名为 MIO (Many Independent Objectives)^[27] 的演化算法来生成测试用例。每个测试目标（如语句覆盖、分支覆盖、返回状态码等）拥有独立的种群和适应度评估机制；测试用例被视为解空间中的个体，随着搜索过程演进，逐步靠近最优解。

表 1-1 现有 REST API 自动化测试工具对比

工具名称	类型	亮点	实现语言	开源时间
RESTler	黑盒测试	接口依赖推断；模糊测试	Python F#	2020
SCHEMATHESIS	黑盒测试	基于属性的测试方法	Python	2019
RESTTESTGEN	黑盒测试	操作依赖图（ODG）	Java	2021
MOREST	黑盒测试	REST 服务属性图（RPG）	Python	2023
RESTCT	黑盒测试	组合测试方法	Python	2022
EvoMaster	黑盒/白盒	基于搜索的测试方法	Kotlin	2016

表1-1总结了当前主流的 REST API 自动化测试工具，展示了它们各自的核心特点、测试类型、实现方式以及开源时间等关键信息。通过上述对相关研究工作的综述，可以得出以下结论：REST API 模糊测试技术已成为当前软件测试领域的一个研究热点，已有多种自动化测试工具相继被提出。这些工具大多采用黑盒测试策略，研究重点集中在如何高效获取并建模 REST 服务各接口之间的依赖关系，以此为基础生成结构合理、执行有效的测试用例，从而提升测试的覆盖率和缺陷发现能力。

尽管现有的 REST API 自动化测试工具已尝试通过多种方式获取接口之间的依赖关系，如静态分析、操作依赖图、REST 服务属性图以及进化算法等，但其实际测试效果仍不尽如人意。相关实验结果表明，在统一的测试基准集上，现有工具在行覆盖率方面的表现较差，平均每个项目的覆盖率最高仅能达到 60%，且能够发现的缺陷类型十分有限^[28]。造成这一现象的根本原因在于，目前的测试工具普遍仅依赖于 REST API 的规格说明文档作为测试知识的唯一来源，缺乏对服务实际业务逻辑深入理解，因而难以生成具有高效覆盖和缺陷探测能力的

测试用例。为弥补上述不足，本文提出了一种基于历史日志的 REST API 模糊测试技术。该系统通过挖掘服务运行过程中的历史日志信息，从中提取带有真实用户行为特征的测试用例切片，进而生成更贴合实际场景的测试用例，有望显著提升 REST API 自动化测试工具在测试用例生成质量和最终测试效果方面的表现。

1.3 本文的主要工作

根据以上对现有 REST API 自动化测试工具方法和不足的分析，本文设计并实现了一种基于历史日志的 REST API 模糊测试技术。该技术以服务运行过程中产生的历史日志作为除规格说明文档之外的另一重要信息来源，挖掘日志中隐含的真实业务场景与调用逻辑。通过截取其中具有代表性的日志片段，构造贴近实际使用情境的测试用例，为后续的模糊测试过程提供更加准确、有效的输入基础，从而显著提升整体测试的覆盖深度与缺陷发现能力。

本技术的实现分为三个阶段：规格文档解析、历史日志解析以及模糊测试执行。规格文档解析负责从待测服务的 Swagger 或 OpenAPI 规格说明文档中提取接口定义、参数信息及其依赖关系，为后续具体请求构造提供基础的“蓝图”；历史日志解析将用户提交的服务端历史调用日志进行切分，提取出具有实际业务逻辑特征的日志切片，用于构建初始种子库，这些种子能够代表真实场景下的调用序列；模糊测试阶段在前两个阶段构建的信息基础上，执行完整的模糊测试流程，涵盖种子队列生成、种子选择、种子变异、测试用例执行与动态反馈等环节。本文提出的技术旨在解决下列难点和挑战：

1) 规格说明文档解析复杂性高。 REST API 的规格说明文档结构层次深、语义丰富，常涉及嵌套的数据模型和复杂的参数约束。此外，当前广泛使用的 Swagger 和 OpenAPI 两种规范在格式和内容表达上存在显著差异，而目前尚无成熟的通用解析库或工具。因此，本技术需自行实现一套具备高度鲁棒性的文档解析机制。

2) 历史日志的处理与提取存在挑战。 尽管日志中蕴含着丰富的用户调用行为和业务逻辑信息，但真实生产环境下的日志数据往往庞杂冗余，有关无关请求混杂在一起。如何从中筛选出有效的、具有关联性的请求，构建出代表某一特定场景的测试用例切片，是一项关键难题。此外，这些从日志中提取的切片往往不

包含所有必要的资源创建请求，系统还需基于接口间依赖关系进行合理补全。

3) 模糊测试流程设计要求精细。模糊测试作为一种动态测试方式，其效果高度依赖于测试用例的生成质量及反馈机制的设计。因而在种子选择策略、变异方法、执行流程、动态反馈机制等多个环节上都需要进行细致设计与调优。

总的来说，本文完成了以下工作：

- 提出了一种基于历史日志的 REST API 模糊测试技术，突破传统测试方法对规格文档的单一依赖，能够结合真实用户行为进行更贴近实际使用场景的测试，从而提高测试的有效性与覆盖深度；
- 设计并实现了多种日志切片策略，这些策略从原始历史日志中提取具有调用依赖关系的请求序列，将其作为高质量的测试用例初始种子，有效保留了真实业务逻辑中的状态约束与调用顺序；
- 设计了一套针对 REST API 的完整模糊测试流程，包括多种变异算子与反馈策略，使得模糊测试过程能够动态调整测试方向，增强缺陷触发能力和代码路径覆盖能力；
- 设计了一套用于解析 REST API 规格说明文档的通用解析模块，支持 Swagger 和 OpenAPI 两种主流文档格式，能够准确提取接口结构、参数定义与依赖信息，为请求构造与变异提供基础支持；
- 在真实 REST 服务上进行了实验评估，验证了本文所提出技术在代码覆盖率、缺陷检测能力等方面的优势。

1.4 本文的组织结构

本文共分为六章，主要组织结构如下：

第一章：引言。介绍基于历史日志的 REST API 模糊测试技术的背景与研究意义，并对现有的主流工具和方法进行综述，最后总结了本文的研究目标和主要工作；

第二章：相关技术与概念。阐述了本文研究所涉及的基础技术与相关概念，包括 HTTP 与 REST 原理、REST API 测试方法与 Swagger/OpenAPI 规范、模糊测试技术以及大语言模型等内容，为后文方法设计与实现提供技术支撑；

第三章：需求分析与概要设计。需求分析部分，在系统整体概述的基础上，

分析了系统的功能性需求与非功能性需求，并通过用例图与用例描述展示系统预期的行为；随后，基于系统整体架构进行模块划分，并采用“4+1”视图模型对系统架构进行多角度描述；

第四章：系统详细设计与实现。按照功能模块对基于历史日志的 REST API 模糊测试系统进行详细设计与实现说明，主要包括：规格解析模块、日志解析模块以及模糊测试执行模块。同时，通过一个具体示例展示了系统的运行流程；

第五章：系统测试与评估。主要介绍对系统进行的测试设计以及实验设计，验证基于历史日志的 REST API 模糊测试的有效性；

第六章：总结与展望。总结本文所完成的主要工作与研究成果，分析了当前技术的局限性，并对未来可能的优化方向与拓展研究进行了展望。

第二章 相关技术与概念

基于历史日志的 REST API 模糊测试系统通过从服务端日志中提取包含真实用户行为的请求切片，并将其构造为测试用例，进而开展全面的模糊测试，以实现对待测服务的有效测试。该系统涉及的核心技术与相关概念包括：HTTP 与 REST API（第2.1节）、REST API 测试与 Swagger/OpenAPI 文档（第2.2节）、模糊测试技术（第2.3节）以及大语言模型（第2.4节）。

2.1 HTTP 与 REST API

超文本传输协议（Hypertext Transfer Protocol, HTTP）是一种应用层协议，用于分布式、协作式超媒体信息系统的数据传输。自 1990 年由 Tim Berners-Lee 提出以来，HTTP 已成为万维网（World Wide Web, WWW）数据通信的核心协议^[29]。HTTP 采用典型的客户端-服务器模型：客户端（如浏览器或应用程序）向服务器发送请求，服务器则根据请求返回相应的响应结果。HTTP 协议本身具有无状态性，即服务器不会保存客户端之间的交互上下文，每一次请求都被独立处理，这一特性简化了服务端的设计和实现。HTTP 定义了一系列具有明确操作语义的请求方法，比如 GET（获取指定资源）、POST（提交数据或创建新资源）、PUT（替换目标资源）、DELETE（删除指定资源）等。服务器的响应结果通常通过三位数的状态码进行表示，以反映请求的处理情况，例如 2XX（请求成功）、4XX（客户端错误）、5XX（服务器错误）。

表述性状态转移（Representational State Transfer, REST）是由 Roy Fielding 于 2000 年在其博士论文中提出的一种面向资源的软件架构风格，旨在为分布式系统的设计提供一套统一的约束性原则^[30]。遵循 REST 架构风格设计的接口被称为 REST API 或 RESTful API。REST API 基于 HTTP 协议，以网络资源为核心，其服务设计强调资源的清晰组织与一致性呈现。图 2-1 展示了一个用于管

理博客资源的 REST API 服务示例¹。该服务涉及三类资源：博客（Blog）、评论（Comment）和话题（Topic），其中评论是博客的子资源，它们均通过统一资源标识符（Uniform Resource Identifier, URI）进行唯一标识。REST 架构将对资源的各种操作映射为 HTTP 协议中定义的标准方法。例如，客户端可以通过对 URI “/blog” 发起 POST 请求来创建博客，或使用 GET 请求查询博客资源；又如，针对特定评论子资源，可以通过 URI “/blog/{blogId}/comment/{id}” 发起 PUT 请求以修改评论，或发起 DELETE 请求以删除评论。

Blog Controller Blog Controller		▼
GET	/blog	get all blogs or blogs of a user
POST	/blog	create a new blog
GET	/blog/{id}	get a blog by id
PUT	/blog/{id}	update a blog
DELETE	/blog/{id}	delete a blog
Comment Controller Comment Controller		>
Topic Controller Topic Controller		>

图 2-1 一个 REST 风格的博客服务实例

REST 本质上不是一种协议或标准，而是一组架构设计准则，主要用于指导如何通过网络对资源进行有效的管理与操作。REST 强调可扩展性、通用接口、组件自治部署和中间组件，旨在减少交互延迟、增强系统安全性。REST 并没有强制规定如何在其底层实现，而是提供了一组高级设计约束，例如关注点的分离、无状态性、缓存支持、统一接口、多层性和代码按需获取等^[31]。

2.2 REST API 测试与 Swagger/OpenAPI 文档

REST API 服务常被用作软件系统内部负责管理资源的基础服务，其接口被众多的下游服务所依赖，一旦这些服务内部出现故障，极大概率会影响到下游服务的使用，甚至可能会造成整个软件系统的瘫痪，因此保障 REST API 服务的安全和质量至关重要^[32]。REST API 测试的输入/输出形式为 HTTP 请求/响应序列，进行 REST API 自动化测试通常分为四步：1）解析理解 REST API 规格说明文档；2）生成 HTTP 请求调用序列；3）为序列中的请求匹配参数值；4）发送请

1 <https://github.com/dingyangny/blog>

求并进行测试预言^[33]。

REST API 服务的规格说明文档通常采用 Swagger 或 OpenAPI 格式进行编写，它们使用结构化的 JSON 或 YAML 表达方式，详细描述了如何访问 API、支持的操作、输入参数格式、响应结构以及其他元信息。这类文档在自动化测试、客户端代码生成和接口维护等方面具有重要作用。Swagger 是 OpenAPI 早期版本的名称，因此这两个术语在本文中可以互换使用。OpenAPI 的最新版本是 3.0.0，而 Swagger 通常指的是旧版本（3.0.0 之前）。

<pre> swagger: 2.0 info: description: Blog Swagger Documentation version: 1.0 title: Blog host: blog-nginx:80 basePath: / paths: post: operationId: createBlogUsingPOST parameters: - in: body name: blog description: body of the blog creation request required: true schema: \$ref: '#/definitions/BlogCreateReqBody' responses: '200': description: OK schema: \$ref: '#/definitions/Result«Blog»' </pre>	<pre> ... definitions: BlogCreateReqBody: type: object properties: content: type: string description: the content of the blog to be created topics: type: string description: the topics related to the blog userId: type: integer format: int32 description: the id of the user who is creating a blog title: BlogCreateReqBody description: the body of blog create request required: - content - userId ... </pre>
---	---

图 2-2 REST 风格的博客服务的 Swagger 文档

图 2-2 展示了之前提到的博客服务的 Swagger 规格说明文档的示例。图左上方展示了文档的全局配置信息，通过字段如 `swagger`、`info`、`host` 和 `basePath` 指定了文档所使用的规范版本、API 版本、服务器主机名以及服务的基础路径等关键信息。图左下方的 `paths` 字段列出了该服务所支持的所有资源路径及其对应的 HTTP 操作（如 GET、POST、PUT、DELETE 等），每一个路径下都定义了一个或多个具体的端点（`endpoint`）。以图中的博客创建接口为例，其定义包括唯一标识符 `operationId`、请求参数格式 `parameters`（如参数类型、是否必填、传参位置等），以及响应结构 `responses`，包括状态码、返回内容类型及响应体结构等。图右侧则展示了与该服务接口配套的数据模型（`schema`）定义。以示例中的 `BlogCreateReqBody` 为例，该模型结构清晰地列出了请求体的字段类型、字段名称、属性说明、是否必填等内容。

本文实现的基于历史日志的 REST API 模糊测试系统将对 Swagger 或者 OpenAPI 文档进行解析，从中提取接口的基本信息，包括请求路径、HTTP 方法、参

数类型、参数位置（如 query、path、header、body）及其约束条件等。这些信息为后续模糊测试过程中的具体请求构造以及测试结果验证提供支持。

2.3 模糊测试（Fuzzing）

1990 年，B.P. Miller 等人通过向 90 个程序输入随机字符串进行测试，发现超过 24% 的程序发生了崩溃^[34]。他们将用于生成随机输入的程序命名为“Fuzzing”，这一技术随后逐步演化为今天广泛应用的模糊测试方法。如今，模糊测试已成为验证程序可靠性和检测软件安全缺陷最有效的技术手段之一^[35]，被应用到各种领域当中，例如协议测试^[36-37]、编译器测试^[38]、移动应用 GUI 测试^[39]等。

模糊测试通常遵循一个循环迭代的流程，在该过程中持续执行种子选择^[40]、输入变异、测试运行以及缺陷检测等关键步骤^[41-42]。整个模糊测试系统主要由三个核心组件构成：输入生成器、执行器和缺陷监控器。输入生成器负责为执行器提供大量测试输入，这些输入包括初始种子以及通过筛选和变异生成的测试样本。执行器接收输入后运行目标程序，并在过程中由缺陷监控器实时监控其行为，以识别潜在的执行路径或触发程序异常。与此同时，执行器和监控器还会将收集到的执行信息反馈回输入生成器，用于优化后续种子的选择与变异策略，从而实现测试效率和缺陷发现能力的持续提升。

本文提出的基于历史日志的 REST API 模糊测试系统与现有的大多数 REST API 测试工具相同，采用模糊测试作为核心测试框架。此类工具通过不断生成测试序列，反复向目标 REST API 服务发送请求，以触发异常行为并挖掘潜在的软件缺陷^[43]。为提升测试的有效性，这些工具通常引入响应反馈机制，即根据每轮测试的执行结果动态调整测试策略，引导后续测试用例的生成方向。在 REST API 的测试场景中，模糊测试工具主要通过变异 HTTP 请求序列的调用顺序和参数值，持续构造多样化的测试用例。此外，这些工具会根据服务返回的 HTTP 响应状态码或（在白盒测试场景下）代码覆盖率指标（例如 EvoMaster 所采用的做法）来评估当前测试效果，并据此优化后续的测试生成与变异策略，从而进一步提升测试的覆盖率与缺陷检测能力。

2.4 大语言模型

大语言模型（Large Language Models, LLMs），如基于转换器的生成式预训练模型（Generative Pre-trained Transformer, GPT）系列，是自然语言处理（Natural Language Processing, NLP）领域最前沿的技术之一^[44]。大语言模型本质上是一种人工智能技术，能够理解、解释和生成文本，其工作方式与人类使用语言的方式类似。这些模型通过在大量的文本数据上进行训练，学习到多种语言模式和风格，从而具备了广泛的语言理解能力^[45]。大语言模型的核心原理基于深度学习技术，尤其是转换器（Transformer）架构。转换器架构通过自注意力机制（Self-Attention）和层级编码方式，使模型能够有效处理长距离依赖关系和复杂的语言结构。通过对海量文本进行无监督学习，模型能够捕捉语言中的复杂结构、上下文信息和语义关系，从而生成与输入内容相关且符合语法规则的自然语言输出。由于其强大的生成能力，大语言模型不仅可以完成传统的文本生成任务，如机器翻译、文本摘要和等，还能进行更复杂的任务，如问答和对话生成。

得益于其强大的语言理解与生成能力，大语言模型近年来已被广泛引入到软件工程领域，尤其是在软件测试方面展现出巨大潜力^[46]。例如，它们已被用于图形用户界面（GUI）测试^[47]、网络协议测试^[37]、单元测试生成^[48]等多个方向，这进一步证明了大语言模型具备软件测试领域的知识以及测试用例生成能力。

然而，当前将大语言模型应用于 REST API 测试的研究仍处于起步阶段，相关工作数量有限且多为探索性尝试。2023 年，Kim 等人提出的 RESTGPT 系统^[49]尝试利用大语言模型对 REST API 规格说明文档中的自然语言描述进行理解，以增强模型对接口参数含义的把握，从而提升参数生成的质量。实验表明，RESTGPT 在提升参数生成的准确性方面取得了一定效果，但对于复杂嵌套结构的参数，仍存在理解不完整、生成不准确等问题。

本文实现的基于历史日志的 REST API 模糊测试系统与 RESTGPT 在使用大语言模型的方式上存在本质差异：本文实现的系统并非将大语言模型作为测试用例的直接生成器，而是将其作为整套技术中自然语言处理的辅助工具，服务于日志信息的语义解析与接口依赖关系的识别两个功能。

2.5 本章小结

本章围绕构建基于历史日志的 REST API 模糊测试系统所涉及的核心技术与概念进行了系统梳理。

首先，介绍了 HTTP 协议及 REST 架构风格的基本原理，明确了 REST API 的资源导向设计方式及其通过标准 HTTP 方法进行资源操作的机制；随后，阐述了 REST API 测试流程及 Swagger/OpenAPI 规格说明文档在接口理解与测试生成中的关键作用；接着，详细介绍了模糊测试作为主流测试技术的基本框架、工作机制以及其在 REST API 场景下的实际应用方式；最后，对当前自然语言处理领域的前沿技术——大语言模型进行了深入概述，重点指出其在软件测试中的典型应用场景及其在 REST API 测试中的潜在价值与当前局限。

本章内容为后续章节奠定了技术基础。在此基础上，下一章将介绍本文提出的基于历史日志的 REST API 模糊测试系统的需求分析与概要设计。

第三章 REST API 模糊测试系统的需求分析与概要设计

本章将对基于历史日志的 REST API 模糊测试技术进行需求分析与概要设计，分为三部分：1) 系统整体概述；2) 需求分析；3) 系统概要设计。系统整体概述部分将通过一个系统流程概览图展示系统的总体运行流程并对各个环节进行简要解释；需求分析部分将以涉众分析为基础，明确系统用户主体后对系统的功能性需求和非功能性需求进行分析，最后给出系统的用例图以及用例描述；系统概要设计部分则将围绕“4+1”视图进一步对本系统的架构进行剖析和描述。

3.1 系统整体概述

REST 风格的接口目前广泛应用于构建以资源为中心的互联网 API 服务，因此对这些 REST API 进行充分的测试显得尤为重要。现有的测试方法均依赖于 REST API 服务的规格说明文档来识别接口之间的前后依赖关系并进一步生成测试用例。然而，这些文档通常由人工编写或者机器自动生成，其中存在大量内容错误或信息缺失的问题，影响工具的测试用例生成，最终导致测试效果不佳。

为了提取更加有效的 REST API 信息从而提升测试效果，本文提出并且实现了一种基于历史日志的 REST API 模糊测试技术，图3-1展示了围绕该技术搭建的软件系统的流程概览。基于历史日志的 REST API 模糊测试技术创新性地选择服务端的请求调用日志（或者网关的请求转发日志）作为系统的重要信息来源，利用日志中蕴含真实依赖关系的请求序列构造测试用例，最终基于这些高质量的测试用例搭建一套完整的模糊测试流程对目标服务进行充分且有效的测试。

整个系统可分为规格说明文档解析、历史日志解析、模糊测试三个阶段，每个阶段包含多个环节，其中的部分环节使用大语言模型进行处理，图3-1中的虚线框环节为可选环节。规格说明文档解析阶段，系统会首先从服务的文档中提取接口的必要信息（步骤①），比如接口调用地址、接口参数要求、接口响应结构等；同时，系统根据预先设定的规则识别目标服务可能存在的资源（步骤②）；根

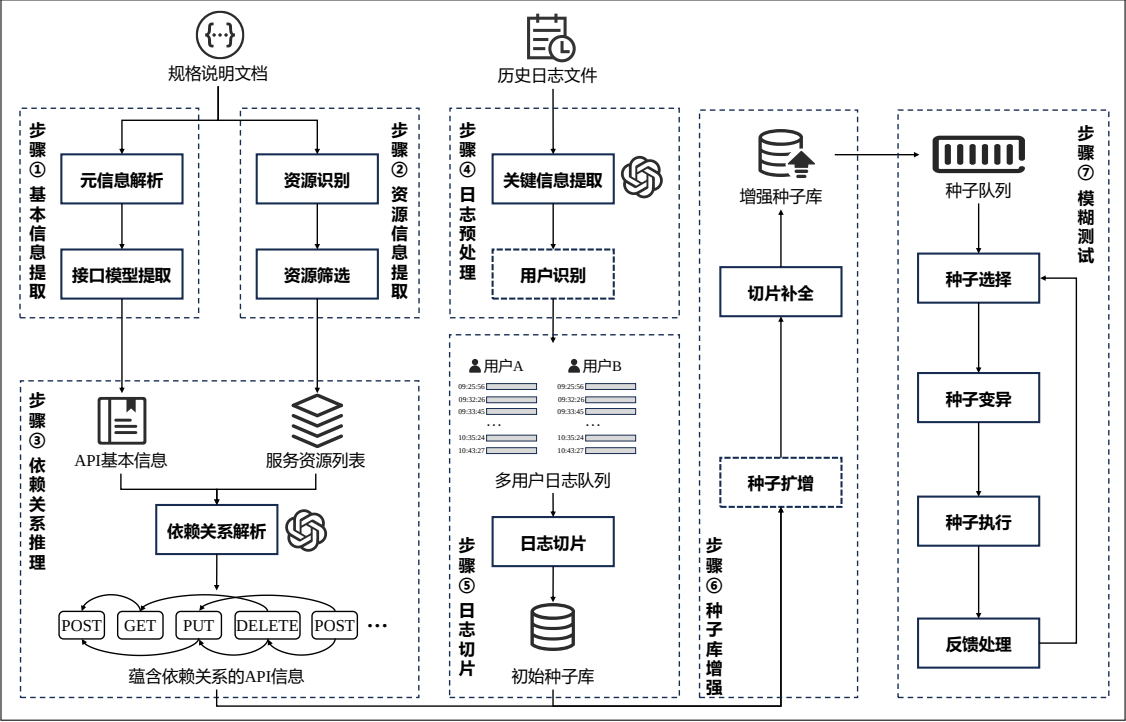


图 3-1 基于历史日志的 REST API 模糊测试系统概览图

据之前获取的 API 基本信息和服务资源列表，系统利用大语言模型的推理能力逐一分析某个单一接口可能依赖的前置资源，最终构建用于指导后续测试用例生成的蕴含依赖关系的 API 信息（步骤③）。历史日志解析阶段，系统首先利用大语言模型的文本理解能力将不定格式的日志文件统一规格并提取其中的关键信息（步骤④），包括请求时间、请求 URL、请求参数、HTTP 方法、请求调用者等信息；由于日志中不同用户的请求往往不具有相关性，系统将按照请求调用者将日志划分为多用户日志队列（步骤④）；之后，受时空局部性原理的启发，系统将按照多种策略将多用户日志队列切片并构造初始种子库（步骤⑤）；初始种子库中的种子丰富度较低且往往缺乏前置资源创建请求，因此系统将根据规格说明文档解析阶段获取的依赖关系信息进行种子扩增和切片补全，最终获取增强的种子库用于模糊测试（步骤⑥）。模糊测试阶段（步骤⑦），系统将依照经典的模糊测试流程，从增强的种子库中构造种子队列，在测试执行过程中，不断选择种子队列中的种子进行变异并执行，获取测试执行结果后根据 HTTP 响应码选择是否将新的种子加入种子队列，之后选择队列中的下一个种子重复上述流程。

3.2 需求分析

基于历史日志的 REST API 模糊测试系统能够帮助用户对其软件系统中的 REST API 服务或者软件系统依赖的上游 REST API 服务进行充分的模糊测试以确保其可靠性和安全性。除了该核心功能性需求外，本系统还应具备鲁棒性（系统能够应对各种类型的 REST API 服务）、可靠性（系统生成的测试用例大部分为有效测试用例）等重要的非功能性需求。本节将从涉众分析出发，论述系统的两类主要用户主体以及它们与本软件系统之间的关系；在此基础上，本节将继续对系统的功能性需求和非功能性需求进行分析和阐述；最后，根据以上分析得到的涉众类别和需求列表，汇总得到本系统的系统用例图以及具体的用例描述。

3.2.1 涉众分析

基于历史日志的 REST API 模糊测试系统能够帮助用户对其指定的 REST API 服务进行测试以确保其可靠性。这类用户分为三类：REST API 服务开发者，REST API 服务调用者以及 REST API 测试领域研究人员。表3-1展示了对这三类涉众的分析结果，包括这些涉众的特征以及他们对本软件系统的期望。

表 3-1 涉众分析结果

涉众名称	涉众特征	涉众期望
REST API 服务开发者	REST API 服务开发者负责设计、实现和维护 REST API 服务。这类涉众技术背景深厚，具备较强的后端开发能力，精通待测服务的编程语言和开发框架；同时，这类涉众熟悉 REST 接口设计规范，比如资源的定义、HTTP 方法的选择等。	REST API 服务开发者期望本系统能够为他们的开发过程提供自动化测试支持，减少人工测试的负担；另外，他们希望本系统能够通过提供服务端日志来提升测试效果，在 API 进行变更时能够检测是否出现兼容问题，避免破坏已有功能。
REST API 服务调用者	REST API 服务调用者是使用 API 的前端开发人员、第三方应用开发者或系统内其他微服务开发者。这类涉众是待测 REST API 服务接口的使用者。	REST API 服务调用者期望基于历史日志的 REST API 模糊测试系统能够自动检测待测 API 是否按照文档规范返回正确的数据结构和状态码；同时，他们希望本系统能够在不提供服务端日志的情况下对待测 REST API 服务进行基本的测试。
REST API 测试领域研究人员	REST API 测试领域的研究人员主要是从事 API 测试相关技术研究的学者、测试工程师或企业的测试架构师。这类涉众具备专业的软件测试领域研究素养，通常关注本测试系统的测试方法以及创新点。	REST API 测试领域研究人员期望本系统具有足够的鲁棒性，能够将其运行在各种待测软件系统上；同时为了比较各种测试方法的效果，他们通常期望本系统能够收集生成的测试用例，提供测试执行报告。

对本系统而言，上述三类涉众的不同之处体现在是否掌握服务端请求调用历史日志以及测试关注点两个方面。1) REST API 服务开发者是待测 REST API 服务的拥有者，他们负责设计、实现、更新和维护待测 REST API 服务；相应地，他们拥有完整的服务端接口调用日志或者网关请求转发日志。因此，这类涉众适合使用本系统上传日志从而对自己的 REST API 服务进行充分且有效地测试，尤其是用于验证变更接口是否影响系统中的其他接口的使用；2) REST API 服务的调用者通常是系统内下游微服务的开发者，或者其他第三方软件系统的开发者。这类涉众根据规格说明文档调用响应的接口来在自己的服务中使用 REST API 服务管理的资源，因此他们侧重于使用本系统检测对应 REST API 服务能否按照文档中的要求返回结果以及是否具有稳定性从而不影响自己服务的使用；3) REST API 测试领域研究人员负责 REST API 测试领域的相关技术研究与方法探索，他们通常将本系统的方法以及其他工具的方法应用在不同的待测项目之上，最终对比实验效果，从而验证方法的有效性和发现方法存在的不足之处。

3.2.2 功能性需求分析

通过对学术界和工业界已有的 REST API 自动测试工具的调研与分析，基于历史日志的 REST API 模糊测试系统利用服务端接口调用日志中蕴含的真实依赖信息生成有效的测试用例对待测服务进行模糊测试。因此，本系统应该包含规格说明解析、历史日志解析、种子库增强、模糊测试执行、测试报告生成五个方面的基本功能性需求。这些功能性需求的具体内容见表3-2。

表 3-2 功能性需求列表

需求 ID	需求名称	需求描述
R1	规格说明解析	用户可以上传待测 REST API 服务的规格说明文档用于系统解析，指引后续模糊测试过程中测试用例的生成
R2	历史日志解析	用户可以上传服务端或网关的接口调用日志用于系统解析，处理构建为初始种子库
R3	种子库增强	系统应该可以将初始种子库通过补全和扩增的方式得到增强种子库
R4	模糊测试执行	系统应该可以基于增强后的种子库开展模糊测试
R5	测试报告生成	对于一轮模糊测试，系统应该可以将测试结果总结为报告

本系统模糊测试的开展基于用户提交的待测服务的规格说明文档以及服务端的历史接口调用日志（可选），因此系统首先需要对这两类文本材料进行处理和解析。规格说明解析（R1）和历史日志解析（R2）均为基于历史日志的 REST API 模糊测试系统的两个重要的用户需求。用户提交待测服务的规格说明文档，系统将提取其中的基本信息（例如服务基址、接口参数类型、接口响应结构等）和资源信息（REST API 服务管理的资源列表），之后借助大语言模型的推理能力推导出各个端点之间的资源依赖信息；同时，用户还能够附加各种格式的历史日志记录来增强后续的测试阶段，系统将提取日志中的关键信息（例如时间戳、请求 URI、HTTP 方法、请求参数等）并通过切片的方式构造出初始种子库。

种子库增强（R3）是基于历史日志的 REST API 模糊测试系统的一个重要系统级需求，它要求系统能够通过种子补全和种子扩增的方式优化之前获取到的初始种子库，从而提升模糊测试的执行效果。由于用户创建资源的操作往往大幅度提前于对该资源的查询、修改、删除操作（比如用户昨天新建了博客 A，今天又对博客 A 进行了修改），因此无论通过哪种方式得到的这类种子切片都是残缺的，需要通过之前分析得到的资源依赖关系补全种子中缺少的资源创建操作。另一方面，日志中记录的请求总是片面的，不能覆盖待测服务的所有接口，因此除了通过切片、补全获取到的种子，还需要基于规格说明文档中记录的所有接口信息生成相关的种子，从而扩增初始种子库，得到涉及所有接口的增强种子库。

模糊测试执行（R4）是基于历史日志的 REST API 模糊测试系统的核心系统级需求。首先，系统会将之前得到的增强种子库中的每个种子执行一次，排除无效种子，保留有效种子，得到初始种子队列。之后，系统循环遍历种子队列中的种子，按照相应的策略对其执行变异操作，生成多个具体的测试用例并执行，根据反馈选择是否将该测试用例作为新的种子添加到种子队列当中。除非用户手动停止或者设置运行时间，否则该模糊测试流程将无休止地运行下去。

测试报告生成（R5）是基于历史日志的 REST API 模糊测试系统的另一重要系统级需求。系统在模糊测试过程中会记录各种运行信息，包括已执行测试用例个数、测试用例平均长度、发现错误个数、发现错误信息等。用户通过开关可以选择是否将这些信息转化为一份测试执行报告帮助自己进行后续决策。

3.2.3 非功能性需求分析

除了上一小节中提到的功能性需求外，非功能性需求也是软件系统设计过程中需要重点考虑的部分。非功能需求由性能需求、质量属性、约束三大部分组成。表3-3列出了基于历史日志的 REST API 模糊测试系统的重要非功能性需求。

表 3-3 非功能性需求

需求 ID	需求类型	需求描述
PR1	性能需求	系统应该支持每分钟发送至少 100 个 HTTP 请求
PR2	性能需求	系统应该能在 3s 内完成对日志中一条记录的解析
PR3	性能需求	系统应该能在 3s 内完成对一个接口的资源依赖分析
PR4	性能需求	系统应该能够将包含 1000 个接口以内的规格说明文档解析到内存当中
QA1	可用性	系统的核心功能，例如模糊测试执行，能够达到 98% 的可用性
QA2	可扩展性	系统应该允许用户扩展代码处理不同格式的日志
QA3	鲁棒性	系统应该能够正常解析 REST API 服务的各种版本的规格说明文档
QA4	可维护性	如果系统要添加新的变异算子，能够在 1 人月内完成
QA5	易用性	在文档的指导下，系统使用者能够在半天内运行系统对指定服务进行测试
CS1	约束	系统使用 Python 语言进行开发

性能需求定义了系统在响应时间、吞吐量、容量和并发能力等方面的要求。针对基于历史日志的 REST API 模糊测试系统，其在发送请求、解析接口依赖关系以及处理日志等环节对速度具有较高要求。在容量方面，系统应具备将包含最多 1000 个接口的规格说明文档加载至内存的能力。

基于历史日志的 REST API 模糊测试系统需满足可用性、可扩展性、可维护性、鲁棒性和易用性五个关键的质量属性。作为系统的核心功能，模糊测试的可用性应达到 98% 以上。同时，系统应具备良好的可维护性和可扩展性，使用户能够在不修改源码的情况下扩展代码以处理不同格式的日志文件，并允许开发者在限定时间内添加新的变异算子。鲁棒性方面，系统需要支持不同版本的规格说明文档，如 Swagger 文档或 OpenAPI 文档。易用性方面，用户应能在较短时间内完成系统部署并使用本系统对待测服务开展模糊测试。

考虑到跨平台移植性和开发效率，本系统应该使用 Python 语言进行开发。

3.2.4 系统用例图

基于3.2.1小节的涉众分析以及3.2.2小节的功能性需求分析，本小节对相关描述内容进行了实体识别、用例拆分与用例筛选，最终构建了基于历史日志的REST API 模糊测试系统的系统用例图，如图3-2所示。

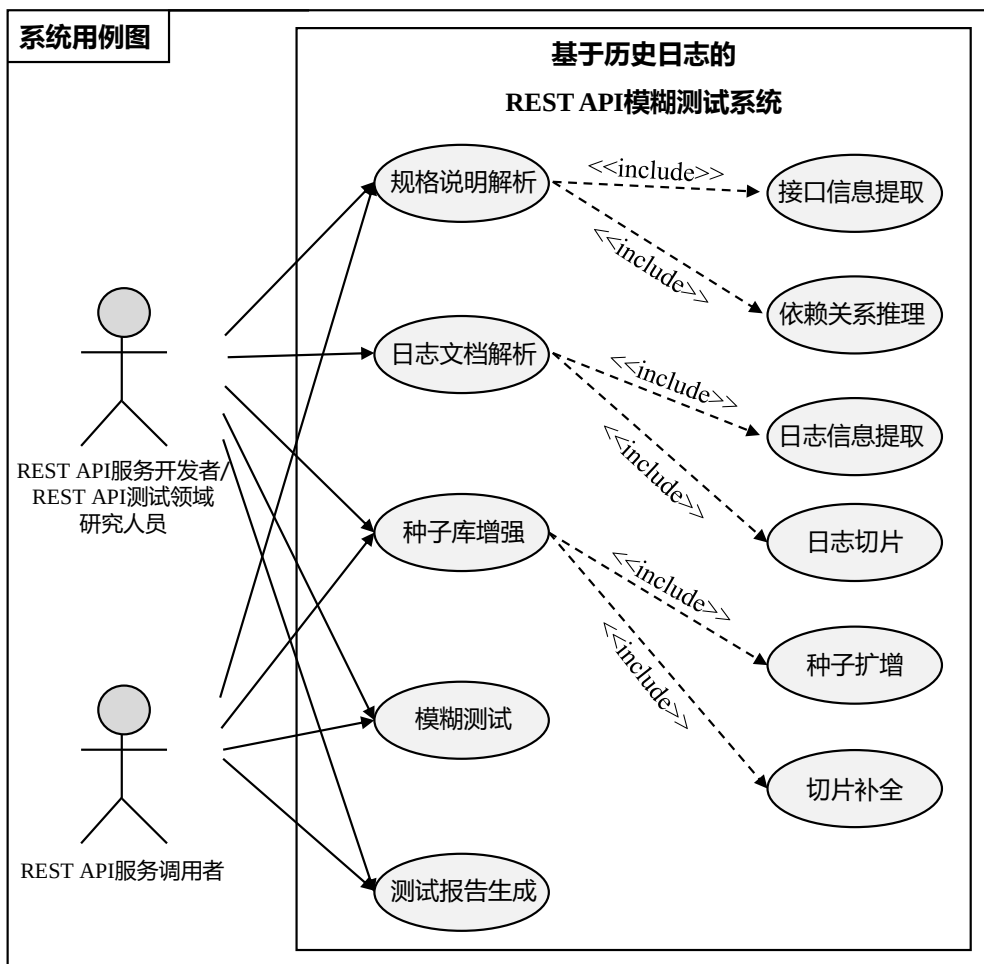


图 3-2 基于历史日志的 REST API 模糊测试系统用例图

基于历史日志的 REST API 模糊测试系统包括规格说明解析、日志文档解析、种子库增强、模糊测试、测试报告生成 5 个用例，其中规格说明解析可细分为接口信息提取、依赖关系推理 2 个用例，日志文档解析可细分为日志信息提取、日志切片 2 个用例，种子库增强可细分为种子扩增、切片补全 2 个用例。

REST API 服务开发者与 REST API 测试领域研究人员两类涉众涉及到的系统中的用例是一致的，包含 5 个用例：规格说明解析、日志文档解析、种子库增强、模糊测试和测试报告生成。REST API 服务调用者与前两类涉众不同，他们与日志文档解析用例无关，因为这类涉众通常无法获取到服务端的日志文件。

3.2.5 系统用例描述

3.2.4小节中的系统用例图从用户的视角展示了基于历史日志的 REST API 模糊测试系统的行为与用户需求之间的关系。本小节则将详细描述每个系统用例的内容细节，帮助开发人员理解系统功能，指导后续的系统设计与开发工作。

接口信息提取是基于历史日志的 REST API 模糊测试系统获取接口基本信息用于构造具体测试用例的基础操作。在用户上传待测目标服务的规格说明文档后，系统将解析其中的内容，提取三类关键信息：1) 服务元信息，包括服务的地址、服务的版本、服务的调用方式等；2) 接口基本信息，包括接口 URI、接口 HTTP 方法、接口参数等；3) 资源信息，即待测 REST API 服务负责管理的资源列表。表3-4展示了用例“接口信息提取”的详细描述。

表 3-4 用例“接口信息提取”详细描述

用例编号	UC1
用例名称	接口信息提取
优先级	高
参与者	所有三类涉众
触发条件	用户在系统中上传待测服务的规格说明文档
前置条件	用户提交规格说明文档
后置条件	系统将服务中接口的基本信息解析到内存中
正常流程	1. 用户上传规格说明文档 2. 系统检测文档格式内容是否符合要求 3. 系统提取文档中有关待测服务的元信息 4. 系统提取文档中有关待测服务的接口信息 5. 系统提取文档中待测服务可能包含的资源列表
扩展流程	格式或内容不正确时，提醒用户问题所在
特殊需求	系统能够处理 Swagger 和 OpenAPI 两种不同版本的规格说明文档，同时支持 JSON 和 YAML 两种结构化文档格式

依赖关系推理是基于历史日志的 REST API 模糊测试系统生成高质量测试用例的关键。经过接口信息提取，系统获取到接口的基本信息和服务资源列表；接下来，系统将利用这两类重要数据，同时借助大语言模型的推理能力，推导出某个特定接口的参数可能需要依赖的前置资源，最终得到蕴含依赖关系的接口信息。表3-5展示了用例“依赖关系推理”的详细描述。

日志信息提取是基于历史日志的 REST API 模糊测试系统利用日志中的信息进行模糊测试的基础步骤。对于 REST API 服务调用者，这类涉众通常接触不

表 3-5 用例“依赖关系推理”详细描述

用例编号	UC2
用例名称	依赖关系推理
优先级	高
参与者	所有三类涉众
触发条件	系统完成对规格说明文档的接口信息提取
前置条件	系统获取到接口基本信息和服务资源列表
后置条件	系统得到蕴含依赖关系的接口信息
正常流程	1. 系统得到接口基本信息和服务资源列表 2. 系统检测格式是否符合要求 3. 系统依次选取接口 4. 系统遍历接口中的参数 5. 系统利用大模型分析参数可能依赖的服务资源
扩展流程	如果待测服务的某个接口格式不符合要求，直接跳过
特殊需求	系统能够应对大语言模型接口不稳定的情况

到服务端的接口调用日志，因此不属于该用例的参与者。在用户上传历史日志后，系统将逐行解析日志中的每条记录，从中提取关键信息，比如记录时间戳、请求 URI、请求 HTTP 方法、请求参数等。这些信息有助于系统搭建逻辑日志序列，从而提取其中蕴含的高质量测试用例。表3-6展示了本用例的详细描述。

表 3-6 用例“日志信息提取”详细描述

用例编号	UC3
用例名称	日志信息提取
优先级	高
参与者	REST API 服务开发者/REST API 测试领域研究人员
触发条件	用户在系统中上传历史日志文件
前置条件	1. 用户提交历史日志文件 2. 用户针对自己的日志格式扩展了解析日志的方法
后置条件	系统得到逻辑日志序列
正常流程	1. 用户上传历史日志文件 2. 系统检测格式内容是否符合要求 3. 系统依次提取每条日志记录中的关键信息
扩展流程	如果日志文件的某条记录缺失关键信息，直接跳过
特殊需求	系统能够允许用户扩展从而处理各种格式的日志文件

日志切片是基于历史日志的 REST API 模糊测试系统利用日志构建高质量测试用例的关键步骤。同日志信息提取用例一致，REST API 服务调用者可能不属于本用例的参与者。如果日志中包含用户信息，系统将会首先按照用户将之前

得到的逻辑日志序列拆分为多用户日志队列。之后，系统将会根据用户设定的策略将同一用户日志队列切分成多个日志切片，每个切片均包含多个相关的接口调用记录。最终，系统将这些切片汇总为用于模糊测试的初始种子库。表3-7展示了用例“日志切片”的详细描述。

表 3-7 用例“日志切片”详细描述

用例编号	UC4
用例名称	日志切片
优先级	高
参与者	REST API 服务开发者/REST API 测试领域研究人员
触发条件	系统完成日志信息提取
前置条件	系统根据日志文件获取逻辑日志序列
后置条件	系统得到初始种子库
正常流程	1. 系统获得逻辑日志序列 2. 系统按照用户将日志转化为多用户日志队列 3. 系统按照策略对多用户日志队列进行切片并汇总为初始种子库
扩展流程	如果日志队列不包含用户信息，则多用户日志队列只包含一个队列
特殊需求	用户可以在系统中选择不同的日志切片策略

种子扩增是基于历史日志的 REST API 模糊测试系统提高测试接口覆盖率的重要步骤。通过对日志进行解析、切片得到的初始种子库包含的接口类型是不丰富的，通常包含的是用户使用频率较高的接口。然而，REST API 测试的目标是尽可能多地覆盖服务所有的接口；因此，需要按照规格说明文档制造并添加新的种子。表3-8展示了用例“种子扩增”的详细描述。

表 3-8 用例“种子扩增”详细描述

用例编号	UC5
用例名称	种子扩增
优先级	高
参与者	所有三类涉众
触发条件	初始种子库构造完成
前置条件	系统获取初始种子库
后置条件	系统获取扩增后的种子库
正常流程	1. 系统获得初始种子库 2. 系统遍历服务所有接口制造新种子并补充至种子库中
扩展流程	接口格式或内容不正确时，跳过该接口
特殊需求	用户可以在系统中选择是否扩增，以及是否“重复扩增”

切片补全是基于历史日志的 REST API 模糊测试系统生成有效测试用例的关键步骤。通常来说，用户创建一个资源之后，会间隔一段时间再次对该资源进行操作；这种情况下，对日志进行切片获取到的种子是不完整的，缺少关键的资源创建请求。切片补全利用之前依赖关系推理获取到的依赖关系，合理地将资源创建请求补充到种子前面，得到有效的测试用例用于模糊测试。表3-9展示的是用例“切片补全”的详细描述。

表 3-9 用例“切片补全”详细描述

用例编号	UC6
用例名称	切片补全
优先级	高
参与者	所有三类涉众
触发条件	初始种子库构造完成
前置条件	系统获取初始种子库
后置条件	系统获取增强后的种子库用于模糊测试
正常流程	1. 系统获得初始种子库 2. 系统遍历种子库的种子 3. 判断种子是否缺少前置资源创建请求 4. 根据依赖关系补全种子

模糊测试是基于历史日志的 REST API 模糊测试系统的采用的测试方法。经过之前的各种环节，系统得到了全面覆盖待测服务接口的种子库，并将基于此开展模糊测试。表3-10展示了用例“模糊测试”的详细描述。

表 3-10 用例“模糊测试”详细描述

用例编号	UC7
用例名称	模糊测试
优先级	高
参与者	所有三类涉众
触发条件	增强种子库构造完成
前置条件	系统获取增强种子库
后置条件	系统完成测试
正常流程	1. 系统获得增强种子库 2. 系统执行种子库筛选，获取初始种子队列 3. 系统循环遍历种子队列中的种子，按照策略执行变异操作 4. 系统执行测试用例，获取响应结果 5. 系统根据响应结果判断是否将该测试用例添加至种子队列当中
特殊需求	用户可以手动暂停模糊测试流程

测试报告生成是基于历史日志的 REST API 模糊测试系统的一个重要用例。在测试执行完成后，系统应该能够根据用户要求生成响应的测试报告对测试结果进行总结。表3-11展示了用例“测试报告生成”的详细描述。

表 3-11 用例“测试报告生成”详细描述

用例编号	UC8
用例名称	测试报告生成
优先级	高
参与者	所有三类涉众
触发条件	模糊测试执行完成
前置条件	系统或者用户结束模糊测试
后置条件	用户得到测试报告
正常流程	1. 模糊测试执行完成 2. 系统收集测试执行过程数据并汇总为测试报告 3. 用户查看对应测试任务的测试报告
扩展流程	用户可以选择是否生成测试报告，并且可以自由选择报告中的内容

3.3 系统概要设计

概要设计是软件开发生命周期中的关键阶段，旨在将需求转化为系统的整体架构和模块划分。在3.2节需求分析的基础之上，本节对基于历史日志的 REST API 模糊测试系统进行概要设计。首先，本节围绕系统的功能性需求和用例进行整体架构设计，并展示系统架构图；随后，详细阐述架构图中的各模块的职责及其相互关系；最后，给出系统的“4+1”视图，包括逻辑视图、开发视图、进程视图、物理视图和场景视图。其中场景视图即为本系统的用例图，已在3.2.4小节中的图3-2给出，这里不再赘述。

3.3.1 系统整体架构设计

图3-3展示的是基于历史日志的 REST API 模糊测试系统的整体架构图。本系统基于用户上传的规格说明文档和历史日志对待测服务开展模糊测试，挖掘其中潜在的缺陷，验证服务的可靠性。图3-3按照模块将系统进行功能进行了划分，并且使用箭头标明了不同模块之间的依赖关系。

本系统架构采用自底向上的分层式设计，整体由六个功能模块组成，包括持

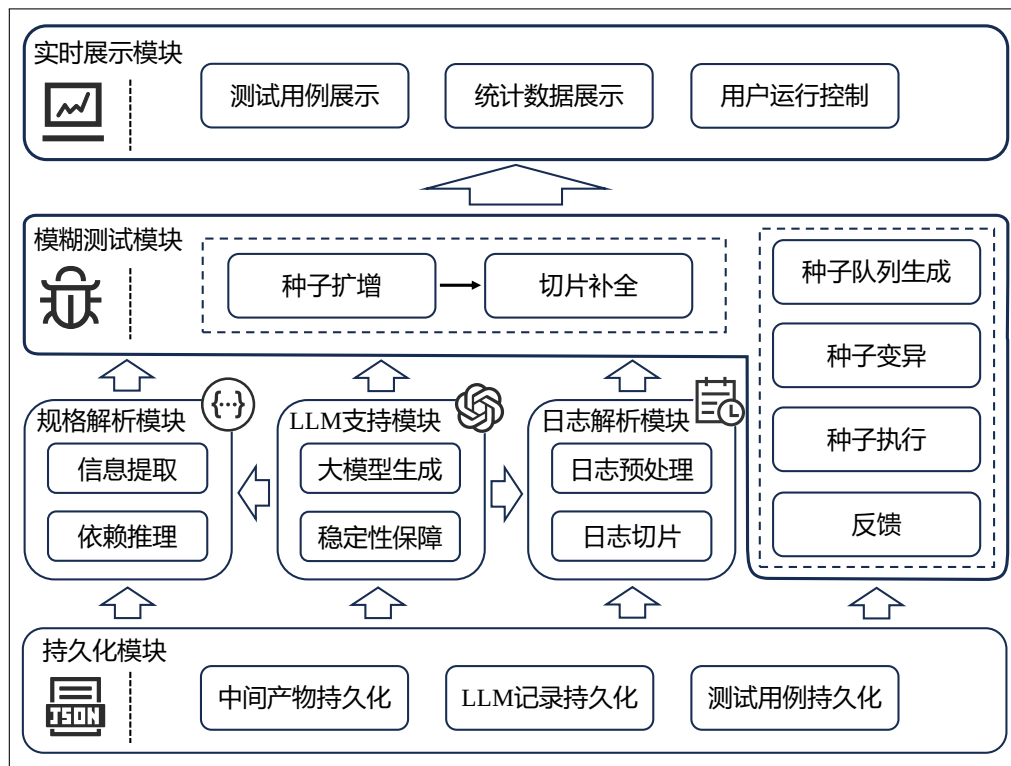


图 3-3 基于历史日志的 REST API 模糊测试系统整体架构图

持久化模块、规格解析模块、LLM 支持模块、日志解析模块、模糊测试模块以及实时展示模块。各模块相互协作，共同完成用户的需求，保障系统的高效运行。

持久化模块负责记录系统运行过程中生成的关键数据，特别是用户关注的测试结果、执行日志及中间状态，并将其以文件形式存储，便于后续分析、复现或共享；规格解析模块旨在解析用户提供的 REST API 规格说明文档，自动提取接口定义、请求参数及返回格式等关键信息，为后续的模糊测试过程提供必要的支持；日志解析模块主要用于分析用户提供的服务端历史日志，提取其中的关键调用信息，并基于这些日志构造出能够反映真实调用关系的测试用例，以提高测试的覆盖度和有效性；模糊测试模块是整个系统的核心模块，该模块首先将初始种子库进行增强，之后在此基础上开展包括种子队列生成、种子变异、种子执行与反馈等多个环节的完整模糊测试，从而最大程度地挖掘潜在漏洞；LLM 支持模块负责大模型调用的相关事务，同时该模块也负责保证与大模型交互的稳定性，避免因调用不稳定或异常响应影响测试流程；最后，实时展示模块负责在运行时展示测试的执行过程，包括当前执行的测试用例、执行结果、统计数据等，同时该模块还负责与用户交互从而控制测试流程。各个模块的划分原则以及详细设计将在下一小节中详细讨论。

3.3.2 系统模块划分

在需求分析的基础上,对系统进行模块化划分可以有效降低其复杂度,使系统结构更加清晰合理,提升后续的开发效率和软件质量。基于历史日志的 REST API 模糊测试系统按照用户的需求点划分为持久化、规格解析、日志解析、LLM 支持、模糊测试、实时展示六个功能模块,下面将对这些模块进行逐一介绍,最后还将对模块组件之间的依赖关系进行解释。

日志解析模块。与其他 REST API 模糊测试系统不同,基于历史日志的 REST API 模糊测试系统选择使用待测 REST API 服务的历史日志来生成模拟真实使用场景的高质量测试用例,该功能由日志解析模块负责。日志解析模块对应于需求分析中的功能性需求 R2 以及系统用例 UC3 和 UC4。本模块首先读取用户上传的各种类型的日志,从中提取日志记录时间戳、请求参数、请求 URI、请求方法等关键信息;接着,为了保证日志中请求的相关性,本模块按照用户将原始日志队列拆分为多个不同的日志队列;最后,模块将按照用户设定的切分策略将日志切分为多个日志切片,构成初始种子库。

规格解析模块。规格解析模块对应于需求分析中的功能性需求 R1 以及系统用例 UC1 和 UC2,负责提取 REST API 规格说明文档中的有效信息,为后续测试用例的生成提供基本的“说明书”。用户上传规格说明文档后,规格解析模块将首先提取其中的接口基本信息和资源信息;之后,基于上述两类信息,该模块利用大模型的推理能力对每一个 API 进行依赖关系推理,用于支持后续的切片补全功能。

模糊测试模块。这一模块承担了本系统的主要职责,即对待测服务进行充分的模糊测试以挖掘其中的缺陷。该模块对应于需求分析中的功能性需求 R3 和 R4 以及系统用例 UC5、UC6 和 UC7。由于日志的不全面性,该模块首先对初始种子库执行种子扩增,使种子库尽可能地覆盖所有的待测接口;另一方面,由于日志切片的不完整性,该模块利用之前获取到的依赖信息对种子进行补全,最终获得增强的种子库。随后,该模块将开展完整的模糊测试过程,即包括种子队列生成、种子选择、种子变异、测试执行、反馈等多个环节。

LLM 支持模块。该模块并不对应于需求分析中的某一个功能性需求或者系统用例,但是它为其他模块的实现提供了重要的技术支持。该模块负责整个系统

中调用大模型进行回复的事务，在系统初始化时会根据用户的设定初始化对应的大模型引擎用于调用；同时，该模块通过提示词模板、输出检测、重试等机制有效地保障了调用大模型时回复的准确性和稳定性。

持久化模块。这一模块负责整个系统的持久化功能，通过将一些重要的信息通过文件的方式存储下来，能够有效地帮助用户进行复用和查看，降低用户的使用成本。该模块对应于需求分析过程中的功能性需求 R5 以及系统用例 UC8。除了测试报告生成外，持久化模块的核心功能是大模型增强信息的持久化。用户只需要执行规格解析和日志解析一次即可将解析过的数据通过 JSON 文件的方式持久化到本地，之后再次执行该服务的测试时借助该 JSON 文件即可启动，无需重新解析规格说明文档和日志文件导致时间浪费和资源消耗。

实时展示模块。这一模块负责展示本系统运行时的测试数据，同时帮助用户在运行时控制测试。运行时的测试数据包括实时的测试信息，比如当前执行的测试用例，当前的种子队列信息，当前采用的变异策略等；另一方面，测试数据还包括运行时的统计信息，例如已经发现的缺陷数量，有效测试用例的个数，REST API 端点覆盖率等。

模块之间的依赖关系。图3-3展示的系统架构是基于需求分析得到的功能性需求和系统用例进行模块化划分的，不同模块之间存在依赖关系，包括数据依赖、功能依赖、状态依赖三类。整体来看，基于历史日志的 REST API 模糊测试系统采用的是分层架构，即模块之间的依赖关系具备层级性和单行性，上层依赖于下层，而下层不会依赖于上层。这样的分层设计符合依赖倒置原则，能够有效降低系统不同组件的耦合度，提升系统的可维护性和可扩展性。

模糊测试模块对规格解析和日志解析模块的依赖关系属于数据依赖，模糊测试的运行需要使用到规格解析模块提供的蕴含依赖信息的接口信息和日志解析模块提供的初始种子库。系统中其他模块对 LLM 支持模块和持久化模块的依赖关系属于功能依赖。LLM 支持模块负责为其他模块提供大模型相关的调用服务，包括大模型生成和稳定性保障；持久化模块则负责提供持久化相关的所有功能，包括中间产物持久化、LLM 记录持久化和测试用例持久化。实时展示模块与模糊测试模块之间的依赖关系属于状态依赖。实时展示模块依赖于模糊测试模块的实时状态数据，通过这些数据的展示帮助用户了解本系统的执行情况。

3.3.3 系统逻辑视图

逻辑视图关注系统的功能性需求，描述系统的主要组件、类、对象以及它们之间的关系，通常使用类图表示，强调模块职责和数据流，用于支持后续代码设计与实现。图3-4为基于历史日志的 REST API 模糊测试系统的逻辑视图。

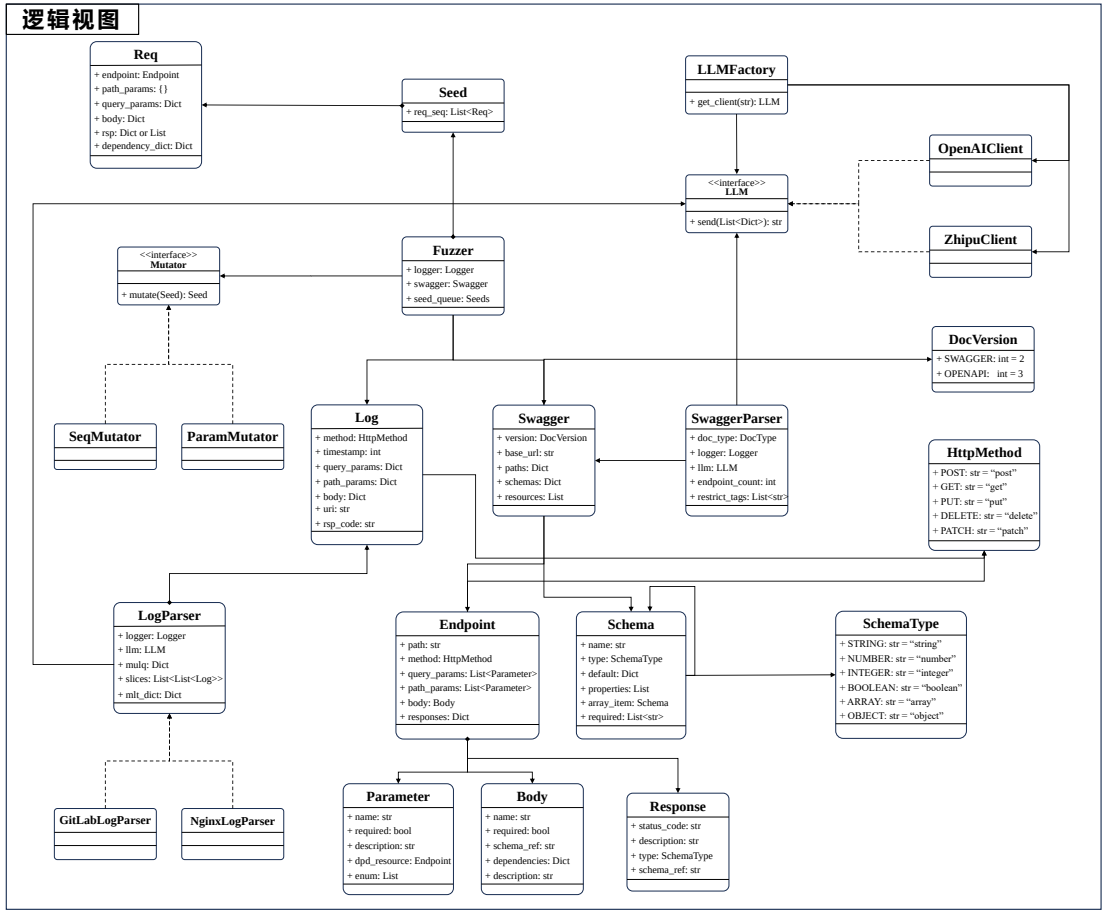


图 3-4 基于历史日志的 REST API 模糊测试系统逻辑视图

上述逻辑视图采用概念类图的形式呈现了本系统的静态概念架构，展示了核心类及其之间的依赖关系。接下来，将按照先前划分的功能模块，分别介绍各模块所包含的类及其设计。由于概念类图主要关注系统的核心功能，因此其中仅涵盖了规格解析、日志解析、模糊测试和 LLM 支持四个核心功能模块。

规格解析模块的核心由多个关键类组成，主要包括逻辑视图右下角的 SwaggerParser、Swagger、Endpoint、Schema、Parameter、Body、Response 等。其中 SwaggerParser 类提供了一系列用于解析规格说明文档的方法，例如解析文档版本、解析服务基址、解析端点等。这些方法最终协同作用，构造出一个 Swagger 对象。Swagger 对象存储了待测 REST API 服务

的各种基本信息，包括端点信息（Endpoint 类）、参数信息（Parameter 类用于存储查询参数和路径参数；Body 类用于存储请求体中的参数）、响应信息（Response 类）、结构体信息（Schema 类）等。这些解析得到的信息将为后续的具体请求生成提供指导。值得注意的是，Schema 类采用组合模式设计，以便有效处理结构体的嵌套情况，从而增强系统对复杂数据结构的适应能力。

日志解析模块包括 Log、LogParser、GitLabLogParser、NginxLogParser 四个核心类。同样地，LogParser 类提供一系列用于解析日志的方法，最终将用户上传的日志文件转化为初始种子库存储在内存当中。由于日志的类型和格式众多，为了提高本系统鲁棒性，本模块采用模板方法模式，将 LogParser 基类中的 `_parse_one_log()` 方法设为抽象方法，用户可以自己扩展不同的子类来处理自己的日志。本系统默认已经支持 GitLab 和 Nginx 网关请求转发两种不同类型的日志。

模糊测试模块包括 Fuzzer、Seed、Req、Mutator、SeqMutator、ParamMutator 等多个核心类或接口。Fuzzer 类负责启动和掌控整个模糊测试流程，但在模糊测试开始之前，它还会执行一些数据转换和种子库增强的预处理操作。预处理操作结束后，模糊测试模块获得增强种子库，该库中存储了众多 Seed 对象，而每一个 Seed 对象有是一个 Req 对象列表，代表了一个测试用例。为了便于扩展，本系统的变异算子采用策略模式设计，Mutator 接口定义了变异相关的操作，存在多个子类实现这些接口，进而执行不同的变异操作，比如 SeqMutator 负责执行测试序列的变异，而 ParamMutator 负责执行参数的变异，用户也可以设置自己的变异算子。

LLM 支持模块负责为其他模块提供大模型相关的功能支持，比如大模型问答等。该模块包括 LLM、LLMFactory、ZhipuClient、OpenAIClient 等多个核心类或接口。该模块采用简单工厂的设计模式，通过 LLMFactory 统一管理不同大模型客户端的创建和配置，以提高系统的灵活性和可扩展性。LLMFactory 根据用户设定或系统需求，实例化相应的大语言模型客户端，如 ZhipuClient（用于对接智谱 AI 模型）或 OpenAIClient（用于调用 OpenAI 的 GPT 系列模型）。这些客户端封装了与大模型的交互逻辑，包括请求构造、响应解析、错误处理以及重试机制等，以确保模型调用的稳定性和高效性。

3.3.4 系统开发视图

开发视图关注软件系统的模块化结构和代码组织方式，描述代码如何被分解为模块、组件和包，以及它们之间的依赖关系。图3-5展示了基于历史日志的 REST API 模糊测试系统的开发视图。

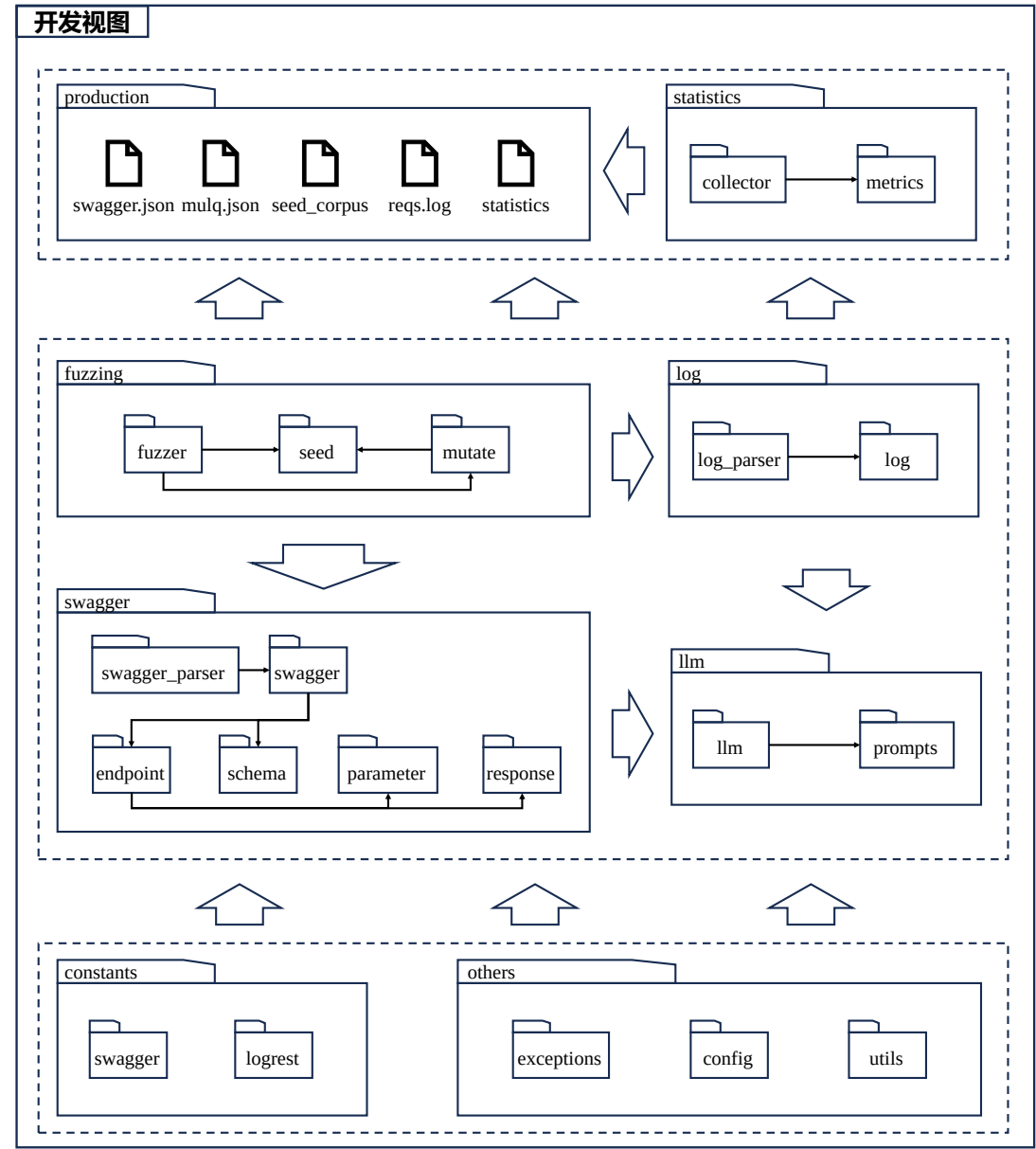


图 3-5 基于历史日志的 REST API 模糊测试系统开发视图

上述开发视图采用包图的形式展示了本软件系统的代码组织结构，面向开发人员设计，旨在确保后续开发过程中具备清晰的架构和良好的管理。该开发视图同样遵循分层设计原则，自底向上划分为三个主要模块：共同依赖模块、运行时模块和状态统计模块，各模块相互协作，以提升系统的可维护性和扩展性。

共同依赖模块包含整个系统所依赖的公共资源或者配置，分为常量和和其他两部分。常量部分有 `swagger` 和 `logrest` 两个包，用于存储系统可能用到的常量和枚举值。其中，`swagger` 包存储 REST API 规格说明文档相关的值，例如文档的版本 `DocVersion`、请求的方法 `HttpMethod` 等；`logrest` 包存储本系统特色的值，例如系统支持的文档格式 `DocType`、系统支持的大模型 `SupportedLLM`、系统支持的时间格式 `TimeFormat` 等。其他部分则包括 `exceptions`、`config`、`utils` 三个包。`exceptions` 包记录系统定义的异常情况；`config` 包存储系统的配置，比如日志级别、大模型引擎、切片的最大前置时间等；`utils` 包则定义了各种工具函数，比如各种时间相关、文件相关的函数。

运行时模块包含系统在运行过程中执行用户需求所涉及的核心代码，整体可分为模糊测试执行、规格文档处理、日志处理、大模型调用四个部分。规格文档处理（`swagger`）包含 `swagger_parser`、`swagger`、`endpoint`、`parameter` 等包，负责解析用户提交的规格说明文档，并提取关键信息，如端点定义、请求参数、响应结构等。其依赖关系与逻辑视图中的规格解析模块类似，为模糊测试提供必要的输入数据。日志处理（`log`）包括 `log_parser` 和 `log` 两个包，负责解析用户提交的服务端日志，以支持测试用例的生成与优化。模糊测试执行（`fuzzing`）由 `fuzzer`、`seed`、`mutate` 三个包组成，分别负责模糊测试的整体流程控制、种子输入的格式管理以及测试数据的变异处理。大模型调用（`llm`）包括 `llm` 和 `prompts` 两个包。其中，`llm` 包定义了 LLM 接口并内置了系统支持的两类大模型客户端（`OpenAI` 和 `Zhipu`），用于处理自然语言交互任务；`prompts` 包则存储了系统中用于不同任务的提示词模板。

状态统计模块则包含系统的统计组件和系统的运行产物，分为统计和产物两部分。统计组件（`statistics`）包括 `collector` 和 `metrics` 两个包。`collector` 用于收集测试过程中产生的数据，如执行时间、通过/失败数量等；`metrics` 包定义各种统计指标，如平均执行时间、失败率、标准差等。产物组件（`production`）则包括了系统运行过程中的各种产物，比如系统解析后的接口信息 `swagger.json`、系统获取到的多用户日志队列 `mulq.json`、系统生成的种子库 `seed_corpus`、系统发送请求的日志 `reqs.log` 等等。

3.3.5 系统进程视图

进程视图关注系统运行时的行为，描述系统的并发性、进程间通信以及同步机制，帮助开发人员确定软件系统的运行流程。图3-6展示了基于历史日志的 REST API 模糊测试系统的进程视图。

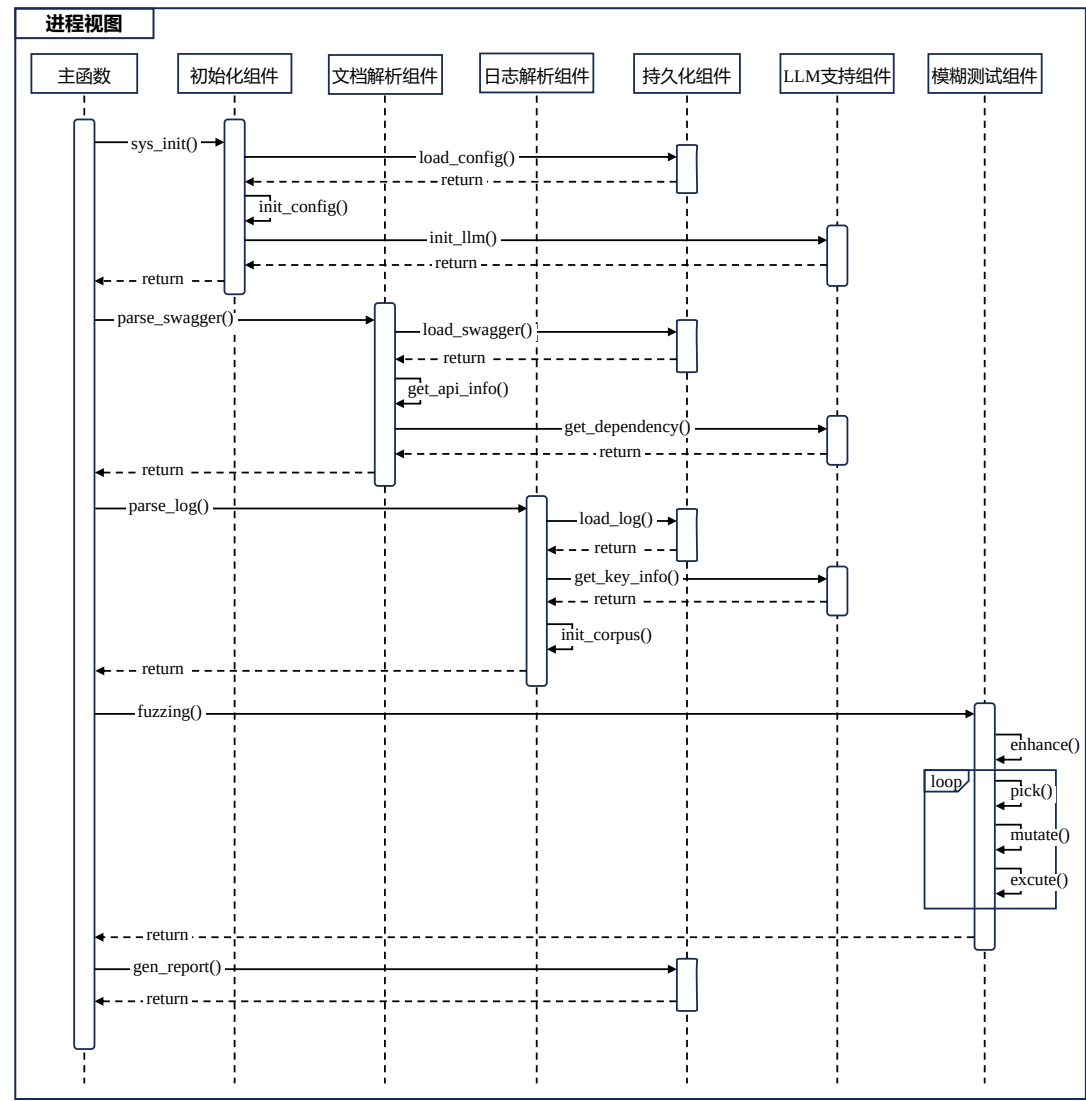


图 3-6 基于历史日志的 REST API 模糊测试系统进程视图

上述进程视图采用顺序图的形式展示了本软件系统的运行流程及进程间的交互细节，重点强调事件触发和数据流转，以确保系统设计符合用户需求。该顺序图清晰地描述了主函数如何与各个组件交互，执行一系列任务，并最终完成对待测服务的模糊测试。除了主函数之外，系统的核心功能由多个关键组件协同完成，包括初始化组件、文档解析组件、日志解析组件、持久化组件、LLM 支持组件以及模糊测试组件。

主函数首先调用 `sys_init()` 函数进行本系统的初始化。初始化组件主要负责配置初始化和大模型引擎初始化两个任务。配置初始化需要持久化组件加载用户的 `config.yml` 文件，之后将其中的信息读取到内存中；大模型引擎初始化则将读取用户的配置，初始化指定的大模型客户端，比如 `OpenAI` 或者 `ZhiPu` 等各种类型的大模型。

系统初始化完成后，主函数将调用 `parse_swagger()` 函数对规格说明文档进行解析，这将由文档解析组件完成。文档解析组件先通过持久化组件加载用户上传的规格说明文档；之后，调用自己的 `get_api_info()` 函数从中提取基本的接口信息；最后，在 `LLM` 支持组件的协助下，完成对待测服务接口依赖信息的推理，将得到的蕴含依赖关系的接口信息返回给主函数。

紧接着，主函数调用 `parse_log()` 函数，对用户上传的服务端日志进行解析，该过程由日志解析组件负责执行。日志解析组件同样依赖持久化组件来加载用户提交的服务端日志文件，确保日志数据可供后续处理；加载完成后，调用 `get_key_info()` 函数，从日志文件中提取日志的关键信息，如日志记录时间、请求参数、请求地址等；随后，调用 `init_corpus()` 函数，将日志数据从逻辑日志队列转化为多用户日志队列，并按照指定策略（如请求时间粘连、时间窗口等）对日志进行切片，以便获取高质量的初始种子用于模糊测试执行；最终，日志解析组件将这些切片构成的初始种子库返回给主函数。

在规格解析和日志解析完成后，主函数将调用模糊测试组件的 `fuzzing()` 函数，正式启动模糊测试流程。在测试执行前，模糊测试组件首先调用 `enhance()` 函数，对初始种子库进行扩增和补全，以提升种子的质量和覆盖度，确保后续测试的有效性和全面性。基于优化后的种子库，模糊测试组件进入正式的模糊测试循环，包括种子选择 (`pick()`)、种子变异 (`mutate()`)、测试用例执行 (`execute()`)、反馈 `feedback()` 等环节。最终，当用户手动停止测试或者测试运行时间达到预设阈值时，模糊测试组件将终止测试流程，并对测试数据进行统计，将结果返回给主函数。

模糊测试结束后，主函数调用持久化组件的 `gen_report()` 函数生成测试执行报告。测试执行报告包含测试过程中收集的关键数据，比如已执行的测试用例、对应的输入参数、API 响应状态、错误日志等。

3.3.6 系统物理视图

物理视图关注系统的部署架构，描述软件如何映射到硬件资源（服务器、网络、云 API、存储等），面向系统管理员或者运维人员。图3-7展示了基于历史日志的 REST API 模糊测试系统的物理视图。

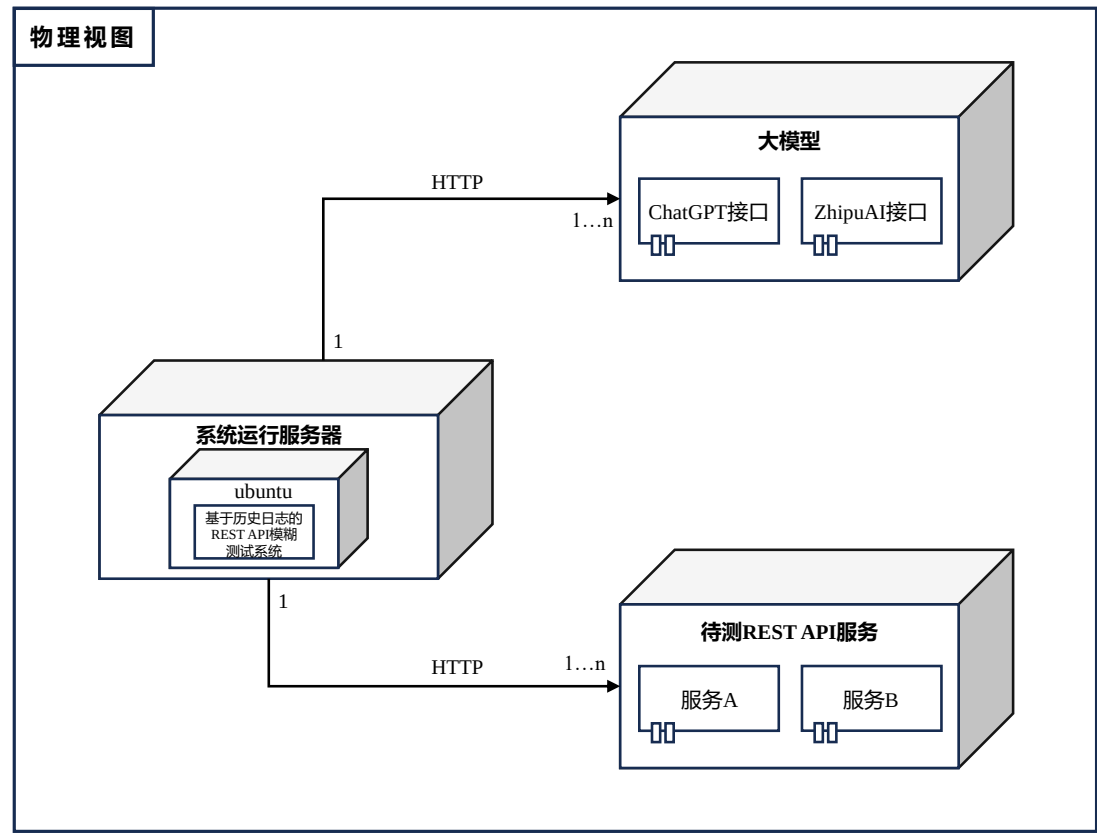


图 3-7 基于历史日志的 REST API 模糊测试系统物理视图

上述物理视图采用 UML 部署图的形式展示了本软件系统的部署结构，强调系统的服务器节点和通信协议，用于确保系统可以在实际运行环境中正常运行。该物理视图展示了系统的三大物理核心组件及其交互方式，包括系统运行服务器、大模型服务和待测 REST API 服务。在系统部署中，系统运行服务器承载了基于历史日志的 REST API 模糊测试系统，并运行在 Ubuntu 环境上。该服务器通过 HTTP 协议与外部服务进行交互。一方面，它与大模型服务（如 ChatGPT 和 ZhipuAI）建立连接，以获取 LLM 支持，用于接口依赖推理和日志关键信息提取。另一方面，它通过 HTTP 连接到待测 REST API 服务，其中包含多个具体服务实例，用于执行模糊测试并收集测试结果。

3.4 本章小结

本章主要介绍了基于历史日志的 REST API 模糊测试系统的需求分析结果和系统概要设计，用于指导后续的系统详细设计和软件开发过程。

首先，本章节给出了基于历史日志的 REST API 模糊测试系统的整体概述。通过一个完整的系统概览图，阐述了系统的三大运行阶段（规格说明文档解析、历史日志解析、模糊测试）以及各个阶段的重要步骤（基本信息提取、资源信息提取、依赖关系推理、日志预处理、日志切片、种子库增强、模糊测试）。

然后，本章节开始对基于历史日志的 REST API 模糊测试系统进行需求分析，包括涉众分析、功能性/非功能性需求分析、系统用例图及用例描述。涉众分析环节，本章节总结了系统的三大类涉众，即 REST API 服务开发者、服务调用者和测试领域研究人员；功能性/非功能性需求分析环节，本章节总结并解释了系统的 5 个基本功能性需求（规格说明解析、历史日志解析、种子库增强、模糊测试执行、测试报告生成）以及众多非功能性需求；用例分析环节，本章首先基于之前分析的涉众类型和功能性需求，得到了系统用例图，之后对用例图中的各个用例进行了详细描述。

最后，在用例分析的基础上，本章节完成了对基于历史日志的 REST API 模糊测试系统的概要设计。具体来说，本章首先对系统进行了模块划分，并解释划分的原则和每个模块的职责；之后，使用“4+1 视图”从各个维度描述了系统的架构设计。其中，逻辑视图定义了系统的主要功能类及其依赖关系；开发视图从代码组织的角度，阐述了本系统的包结构及开发过程中的管理方式；进程视图展示了系统在运行时的主要流程，包括各模块的调用顺序和数据流转；物理视图详细说明了系统的部署结构，明确了系统服务器、待测 REST API 服务以及大模型服务之间的通信方式。

第四章 REST API 模糊测试系统的详细设计与实现

基于第三章的“需求分析与概要设计”，本章将对基于历史日志的 REST API 模糊测试系统的核心模块进行详细设计分析与具体实现展示，涵盖规格解析模块、日志解析模块和模糊测试模块，深入剖析各模块的内部结构和实现机制。具体而言，每节将首先借助类图和顺序图阐述该功能模块的详细设计；之后，在此基础上，结合关键代码片段，进一步展示核心功能的具体实现，解析关键逻辑、算法设计和优化策略。

4.1 规格解析模块详细设计与实现

规格解析模块是本系统的基础模块，其主要职责是解析用户提交的规格说明文档（如 Swagger 文档或者 OpenAPI 文档），提取其中的接口信息和依赖关系，为后续模糊测试流程提供必要的数据库支持。本节首先通过类图和顺序图详细阐述该模块的设计思路；随后，结合关键代码片段和相关图示，深入解析该模块的核心功能实现，重点介绍基本信息提取和依赖关系推理的具体操作流程。

4.1.1 规格解析模块详细设计

图4-1展示了基于历史日志的 REST API 模糊测试系统的详细设计类图，包括该模块涉及的关键类以及类中的关键属性和方法。SwaggerParser 类将被主函数用于解析规格说明文档，提取其中的基本信息以及依赖关系，包括 `parse()`、`_analyse_dependencies()`、`_parse_version()`、`_parse_paths()` 等方法。其中，`parse()` 方法是该功能模块的入口，负责组织调度对规格文档的解析；其余以下划线开头的 `parse` 方法为 SwaggerParser 类的内部方法，负责逐步提取文档中的有效信息，比如规格文档版本、接口信息、数据模型等；最后的 `_analyse_dependencies()` 方法则负责调用大模型推理接口之间的依赖关系，是本系统的亮点功能之一。

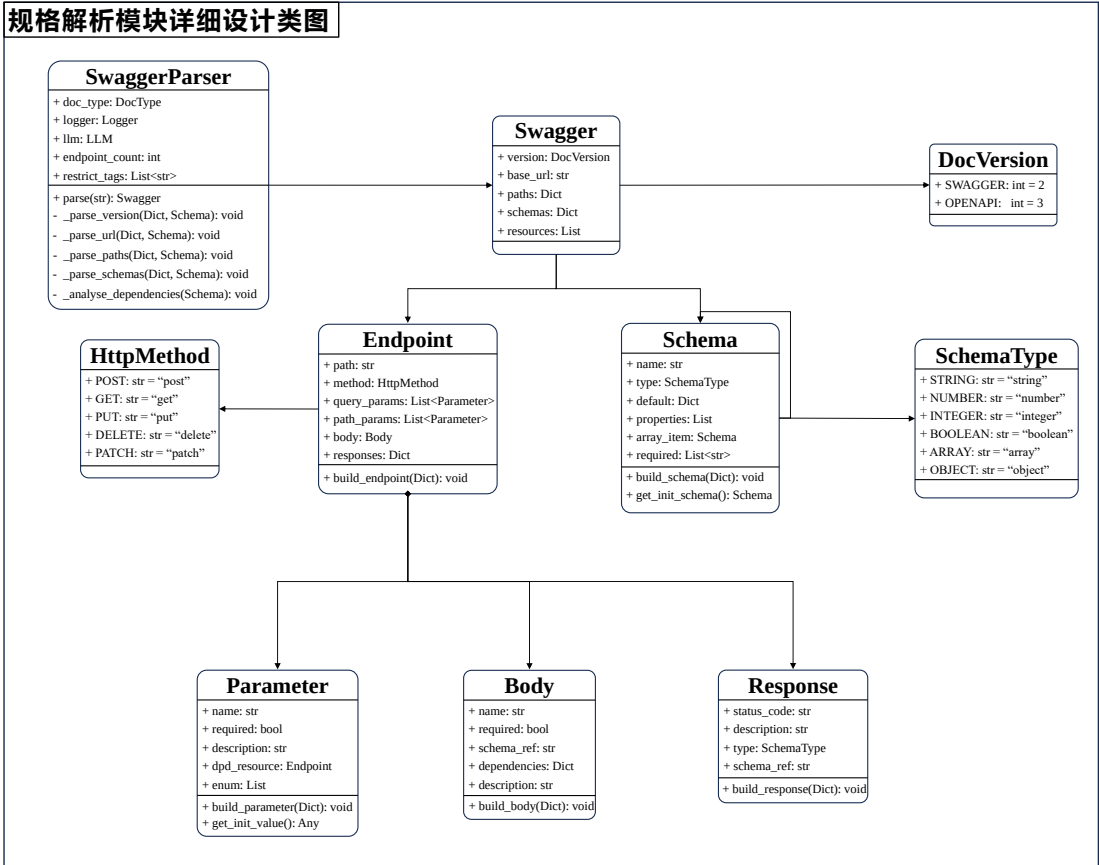


图 4-1 规格解析模块详细设计类图

SwaggerParser 类解析完成后，将生成一个 Swagger 类型的对象，该对象采用统一的数据结构来存储不同版本的 REST API 规格说明文档信息，从而便于后续的标准化处理和操作。在 Swagger 对象中，系统将接口信息和数据模型信息分别存储为 Endpoint 对象和 Schema 对象。其中，Endpoint 对象进一步细化，包含 Parameter、Body、Response 对象，分别用于存储接口的参数信息、请求体结构信息和响应结构信息。除了这些用于表示规格说明文档数据的核心类外，该功能模块还定义了一些静态枚举类，比如 DocVersion、SchemaType 和 HttpMethod。DocVersion 枚举用于表示不同版本的 API 规格说明文档（如 Swagger 或 OpenAPI）；SchemaType 负责定义数据模型的类型（如基本类型、数组、对象）；HttpMethod 枚举则用于表示系统支持的 HTTP 请求方法（包括 POST、DELETE、PUT、GET、PATCH）。这些枚举类的引入，提高了代码的可读性和可维护性，使得系统能够更灵活地适配不同的 API 规范。

图4-2展示了本系统规格解析模块的详细设计顺序图，其中的组件/参与者为该模块的核心功能类，不同组件之间通过方法调用进行交互。

主函数首先调用 `SwaggerParser` 的 `parse()` 方法，启动规格解析流程。作为规格解析的核心组件，`SwaggerParser` 负责整个解析过程的执行。首先，解析器调用自身的方法提取规格说明文档中的版本信息和基址信息，这一阶段无需与其他类交互，仅依赖文档内容进行解析。随后，解析器使用 `Endpoint` 类的构建方法初始化规格说明中的接口信息，并在此过程中与 `Body` 类和 `Parameter` 类交互，以解析并构建请求体和参数信息，确保 API 端点的完整定义。接着，解析器调用 `Schema` 类的构建方法，初始化文档中涵盖的数据模型。由于 API 规格文档可能包含嵌套结构的数据类型，`Schema` 采用递归解析的方式，确保能够正确解析复杂的对象关系。在完成基本的规格解析后，解析器进一步与 LLM 支持模块进行交互，利用系统内预设的提示词模板，逐一推理各接口所依赖的前置资源。这一过程能够自动推导出接口之间的调用依赖关系，为后续的模糊测试种子补全功能提供必要的数据库支持，确保模糊测试种子库中的测试用例的有效性。

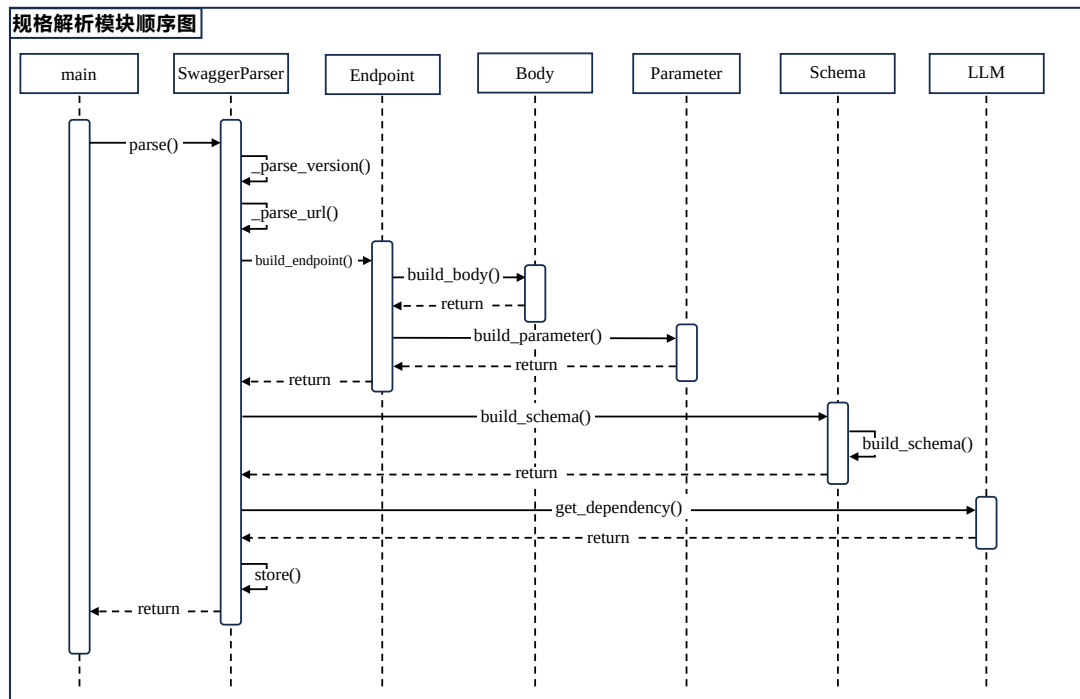


图 4-2 规格解析模块详细设计顺序图

4.1.2 基本信息提取功能具体实现

REST API 规格说明中的基本信息包括：1) 规格文档版本；2) 待测 REST API 服务基址；3) 接口信息；4) 接口参数信息；5) 接口请求体；6) 数据模型；7) 待测服务资源列表。在本模块的设计中，每类信息的解析均对应于一个具体的方法，以确保数据提取的清晰性和可维护性。本小节将重点介绍规格文档版本、接口信息、数据模型三类关键信息的解析过程，并结合算法设计和代码示例，详细阐述其具体实现方式。

规格文档版本识别。本系统支持 REST API 待测服务的两种不同版本的规格说明文档：Swagger2.0 和 OpenAPI3.0。这两种版本在文档结构、路径定义、操作方式、示例位置等多个方面存在显著差异，因此在解析之前，系统需要准确识别文档版本，以确保后续处理逻辑的正确性。在实现上，系统通过解析文档的关键字段来进行版本识别。图4-3展示了本系统识别待测服务文档版本的具体代码示例。该方法接收系统读取到的规格说明文档作为参数，判断键值“swagger”或者“openapi”是否存在于其中，进而判断文档的类型。如果两者均不存在，该方法抛出 `SwaggerComponentMissingError` 异常；如果存在，则将其以枚举类型的形式保存在 `Swagger` 对象当中，用于指导后续的解析步骤。

```
def _parse_version(self, data, swagger):
    if DocLevelObjs.SWAGGER in data:
        swagger.version_id = data[DocLevelObjs.SWAGGER]
        swagger.version = DocVersion.SWAGGER
    elif DocLevelObjs.OPENAPI in data:
        swagger.version_id = data[DocLevelObjs.OPENAPI]
        swagger.version = DocVersion.OPENAPI
    else:
        self.logger.error("version info is missing")
        raise SwaggerComponentMissingError()
    self.logger.info(f"Version Parsing Over: {swagger.version_id}")
```

图 4-3 规格版本识别功能代码示例

接口信息提取。接口信息部分是规格说明文档的主体部分，包括接口的 URI、HTTP 方法、参数结构、响应结构等重要信息，是后续测试过程中构造具体请求的“蓝图”。图4-4展示了本系统解析并提取待测服务规格说明文档的接口信息的代码示例。在 `SwaggerParser` 类中，`_parse_paths` 方法负责解析 Swagger 或 OpenAPI 文档中的接口/端点信息（paths），并将其转化为内部的端点表示。具体

而言，该方法首先检查文档中是否包含路径信息（`DocLevelObjs.PATHS`），若缺失则记录错误日志并抛出 `SwaggerComponentMissingError` 异常。随后，它遍历路径及其对应的内容，逐一解析每个 HTTP 方法（如 GET、POST 等）下的端点定义。在解析过程中，该方法支持基于标签（tags）的过滤机制：若启用了标签限制（`restrict_tags`），则仅处理包含支持标签的端点。此外，每个端点通过 `Endpoint` 类实例化，并调用其 `build_endpoint_from_doc` 方法完成端点的详细构建。

```
def _parse_paths(self, data, swagger):
    # 异常识别
    if DocLevelObjs.PATHS not in data:
        self.logger.error("SwaggerParser|_parse_paths the paths info is missing")
        raise SwaggerComponentMissingError(DocLevelObjs.PATHS)
    # 提取paths
    paths = data[DocLevelObjs.PATHS]
    for path, pathContent in paths.items():
        endpoints = {}
        for method, endPointContent in pathContent.items():
            if method not in [m.value for m in HttpMethod]: continue
            # 标签过滤
            if self.restrict_tags:
                if EndpointLevelObjs.TAGS not in endPointContent:
                    continue
                tags = endPointContent[EndpointLevelObjs.TAGS]
                for tag in tags:
                    if tag in self.supported_tags:
                        break
            else:
                continue
            # endpoint构造
            endPoint = Endpoint(swagger.base_url+path, method)
            endPoint.build_endpoint_from_doc(endPointContent)
            endpoints[HttpMethod(method).value] = endPoint
            self.endpoint_count += 1
        if endpoints == {}: continue
        swagger.paths[path] = endpoints
        self.path_count += 1
    self.logger.info(f"Paths Parsing Over")
```

图 4-4 接口信息提取功能代码示例

数据模型提取。 REST API 规格说明文档中的数据模型（schemas）用于定义待测服务 API 请求和响应中的数据结构，比如 HTTP 请求的请求体结构或者 HTTP 响应的响应体结构。它们描述了数据的字段、类型、格式、约束等，常在测试过程中用于验证数据格式是否符合预期。由于数据模型支持嵌套结构，可能包含多个层级的字段、复合类型或引用其他模型，因此在解析时需要正确解析层级关系并将其映射到预定义的对象结构。这一过程涉及递归解析、引用解析以及类型映射等复杂逻辑，使得准确提取和存储数据模型成为解析过程中的一大

挑战。算法 Algorithm1描述了本系统使用递归的方法解析规格说明文档中的所有数据模型的过程。

Algorithm 1 数据模型提取 $\text{parse_schema}(S, D)$

Require: 待解析的数据模型 S , 全局定义 D

Ensure: 预定义格式的数据模型 S'

```

1: if  $S$  为不支持的数据模型格式 then
2:   return None
3: end if
4: if  $S$  中包含指向另一个数据模型  $M$  的引用 then
5:    $\text{parse\_schema}(M, D)$ 
6:   return  $S'$ 
7: end if
8: if  $S$  的类型是数组 then
9:   获取其中的 items 键对应的数据模型  $M$ 
10:   $\text{parse\_schema}(M, D)$ 
11: else if  $S$  的类型是对象 then
12:   for each 属性  $P$  in  $S$  中键 properties 对应的属性列表  $L$  do
13:      $\text{parse\_schema}(P, D)$ 
14:     将返回结果追加到  $S'$  的 properties 属性列表中
15:   end for
16: else
17:   将  $S'$  中的关键属性进行赋值
18:   return  $S'$ 
19: end if

```

该算法将带解析的数据模型和全局定义（definitions）作为输入，根据不同的类型，递归地解析数据模型，确保对嵌套或复杂架构的支持。首先，算法判断输入的数据模型的格式是否为系统支持的格式，如果不符合，则直接返回空值；接着，算法检查输入数据模型的“ref”属性是否指向另外一个数据模型，如果是，递归调用本算法处理这个新的数据模型，处理完成后返回；如果输入数据模型的类型为数组，则需要提取其中的“items”属性，并递归地调用本算法处理该属性对应的数据模型；如果输入的数据模型类型为对象，则需遍历其中的“properties”属性对应的列表，其中的每一个数据模型都需递归调用本算法处理，并将得到的结果追加到返回的结构体中；如果输入数据模型不为上述两种类型，则一定为基本类型（字符串、整形、浮点型、布尔值），本算法将直接进行初始化，并返回初始化后的 Schema 对象，是本递归算法的正常终止条件。

图4-5展示了本算法在本系统中的具体代码实现示例。在具体的代码实现中，`build_schema_from_doc` 方法不仅体现了算法的核心逻辑，还通过多种机

制增强了代码的健壮性和可维护性。首先，代码通过日志记录（logger）详细了解解析过程中可能出现的异常情况，例如缺失必要的键（如 `type` 或 `items`）时的警告或错误信息。其次，代码对不同类型的模式（`array`、`object`、`primitive`）进行了细粒度的处理，例如在解析数组类型时，递归解析其 `items` 属性，并将其封装为子模式对象；在解析对象类型时，逐一解析其 `properties` 属性列表，并将每个属性构建为独立的 `Schema` 实例。此外，代码还支持对模式的辅助信息（如 `example`、`description`、`enum` 等）的提取和存储。

```
def build_schema_from_doc(self, schema, definitions, logger):
    # 验证关键属性是否存在
    if SchemaObjs.TYPE not in schema and SchemaObjs.REF not in schema:
        logger.error("the schema-type info is missing")
        raise SwaggerComponentMissingError(SchemaObjs.TYPE)

    # 当前数据模型指向了另一个数据模型
    if SchemaObjs.REF in schema:
        ref_schema_name = utils.get_schema_name_from_ref(schema[SchemaObjs.REF])
        ref_schema = utils.get_schema_by_name(ref_schema_name, definitions)
        self.build_schema_from_doc(ref_schema, definitions, logger)
        return

    self.example = schema.get(SchemaObjs.EXAMPLE, {})
    self.description = schema.get(SchemaObjs.DESCRPTION, "")
    self.enum = schema.get(SchemaObjs.ENUM, [])
    self.default = schema.get(SchemaObjs.DEFAULT, None)

    # 判断当前数据模型的类型是否支持
    if schema[SchemaObjs.TYPE] not in [t.value for t in SupportedSchemaType]:
        raise UnsupportedTypeError(schema[SchemaObjs.TYPE])
    self.type = SupportedSchemaType(schema[SchemaObjs.TYPE])

    # 数据模型为数组 (array)
    if self.type == SupportedSchemaType.ARRAY:
        if SchemaObjs.ITEMS not in schema:
            logger.warning(f"schna-items info is missing. Schema name: {self.name}.")
            raise SwaggerComponentMissingError(SchemaObjs.ITEMS)
        self.array_item = Schema("_array_item")
        self.array_item.build_schema_from_doc(schema[SchemaObjs.ITEMS], definitions, logger)
    # 数据模型为对象 (object)
    elif self.type == SupportedSchemaType.OBJECT:
        self.required = schema.get(SchemaObjs.REQUIRED, [])
        if SchemaObjs.PROPERTIES not in schema:
            logger.warning(f"schna-properties info is missing. Schema name: {self.name}.")
            raise SwaggerComponentMissingError(SchemaObjs.ITEMS)
        for name, p_data in schema[SchemaObjs.PROPERTIES].items():
            one_property = Schema(name)
            try:
                one_property.build_schema_from_doc(p_data, definitions, logger)
            except (SwaggerComponentMissingError, UnsupportedTypeError) as e:
                continue
            self.properties.append(one_property)
    # 数据模型为主类型 (string、number、integer、boolean)
    else:
        self.format = schema.get(SchemaObjs.FORMAT, "")
```

图 4-5 数据模型提取功能代码示例

4.1.3 依赖关系推理功能具体实现

接口之间的依赖关系是构造高质量测试用例的关键因素，不同的 REST API 测试工具采用不同的方法来提取这些依赖关系。在基于历史日志的 REST API 模糊测试系统中，我们选择从日志数据中提取真实世界的测试用例，这些请求本身通常已经隐含了 API 之间的依赖关系。然而，日志中的测试用例往往不完整或不够全面，难以覆盖所有可能的依赖路径和测试场景。

为了提升测试的全面性和有效性，系统仍然需要明确接口之间的前后依赖关系，用于指导后续的种子补全过程，确保模糊测试的输入数据覆盖更多 API 调用场景。为此，本系统借助大语言模型（LLM）的推理能力，自动分析 API 规格说明文档及历史日志，以推测和补充潜在的依赖关系，从而提高测试用例的质量和多样性。图4-6展示了本系统进行依赖关系推理的具体代码实现。

```
def _analyse_dependencies(self, swagger):
    count = 0
    for path, endpoints in swagger.paths.items():
        for method, endpoint in endpoints.items():
            count += 1
            print("***** " + method + " " + path + "*****" + str(count))
            # 基于提示词模板构造query
            p = swagger_parse_prompt.get_dependent_resource_prompt(swagger.all_resources, endpoint, swagger.schemas)
            # 获取大模型响应
            rsp = self.llm.send(
                messages = [
                    {"role": "user", "content": p}
                ]
            )
            # 保存信息
            self._store_dependent_info(endpoint, rsp)

def _store_dependent_info(self, endPoint, llm_rsp):
    for dependent_info in llm_rsp.splitlines():
        # 检验格式是否正确
        if not utils.regex_judge(f"#/{(BODY.value)}{PATH.value}|{QUERY.value}}/.+ /.", dependent_info):
            continue
        param = dependent_info.split(' ')[0]
        dependent_resource = dependent_info.split(' ')[1]
        param_elements = param.split('/')
        # 参数是query类型
        if param_elements[1] == SupportedParamLocation.QUERY.value:
            for p in endPoint.query_parameters:
                if p.name == param_elements[2]:
                    p.dependent_resource = dependent_resource
                    break
        # 参数是path类型
        elif param_elements[1] == SupportedParamLocation.PATH.value:
            for p in endPoint.path_parameters:
                if p.name == param_elements[2]:
                    p.dependent_resource = dependent_resource
                    break
        # 参数是body类型
        elif param_elements[1] == SupportedParamLocation.BODY.value:
            if endPoint.body != None:
                endPoint.body.dependent_params['/'.join(param_elements[2:])] = dependent_resource
```

图 4-6 依赖关系推理功能代码示例

在依赖关系推理的实现中，代码主要通过两个方法完成：`_analyse_dependencies()` 和 `_store_dependent_info()`。`_analyse_dependencies()` 方法负责遍历所有的 API 端点，并为每个端点生成提示词，然后通

过调用大语言模型来推理参数的依赖关系。`_store_dependent_info()` 方法负责解析 LLM 返回的信息，经过正则表达验证格式后，根据不同的参数类型以不同的形式将其存储在端点的参数结构中。`_analyse_dependencies()` 方法中，提示词的生成是依赖推理的关键步骤。提示词由 `prompts` 包中的方法生成，该方法根据端点的参数信息和资源信息再提示词模板之上构造了一个具体的提示词，指导 LLM 进行推理。图4-7展示了用于依赖关系推理的提示词模板。

 **提示词模板：依赖关系推理**


You are a rest api testing engineer and now you need to identify ...

Resources
The rest api service contains the resources below:
`{all_resource_str}`

Endpoint to Be Analysed
The endpoint to be identified is `{endpoint.method.value} {endpoint.path}`
The endpoint's parameters are as below:

name	type	description
-----	-----	-----

`{params_str}`

Judgement Method

- Your judgment needs to be based on parameter names, types or descriptions;
- If a parameter represents the ID, name, or key of a resource, then the resource must exist first for that parameter to be used;

Return Format and Example
You should return the result in json format. The json contains many key-value pairs, and the key is the parameter name and the value is its dependent resource.

Attention

- Do **not** return any descriptive content other than the rows of data required above...

Now please give me the answer:

大模型回答样例：

```

{{"#/body/blog_id": "/blog"},
{"#/path/id": "/blog"},
{"#/query/comment_id": "blog/{id}/comment"}}

```



图 4-7 依赖关系推理功能提示词模板

上述提示词模板通过结构化的内容提供了依赖推理任务所需要的全面的上下文信息，包括待测服务所有资源列表、接口的 HTTP 方法、接口的 URI 以及接口的所有参数（使用表格的形式展示）。其中，待测服务的所有资源列表在上一

节提到的接口信息提取环节中完成。本系统会扫描规格说明文档中所有的端点，对于 POST 方法的端点，如果端点的路径不以动态参数（如 id）结尾，则将其视作资源创建操作，并将该资源添加至资源列表中。接口的参数包括路径参数、查询参数和请求体中的参数，其中的路径参数和查询参数直接遍历 Endpoint 对象的列表即可，而请求体中的参数由于其嵌套特性，使用递归的方法提取，具体代码如图4-8所示。此外，该模板通过正反示例和严格的返回格式规范，确保模型输出结果稳定且易于解析。

```
def _get_params_in_schema(schema: Schema, name: str):
    # 数据模型类型为数组
    if schema.type == SupportedSchemaType.ARRAY:
        _get_params_in_schema(schema.array_item, f"{name}/{schema.name}")
    # 数据模型类型为对象
    elif schema.type == SupportedSchemaType.OBJECT:
        for prop in schema.properties:
            _get_params_in_schema(prop, f"{name}/{schema.name}")
    # 数据模型类型为主类型
    else:
        description = f"| {name}/{schema.name} | {schema.type.value} |"
        {schema.description} |"
        nonlocal params_str
        params_str += description + "\n"
```

图 4-8 请求体参数提取功能代码示例

4.2 日志解析模块详细设计与实现

日志解析模块是本系统的核心亮点模块，其主要职责为解析用户提交的待测服务服务端请求历史日志，并从中提取蕴含真实用户使用行为的日志切片，作为后续模糊测试过程的初始种子。从关键产物来看，该模块可划分为两个主要阶段：多用户日志队列提取和日志切片生成。在第一阶段，系统对原始日志进行解析和规格化处理，之后按照用户将日志分割为多用户日志队列；在第二阶段，系统基于多种策略将用户日志队列切分为众多的日志切片。本节将首先借助类图和顺序图详细阐述日志解析模块的设计思路和亮点；之后，结合关键代码和图示说明解析上述两个阶段的具体实现流程。

4.2.1 日志解析模块详细设计

图4-9展示了基于历史日志的 REST API 模糊测试系统日志解析模块的详细设计类图，涵盖本模块的核心类以及类中的关键方法和属性。

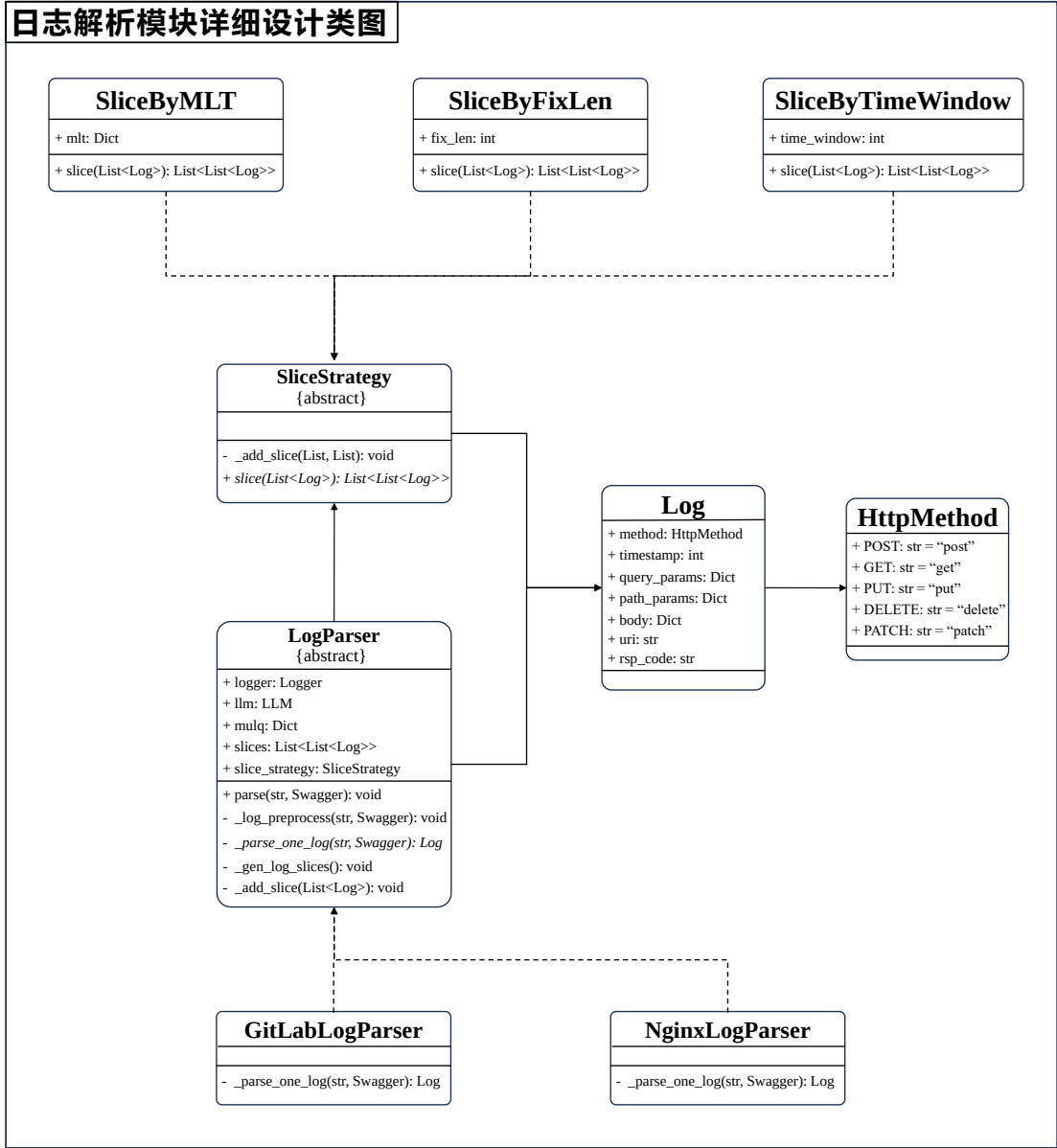


图 4-9 日志解析模块详细设计类图

主函数调用 `LogParser` 对象的 `parse()` 方法，对用户上传的服务端历史日志进行解析，并最终将这些日志转化为日志切片集合，作为模糊测试的初始种子库，为后续测试提供真实、有效的输入数据。`LogParser` 对象通过多个核心方法完成日志解析任务，包括 `_log_preprocess()`、`_parse_one_log` 和 `_gen_log_slices()` 等。其中，`_log_preprocess()` 方法作为钩子方法，

用于对单条日志进行规格化处理,确保不同格式的日志能够以统一的结构存储和解析。值得注意的是,LogParser 采用了模板方法模式,以适应不同格式的日志文档,并提供灵活的扩展性和可维护性。在日志切片的生成过程中,本系统支持三种不同的策略来对用户日志队列进行切割,以便更精准地提取有意义的测试用例。这三种策略为:1)最大前置时间(Max Lead Time, MLT),关注日志切片中请求的时间间隔;2)时间窗口,关注日志切片的时间跨度;3)固定长度,确保切片中的请求数量相同。这些策略的实现采用策略模式,分别对应于 SliceStrategy 抽象类的三个子类:SliceByMLT、SliceByTimeWindow、SliceByFixLen,确保日志切片的生成方式灵活可配置,能够适应不同的测试需求。

图4-10展示了日志解析模块的详细设计顺序图。本图选取 GitLabLog-Parser 作为默认的日志解析子类,并采用 SliceByMLT 作为默认的日志切片策略,以示范整个解析流程的核心步骤。整个流程由主函数调用 LogParser 的 parse() 方法启动。首先,LogParser 依次与 LLM 和 Swagger 对象交互,以提取日志中的关键信息并进行日志的规格统一化。在这一过程中,解析器还会识别用户身份,将原始日志数据按照用户会话信息进行分类,转换为多用户日志队列,为后续的测试输入生成奠定基础。完成日志规格化处理后,解析器根据用户指定的切片策略对用户日志队列进行切割,将日志数据划分为多个日志切片,并汇总至模糊测试初始种子库。

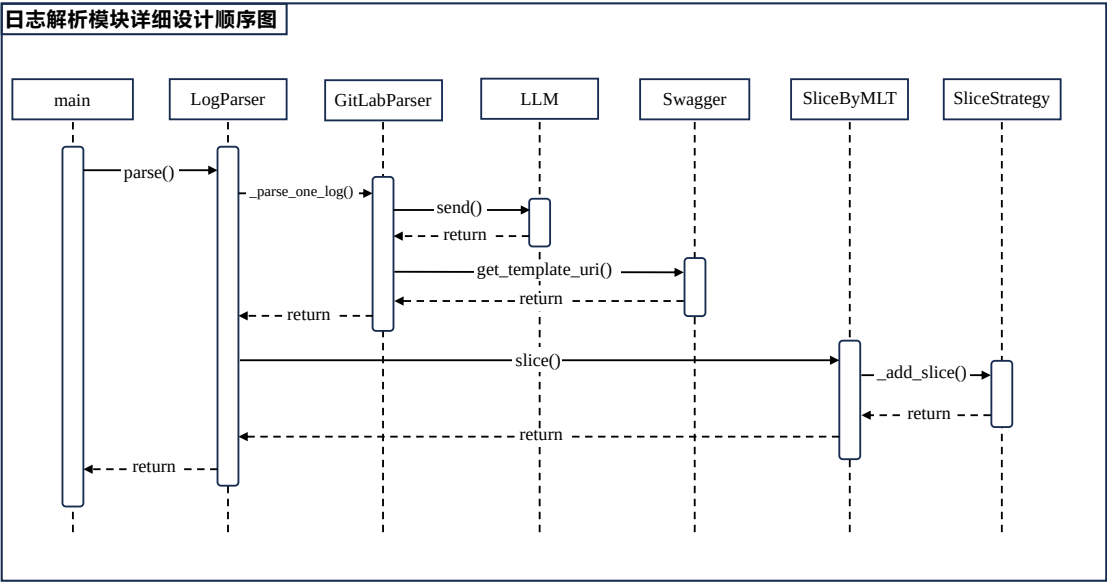


图 4-10 日志解析模块详细设计顺序图

4.2.2 多用户日志队列提取具体实现

在日志解析的第一阶段，系统首先对用户提交的历史日志进行处理，将其转换为多用户日志队列，该过程包括关键信息提取和日志队列划分两个核心步骤。

首先，日志解析器采用不同的解析方法，从不同格式的日志中提取关键信息，并将其存储到统一的数据结构中，以确保后续处理的一致性和高效性。由于不同系统的日志格式存在较大差异，解析器需要具备高度适应性，能够处理结构化（如 JSON、CSV）和非结构化（如纯文本日志）的多种日志类型。为提升对未知日志格式的适应能力，本系统内置了基于大语言模型的日志解析方法。当遇到无法直接解析的日志格式时，系统会调用大语言模型，利用其自然语言理解与推理能力，自动分析日志内容，提取出请求方法、端点、状态码、时间戳、请求参数等关键字段。图4-11展示了用于提取日志中关键信息的提示词模板。

 **提示词模板：日志信息提取**

Now you need to extract information from a log of a REST API service.



Result Components

The information you extract should contain the below components:

- method: str, the request's HTTP method, for example: POST, GET...
- uri: str, the request's concrete uri, for example: /blog, /blog/3/comment...
- time: str, the request's time info, for example...
- time_format: str, the format of the time, for example: ISO, timestamp...
- code: str, the HTTP code, for example: 201, 200, 401, 403, 500...
- params: json, the request's params, which are the parameters of the request
- user: str, the user who sent the req, if the log doesn't contain user information, this field should be null

Result Format

The result you return to me should in json format.
Please do not include any explanation or descriptions in your request instances...

Example

The following is an example...

Now the log is: `{log}`
You should return:

大模型回答样例：



```
{“method”: “POST”, “code”: 200, “uri”: “/blog”, “time”: 1729776416, “params”: {...}, “user”: “Bob”, “time_format”: “timestamp”}
```

图 4-11 日志信息提取功能提示词模板

在成功提取日志中的关键信息后，系统会将其统一封装为标准化的 Log 对象保存在内存中；并且根据提取到的用户标识信息，将原始的日志队列拆分为多用户日志队列（Multi-User Log Queue, MULQ）以恢复不同用户的操作轨迹并保留其上下文关联性，如图4-12所示具体代码。

```
def _parse_one_log(self, line, swagger):
    # 验证返回的有有效性
    .....

    # 将时间统一为时间戳的形式
    time_format = log_json[LogKeys.TIME_FORMAT]
    if time_format.upper() not in [f.value for f in TimeFormat]: return
    log_time = log_json[LogKeys.TIME]
    timestamp = utils.transfer_time_to_timestamp(log_time, TimeFormat(time_format.upper()))

    # 根据具体uri找到模板uri
    concrete_uri = log_json[LogKeys.URI]
    template_uri = swagger.get_template_uri_by_concrete(concrete_uri)
    if template_uri == None: return

    # 识别用户
    user_name = log_json[LogKeys.USER_NAME]
    if user_name == None: user_name = DEFAULT_USER_NAME

    one_log = Log(method=HttpMethod(method.lower()), ...) # 初始化Log对象

    # 统一参数格式
    params = log_json[LogKeys.PARAMS]
    endpoint = swagger.paths[template_uri][HttpMethod(method.lower()).value]
    # 1) query类型的参数
    for param in params:
        for query_param in endpoint.query_parameters:
            if param["key"] == query_param.name:
                one_log.query_params[param["key"]] = param["value"]
                break
        else:
            one_log.body[param["key"]] = param["value"]
    # 2) 路径参数
    concrete_uri_components = concrete_uri.split('/')
    template_uri_components = template_uri.split('/')
    for template, concrete in zip(template_uri_components, concrete_uri_components):
        if template.startswith('{') and template.endswith('}'):
            name = template.strip("{}")
            value = utils.valid_and_convert_int(concrete)
            one_log.path_params[name] = value
    # 按照用户划分队列，获取多用户日志队列
    if user_name in self.mulq:
        self.mulq[user_name].append(one_log)
    else:
        self.mulq[user_name] = [one_log]
```

图 4-12 多用户日志队列生成功能代码示例

具体来说，日志解析器首先检验大模型返回的关键信息是否符合要求，比如是否为 JSON 格式、提取到的信息是否合理等；随后，解析器根据已有信息进行数据转换和数据关联，比如根据具体的 URI 匹配模板 URI 从而获取对应的接口信息、将各种类型的时间格式转化为统一的时间戳形式便于后续处理、将日志中的参数信息根据类型（query、path、body）存储到 Log 对象不同的属性当中；最后，解析器根据大模型提取到的用户标识信息，在遍历的过程中将单一的

日志序列划分为多用户日志队列，使得依赖信息更加集中。

4.2.3 日志切片具体实现

作为本系统亮点功能，日志切片负责将原始用户行为数据转化为高质量测试输入。该模块通过多种切片策略，将用户日志队列切分为若干具有完整调用语义的日志切片，每个切片都保留了用户在实际使用过程中的请求顺序与上下文关系，具有高度的现实代表性与重放价值。本系统目前提供四种策略来实现日志切片，包括基于 HTTP 方法的最大前置时间字典（Max Lead Time Dict of HTTP Method）、时间窗口（Time Window）、固定序列长度（Fixed Sequence Length），以及混合（Mixed）策略。下面将对这些策略的具体实现算法以及关键数据结构进行介绍。本小节的具体实现代码与算法高度重合，因此不再进行示例展示。

基于 HTTP 方法最大前置时间字典的切分策略。该策略允许待测 REST API 服务的开发者根据用户的请求调用习惯，自定义 HTTP 方法最大前置时间字典。系统在解析日志时，以该字典作为参考依据，判断不同请求之间的最大可接受时间间隔，从而将属于同一操作场景的一系列请求聚合为一个完整的日志切片。通过这种方式，系统能够更准确地提取出具有行为连续性和业务相关性的日志片段，提升切片的上下文完整度。图4-13展示了一个 YAML 格式配置文件中的最大前置时间示例。该字典中，键表示具体 HTTP 方法的类型，值则表示该方法对应的最大前置时间。在日志切片过程中，系统会依据此字典判断相邻请求之间的时间间隔。如果当前请求与其上一个请求之间的时间差超过其对应的最大前置时间，则系统会将这两个请求划分为不同的日志切片，以避免将不相关的用户行为错误地聚合在一起。一般来说，用户在执行增删改操作（对应于 POST、PUT、DELETE）之前，往往需要较长的准备时间；而查询操作（GET）则更为频繁、反应更快，因此其前置时间阈值通常较短。

```
Max_Lead_Time_Dict: # HTTP方法最大前置时间字典
  get:      "5m"
  put:      "10m"
  post:     "10m"
  patch:    "10m"
  delete:   "10m"
```

图 4-13 HTTP 方法最大前置时间字典示例

算法 Algorithm2展示了系统根据最大前置时间字典对用户日志队列进行切片的完整流程。首先，系统将待处理的用户日志队列中的首个日志请求时间戳，作为初始的“上一个请求”（即 `time_before`）。之后，系统遍历日志队列中的每条日志记录，计算其时间戳与 `time_before` 之间的时间间隔。如果间隔超出了当前请求类型的最大前置时间，则判定当前请求与前一请求之间存在行为断裂，系统会将当前构建的日志切片 `slice` 作为一个完整的片段添加到日志切片列表中，并重新开始一个新的切片。随后，系统判断当前日志记录的请求是否为有效请求（即请求响应为 20X 或者 50X），若是，则将其添加到当前 `slice` 中。整个流程将持续执行，直到日志队列中的所有日志均被处理完毕。

Algorithm 2 基于最大前置时间字典的日志切片 `slice_by_mlt(Q)`

Require: 日志队列 Q

Ensure: 日志切片列表 $Slices$

```

1: 初始化空切片列表  $Slices \leftarrow []$ 
2: 初始化一个切片  $slice \leftarrow []$ 
3:  $time\_before \leftarrow$  首个请求的时间戳  $Q[0].timestamp$ 
4: for 日志  $log$  in  $Q$  do
5:   if  $log.timestamp - log.mlt > time\_before$  then
6:     将  $slice$  补充到  $Slices$  尾部
7:     重新初始化  $slice \leftarrow []$ 
8:   end if
9:   if 当前日志响应为 20X 或者 50X then
10:    将  $log$  添加到  $slice$  尾部
11:   end if
12:   更新时间  $time\_before \leftarrow log.timestamp$ 
13: end for
14: 将  $slice$  补充到  $Slices$  尾部
15: return  $Slices$ 

```

基于时间窗口的切分策略。该策略通过设定固定的时间窗口，将用户日志队列中处于同一时间段内的请求聚合为一个日志切片。具体而言，系统会以日志中的时间戳为依据，在连续的时间窗口范围内收集所有日志条目，形成一个具有时间连续性的日志片段。用户可通过配置文件中的 `log_parser.timewindow` 字段自定义时间窗口的大小。

算法 Algorithm3展示了系统根据时间窗口对用户日志队列进行切片的完整流程。首先，系统初始化一个空的日志切片列表 $Slices$ ，用于存储所有生成的日志切片。随后，系统从日志队列中的每条日志记录开始，依次作为当前切片的起点 `first`，并初始化一个空的切片 `slice`。对于从 `first` 开始的每条日

志记录，系统判断其是否在时间窗口内以及是否为有效请求。若是，则将其添加到当前切片 *slice* 中。当时间差超出时间窗口大小 T 时，系统判定当前日志记录与起点日志记录之间的行为已超出时间窗口范围，因此将当前构建的日志切片 *slice* 作为一个完整的片段添加到日志切片列表 *Slices* 中，并跳出内层循环，开始处理下一条日志记录作为新的起点。整个流程将持续执行，直到日志队列中的所有日志均被处理完毕。最终，系统返回生成的日志切片列表 *Slices*。

Algorithm 3 基于时间窗口的日志切片 *slice_by_time_window(Q, T)*

Require: 日志队列 Q , 时间窗口大小 T

Ensure: 日志切片列表 *Slices*

```

1: 初始化空切片列表 Slices  $\leftarrow []$ 
2: for 每条日志 first in  $Q$  do
3:   初始化一个切片 slice  $\leftarrow []$ 
4:   for 从 first 开始的每条日志 log in  $Q$  do
5:     if  $log.timestamp - first.timestamp \leq T$  then
6:       if 当前日志响应为 20X 或者 50X then
7:         将 log 添加到 slice 尾部
8:       end if
9:     else
10:      将 slice 补充到 Slices 尾部
11:      跳出内层循环
12:    end if
13:  end for
14:  if slice 非空 then
15:    将 slice 补充到 Slices 尾部
16:  end if
17: end for
18: return Slices

```

固定序列长度切片策略。该策略将用户日志队列切分为固定长度的请求序列，用户可以通过配置文件中的 `log_parser.fixed_len` 字段自定义时间窗口的大小。该策略适用于日志结构较为规整、请求之间不存在强时序依赖关系的场景，例如自动化测试产生的日志或某些批量接口调用的系统。

算法 Algorithm4 描述了该策略的过程，用于将日志队列拆分为若干长度相等的日志切片。该方法以初始化一个空列表 *Slices* 开始，专门用于保存所有生成的切片。算法从日志队列的首项开始遍历，每轮迭代都会从当前位置创建一个新的日志切片 *slice*，并通过计数器 *count* 跟踪当前切片中已添加的有效日志数量。在遍历过程中，系统判断当前切片是否已包含足够数量的日志条目（由参数 L 指定）。如果未达到，系统进一步检查当前日志记录是否为有效请

求。只有满足条件的日志才会被计入切片，同时计数器随之递增。一旦某个切片中的有效日志数量达到了指定上限，系统便将该切片加入结果列表 *Slices*，并从下一个日志记录开始新一轮切片的构建。该过程将持续执行直至日志队列被全部处理完毕。

Algorithm 4 基于固定序列长度的日志切片 *slice_by_fixed_len(Q, L)*

Require: 日志队列 *Q*, 固定序列长度 *L*

Ensure: 日志切片列表 *Slices*

```

1: 初始化空切片列表 Slices  $\leftarrow []$ 
2: for 每条日志 start in Q do
3:   初始化计数器 count  $\leftarrow 0$ 
4:   初始化一个切片 slice  $\leftarrow []$ 
5:   for 从 start 开始的每条日志 log in Q do
6:     if count < L then
7:       if 当前日志响应为 20X 或者 50X then
8:         将 log 添加到 slice 尾部
9:         count  $\leftarrow$  count + 1
10:      end if
11:    else
12:      将 slice 补充到 Slices 尾部
13:      跳出内层循环
14:    end if
15:  end for
16:  if slice 非空 then
17:    将 slice 补充到 Slices 尾部
18:  end if
19: end for
20: return Slices

```

混合策略指的是将前述多种切片策略（如最大前置时间策略、时间窗口策略、固定长度策略）灵活组合应用于用户日志队列，从而生成结构多样、覆盖面更广的初始种子库，以提升模糊测试的效果与全面性。由于其实现逻辑主要依赖于已有策略的组合与调度，故此处不再赘述具体实现细节。需要特别强调的是，用户应根据待测服务的具体日志特征、调用行为规律以及实际测试目标，合理选取或配置不同的切片策略。例如，对于请求密集、交互频繁的系统，可优先采用时间窗口或固定长度策略；而对于操作任务场景丰富的系统，基于 HTTP 方法的前置时间策略则可能更具针对性。混合策略提供了更大的灵活性，但也要求用户对系统行为有更深入的理解，以实现定制化和高效的测试数据生成。

4.3 模糊测试模块详细设计与实现

在规格解析和日志解析的基础上，模糊测试模块承担着本系统核心的测试执行职责，负责驱动整个 REST API 模糊测试流程。模糊测试模块首先对生成的初始种子库进行增强处理。增强过程包括种子丰富和切片补全，旨在提高种子的多样性和有效性，为后续的测试阶段奠定基础。该过程依赖于前期解析模块提取的结构信息与依赖信息，也可结合大语言模型对模糊种子进行智能补全，提升种子的语义合理性。完成种子增强后，模糊测试模块正式启动测试循环。该循环包括种子选择、种子变异、用例执行与反馈收集等多个关键步骤。本节首先展示模糊测试模块的详细设计类图和顺序图，之后通过算法和代码示例描述种子库增强和模糊测试两大功能的具体实现。

4.3.1 模糊测试模块详细设计

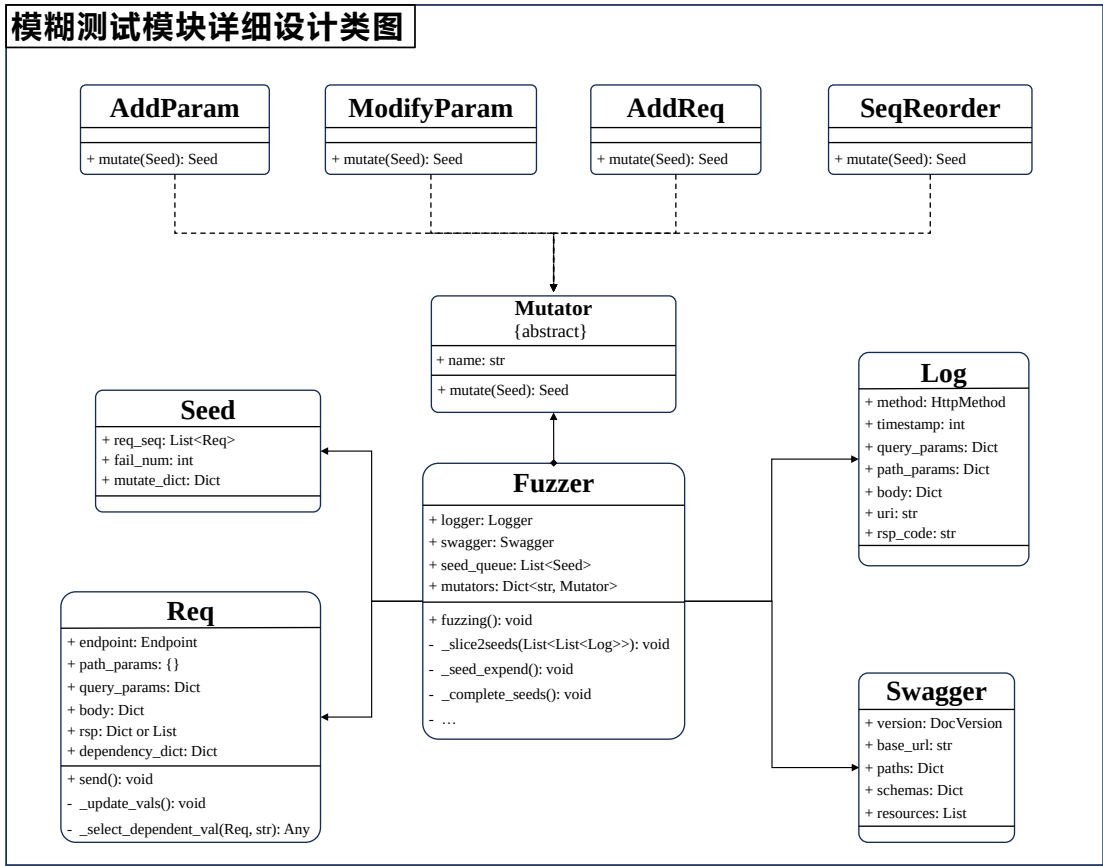


图 4-14 模糊测试模块详细设计类图

图4-14展示了模糊测试模块的详细类图设计，直观体现了模块中各类的职责

划分与协作关系。其中，Fuzzer 类作为该模块的核心调度器，负责组织和驱动整个模糊测试流程。它不仅承担测试过程的生命周期管理（如初始化、执行、终止等），还集成了规格解析模块与日志解析模块所提供的关键信息，例如接口定义、参数结构、接口依赖关系和初始测试种子等，用于指导后续测试用例的生成。在变异机制设计方面，Fuzzer 类内部维护了一个变异算子字典，用于统一管理当前系统支持的所有种子变异操作。该字典通过关键字索引具体的变异策略，从而实现灵活的算子调用与组合。本系统的变异算子适用策略模式实现，不同类型的变异操作通过继承统一的抽象基类 Mutator 并重写其 mutate() 方法来定义各自的具体行为。目前内置的典型变异策略包括：添加参数（AddParam）、修改参数（ModifyParam）、添加请求（AddReq）、重排序列（SwqReorder）等。本系统支持用户自定义算子扩展模糊测试模块的功能和效果。

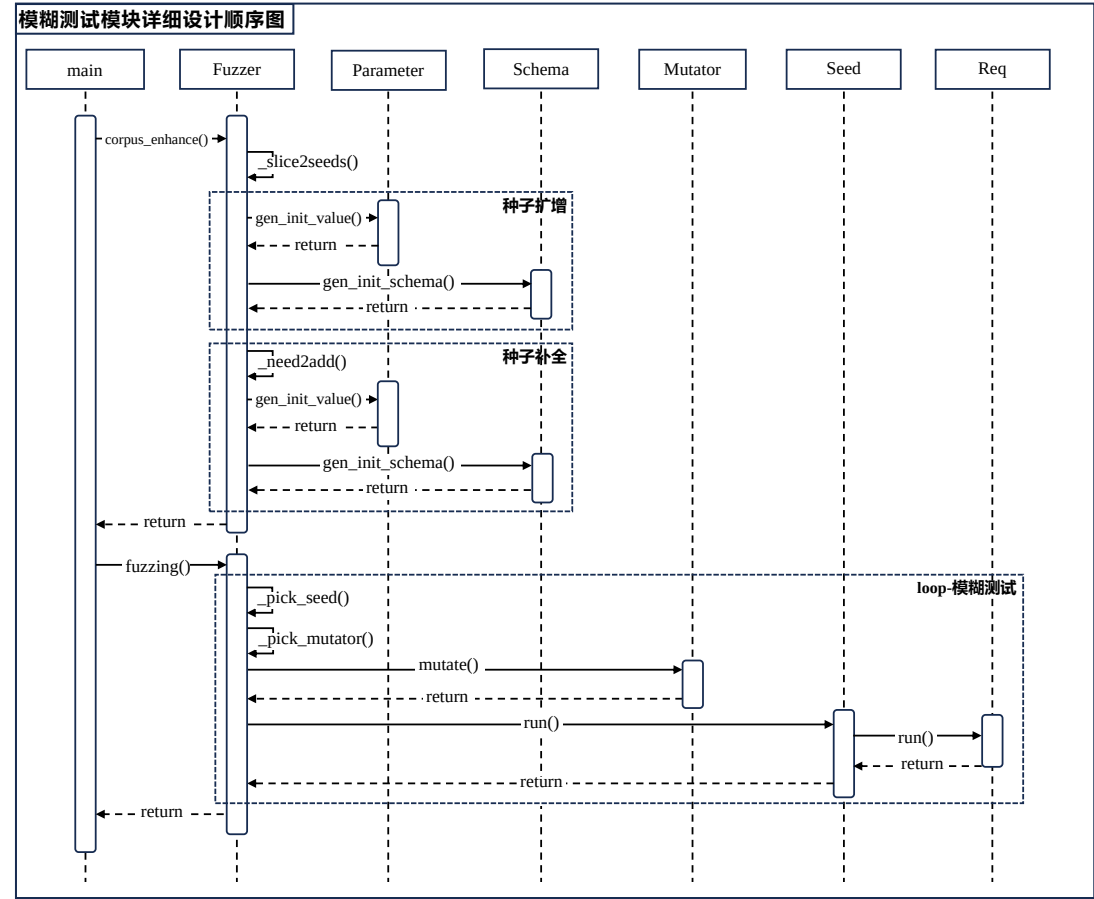


图 4-15 模糊测试模块详细设计顺序图

图4-15为模糊测试功能模块的详细设计顺序图，描述了各组件间的交互过程及其执行逻辑。主函数根据用户配置初始化 Fuzzer 对象，并以此作为调度核心，推动整个模糊测试流程的执行。

首先，主函数调用 `_slice2seeds()` 方法将日志解析模块得到的日志切片集合转化为初始种子库。接下来，Fuzzer 对象调用 `_corpus_enhance()` 方法对初始种子库进行增强。该阶段包括两个关键环节：种子扩增和种子补全。种子扩增环节，系统遍历待测服务的接口，为每个可用接口创建一条具体请求并构造为种子。生成请求的过程中，系统需调用 `Parameter` 对象的 `_gen_init_value()` 方法生成路径参数和查询参数具体的值，同时借助 `Schema` 对象的 `_gen_init_schema()` 方法生成请求体。种子补全环节，系统根据设定的算法判断当前的请求序列是否缺少关键的前置资源构造请求（`need2add()` 方法），若是，构造具体的 `POST` 请求并插入请求序列头部。

种子库完成增强后，主函数调用 `Fuzzer` 对象的 `fuzzing()` 方法正式开启模糊测试。模糊测试以循环方式持续进行。在每一轮测试中，模糊测试组件都会从当前种子队列中选取一个合适的种子（`pick_seed()` 方法），并结合响应的变异算子对其进行变异（`pick_mutator()` 和 `mutate()` 方法）。随后，模糊测试组件执行变异后的请求序列，并对执行结果进行记录和分析，若该序列展现出新的行为特征或满足特定触发条件，则会被视为高价值种子，加入到种子队列中用于后续测试的迭代。当用户手动终止模糊测试流程或者模糊测试达到预设的时间阈值，系统会停止测试并统计测试结果。

4.3.2 种子库增强具体实现

日志解析模块输出的初始种子库存在两方面的缺陷：1）种子库中只包含日志里出现过的请求接口；2）通过切片获取到的种子库中的种子缺少前置的资源创建请求。对于者两方面的缺陷，种子库增强分别采用种子扩增和种子补全两个手段进行弥补。本小节将对这两种手段的具体实现算法和代码进行介绍。

种子库增强。图4-16为种子扩增整体流程的具体代码实现。该方法使用到规格解析组件处理得到的 `Swagger` 对象。具体来说，`_seed_augment()` 方法会遍历待测服务的每一个端点，并调用 `_build_req_from_endpoint()` 方法构造具体的请求。之后，系统将该请求插入到一个空的请求序列当中，并将其作为新的种子放入种子库中。

图4-17展示了此前提到的 `_build_req_from_endpoint()` 方法的具体代码实现。该方法不仅应用于种子扩增环节，还被之后的种子补全环节、模糊

```

def _seed_augment(self):
    for path, eps in self.swagger.paths.items():
        # 遍历每一个path
        for method, endpoint in eps.items():
            # 遍历每一个endpoint
            req = self._build_req_from_endpoint(endpoint)
            seed = Seed()
            seed.req_seq.append(req)
            self.seed_queue.append(seed)

```

图 4-16 种子扩增功能代码示例

测试变异环节广泛应用，用于构造一个初始的请求。该方法首先生成路径参数的具体值，构建过程中采用由前向后的顺序将路径参数添加到 `path_params` 列表中，以确保路径层级和参数匹配的准确性。对于路径参数和查询参数这样简单的基础数据类型，系统在生成具体值时遵循“优先使用已知值”的策略，即系统会根据之前获取到的接口信息判断是否存在 `enum` 枚举值或者 `default` 默认值，如果有，则优先填入这些值，如果没有，则使用系统设定的默认值。

```

def _build_req_from_endpoint(self, endpoint:Endpoint) -> Req:
    req = Req()
    req.endpoint = endpoint
    # 1) 构建具体路径参数（注意顺序）
    for component in endpoint.path.split('/'):
        if not (component.startswith("{") and component.endswith("}")): continue
        name = component.strip("{}")
        for param in endpoint.path_parameters:
            if name != param.name: continue
            req.path_params[name] = param.gen_init_value()
            break

    # 2) 构建具体查询参数
    for param in endpoint.query_parameters:
        if not param.required: continue
        req.query_params[param.name] = param.gen_init_value()

    # 3) 构建具体请求体
    if endpoint.body is not None and endpoint.body.schema_ref != "":
        schema = utils.get_schema_by_ref(endpoint.body.schema_ref, self.swagger.schemas)
        req.body = schema.gen_init_schema()

    return req

```

图 4-17 具体请求构造功能代码示例

由于数据模型的嵌套结构，请求体的构造过程相较于路径参数和查询参数更加复杂，且更具挑战性。图4-18展示了构造具体的请求体的代码示例，示例中的 `gen_init_schema()` 方法使用递归的方式实现。具体而言，如果当前处理的数据模型类型为对象（Object），方法则遍历对象的属性列表，并递归处理

其中标记为必填（required）的属性；如果当前处理的数据模型为数组（Array），系统则默认生成一个长度为 2 的数组返回；如果当前处理的数据类型为主数据类型（Number, Integer, String 或者 Boolean），系统还会按照“优先使用已知值”的策略，生成具体的值返回。

```
def gen_init_schema(self):
    res = None
    # 1) 当前处理类型为对象-object
    if self.type == SupportedSchemaType.OBJECT:
        res = {}
        for prop in self.properties:
            if not prop.name in self.required: continue
            sub_schema = prop.gen_init_schema()
            if sub_schema is not None: res[prop.name] = sub_schema
    # 2) 当前处理类型为数组-array
    elif self.type == SupportedSchemaType.ARRAY:
        if self.array_item == None: return None
        element1 = self.array_item.gen_init_schema()
        element2 = self.array_item.gen_init_schema()
        if element1 is not None and element2 is not None: res = [element1, element2]
    # 递归终止条件
    elif self.type is None: res = None
    # 3) 当前处理类型为主数据类型-number,int,bool,string
    else:
        if self.default is not None: res = self.default # 首选default
        elif len(self.enum) != 0: res = self.enum[0] # 次选enum
        else: res = DEFAULT_VAL_DICT[self.type] # 最后使用本系统默认值
    return res
```

图 4-18 请求体构造功能代码示例

种子补全。基于日志解析和种子扩增所构建的种子库中，部分种子可能存在结构不完整、前置依赖缺失等问题，容易在后续的测试执行过程中导致请求失败或结果无效。为提高测试的完整性与有效性，系统引入了“种子补全”机制，用于自动填补请求序列中缺失的关键前置资源创建请求。

算法 Algorithm5 展示了种子补全过程的核心步骤与逻辑流程，该算法将基于接口之间的依赖关系，自动推导并插入所需的资源创建请求，以修复不完整的调用序列。为了确保层级资源的创建顺序正确，系统采用逆序遍历的方式，从后向前遍历请求序列 *Sequence* 中的每一个请求 *Req*。在处理每个请求时，系统会依次检查其中的参数 *P* 是否依赖于尚未创建的前置资源 *R*。具体而言，系统首先依赖于规格解析模块提供的依赖关系推理功能，获取参数 *P* 所需的前置资源 *R*。随后，系统从请求序列的起始位置开始，正向遍历至当前请求 *Req*，查找是否已存在用于创建资源 *R* 的请求 *Req'*。如果找到了对应的资源创建请求，系统则记录该依赖关系（参数 *P* → 请求 *Req'*）；若未找到，说明该前置资源尚未创建，系统将自动构造一个新的资源创建请求 *Req''*，将其插入到请求序列 *Sequence* 的前部，并记录该新的依赖关系（参数 *P* → 请求 *Req''*）。下面将结合具体的代码示

例，对种子补全过程中的一些重要细节进行解释。

Algorithm 5 种子补全 `complete_seed(Seed)`

Require: 种子 *Seed*

Ensure: 补全后的种子 *Seed'*，依赖信息字典 *Dict*

```

1: for 从后往前遍历请求序列 Sequence 的每一个请求 Req do
2:   for 遍历请求的每一个参数 P do
3:     获取依赖资源  $R \leftarrow P.R$ 
4:     if 资源 R 存在 then
5:       for 从前往后遍历 Sequence 的请求 Req' 直到当前 Req do
6:         if Req' 为资源 R 的创建请求 then
7:           记录该依赖到 Dict 并跳到下一个参数
8:         end if
9:       添加该参数依赖的资源创建请求并记录该依赖到 Dict
10:    end for
11:  end if
12: end for
13: end for
14: return Seed', Dict

```

系统按照以下顺序依次遍历每个请求中的有效参数：路径参数、查询参数以及请求体中的参数。需要特别指出的是，在遍历路径参数时，系统采用从后向前的顺序进行处理，以确保插入的资源创建请求满足层级关系的先后顺序。也就是说，父资源的创建请求会优先于子资源，确保请求序列在逻辑上完整且符合真实服务的依赖结构。图4-19所示的代码示例便展示了该处理逻辑的具体实现。

```

# 逆序遍历路径参数，满足REST API层级性资源特性
for name in reversed(req.path_params):
    for param in req.endpoint.path_parameters:
        if param.name != name: continue
        resource = param.dependent_resource
        if resource is None: break
        # 判断资源是否缺失并添加
        add_or_not, d_req = self._judge_and_add_req(seed.req_seq, idx, resource)
        # 使用规定格式记录依赖
        if d_req is not None: req.dependency_dict[f"#/path/{name}"] = d_req
        # 如果添加了资源，注意遍历索引
        if add_or_not: idx += 1
        break

```

图 4-19 路径参数补全功能代码示例

请求的参数按照是否依赖前置资源可以分为两类：依赖性参数（通常为前置资源的 id 或者名字）和非依赖性参数。非依赖性参数在测试执行前即可确定，而依赖性参数在运行时根据前置资源创建请求的响应内容才能确定，因此本系统在每个请求中使用一个依赖字典记录参数到请求序列中的具体请求的依赖关

系，如图4-20所示即为一个请求参数依赖字典的例子。

```
{
    "#/path/id": <Req-/blog>,
    "#/query/name": <Req-/blog/{id}/comment>,
    "#/body/allowed_to_push/[]/0/user_id": <Req-/user>
}
```

图 4-20 请求参数依赖字典示例

同样由于数据模型的嵌套特性，判断请求体中的参数是否需要补充前置资源创建请求时，必须采用递归方式处理，如图4-21中所示的方法。该方法为种子补全方法的内部函数，其中变量 `idx` 显示引用了外层方法的当前请求索引。

```
# 本方法为_complete_seeds()方法的内部方法
def _traverse_body_and_complete(body, r_req:str, r_param:str):
    nonlocal idx # 显示声明idx使用外层方法的变量

    # 当前为对象类型-object
    if type(body) == dict:
        for key, value in body.items():
            _traverse_body_and_complete(value, r_req+'/'+key, r_param+'/'+key)

    # 当前为数组类型-array
    elif type(body) == list:
        for i in range(0, len(body)):
            _traverse_body_and_complete(body[i], r_req+f'/{i}', r_param+'/_array_item')

    # 当前为主数据类型-number, int, bool, string
    else:
        # 判断参数是否为依赖性参数，不是则直接返回
        if r_param not in dependencies: return

        # 判断该依赖性参数是否缺少资源创建请求，如果缺少，添加请求
        resource = dependencies[r_param]
        add_or_not, d_req = self._judge_and_add_req(seed.req_seq, idx, resource)

        # 使用规定格式记录依赖
        if d_req is not None: req.dependency_dict[r_req] = d_req

        # 如果添加了资源，注意修改遍历索引
        if add_or_not: idx += 1

    # 终止条件
    return
```

图 4-21 请求体参数依赖资源补全代码示例

针对不同类型的数据模型，该方法采用有针对性的处理逻辑。对于对象类型，本方法遍历对象对应的字典中的键值对，并递归处理其中的嵌套对象。对于数组类型，依次遍历数组中的元素，之后同样递归处理。对于主数据类型，方法判断参数是否为依赖性参数以及是否缺失依赖创建请求，如果缺失则进行补全，最后记录该参数与具体资源创建请求之间的依赖关系。

4.3.3 模糊测试具体实现

模糊测试是本系统最终的核心功能模块，承接规格解析与日志解析两个模块提供的上下文信息，借助增强后的种子库对待测 REST API 服务进行高强度、覆盖面广的自动化测试。该过程借助日志切片模拟真实用户的调用行为，并通过多种变异策略持续生成新的请求序列，以最大程度触发潜在的服务端异常和边界行为。本小节首先介绍模糊测试的主体流程实现，之后具体讲解模糊测试中变异算子和反馈的具体实现。

```
def fuzzing(self):
    i = 0
    while True:
        seed_curr = self.seed_queue[i]

        # 运行种子
        seed_curr.run(base_url=self.swagger.base_url)

        # 对种子进行变异
        for mutator in self.mutators:
            for i in range(0, 5):
                seed_mutated = mutator.mutate(seed_curr)
                # 补全种子
                self._complete_one_seed(seed=seed_mutated)
                # 运行变异后的种子
                seed_mutated.run(base_url=self.swagger.base_url)

                # 反馈：判断变异后的种子是否为系统感“兴趣”的种子
                add_or_not = self.feedback(seed_mutated)
                if not add_or_not: continue
                # 添加至种子队列
                self.seed_queue.append(seed_mutated)

        i += 1
        # 循环遍历种子队列
        if i == len(self.seed_queue): i = 0
```

图 4-22 模糊测试流程代码示例

图4-22所示代码为模糊测试整体流程的具体实现。在测试执行阶段，系统会遍历种子队列中的每一个种子，并对其应用各种变异算子的变异操作，生成多个变异后的测试用例。之后，系统执行变异后的测试用例，并根据预设的反馈规则判断是否将变异后的测试用例作为新的种子添加回种子队列，从而进一步优化种子库，增强测试覆盖面。图4-23中展示的代码即为本系统在执行反馈判定时的具体实现，确保变异生成的测试用例能够有效地推动后续测试过程的深入。该方

法会检查 Fuzzer 对象的 `state` 属性（用于存储当前运行状态），判断是否记录了当前变异后的测试用例执行所得到的 HTTP 状态码。如果未记录该状态码，系统会认为已发现新的状态，并将该测试用例添加到种子队列中，供后续测试使用。

```
def feedback(self, seed:Seed):
    """
    对种子执行结果进行分析，判断是否值得添加至种子队列
    """
    add_or_not = False
    for req in seed.req_seq:
        method = req.endpoint.method.value
        path = req.endpoint.path
        if req.rsp_status_code not in self.state.paths[path][method]:
            add_or_not = True
            self.state.paths[path][method].append(req.rsp_status_code)
    return add_or_not
```

图 4-23 模糊测试反馈代码示例

接下来将以修改参数值变异算子为例介绍本系统的变异算子的实现方式，该算子的具体实现代码如图4-24所示。ModifyParamMutator 类是 Mutator 抽象类的一个子类，通过实现 `mutate()` 方法来实现自己变异的功能。

`mutate()` 方法接收待变异的种子作为输入，并输出一个变异后生成的测试用例（同样为 `Seed` 类型）。为了避免对原始种子产生副作用，本方法首先通过深拷贝的方式复制种子对象，并将其中的请求序列构造成带有原始索引信息的元组列表，便于后续对请求元信息进行追踪和修改。随后，变异算子根据各请求已应用当前变异算子的次数对请求序列进行排序，选择应用次数最少的请求作为本轮变异的目标，以实现更均衡的测试覆盖。在选定的请求中，算法会随机选取一个已存在的参数，并对其值进行修改，从而生成具有差异化的新测试用例，为模糊测试提供更多可能触发异常的输入路径。

新的参数值是从一个候选集合中随机选取得到的，该集合包括规格文档中定义的默认值（`default`）、枚举值（`enum`），以及系统自行生成的随机值。其中，默认值和枚举值属于规格文档中定义的合法输入，适用于探测待测服务中尚未覆盖的功能路径；而系统生成的随机值则可能构成非法输入，用于测试系统在异常情况下的鲁棒性，从而挖掘潜在的缺陷或异常处理漏洞。需要注意的是，该变异算子仅针对查询参数和请求体中的参数进行修改，不会对路径参数进行变更。

```

class ModifyParamMutator(Mutator):

    def __init__(self, swagger: Swagger, logger):
        super().__init__(swagger, logger)

    def mutate(self, seed: Seed) -> Seed:

        # 复制种子, 并构造带有原始索引的元组列表
        seed_copy = copy.deepcopy(seed)
        indexed_req_seq = [(index, req) for index, req in enumerate(seed_copy.req_seq)]

        # 按照应用该变异算子的次数将原始序列排序
        sorted_list = sorted(indexed_req_seq, key=lambda item: item[1].modify_param_mutate_count)
        for idx, req in sorted_list:

            # 获取当前请求中存在的参数
            existed_params_dict = req.get_existed_params()

            # 随机选择其中一个参数
            if len(existed_params_dict) == 0: continue
            param = random.choice(list(existed_params_dict.keys()))

            # 1) 参数类型为query参数
            if param.startswith("#/query/"):
                name = param[len("#/query/"): ]
                if name in req.query_params:
                    # 生成一个值, 并修改原始的参数
                    req.query_params[name] = req.endpoint.query_parameters[name].gen_one_value()
                    seed.req_seq[idx].modify_param_mutate_count += 1
                    break

            # 2) 参数类型为body里的参数
            # 获取参数对应的Property
            prop = req.endpoint.body.get_property('/'.join(param.split('/')[3:]))
            # 生成一个参数值
            value = prop.gen_one_value()
            # 修改请求体中的参数
            req.body.insert_param_to_body('/'.join(param.split('/')[3:]), value)
            seed.req_seq[idx].modify_param_mutate_count += 1
            break

        return seed_copy

```

图 4-24 参数变异算子修改功能代码示例

原因在于路径参数通常具有依赖性, 其取值往往依赖于前置请求的实际响应结果, 随意修改可能破坏调用链的逻辑一致性, 影响测试的准确性和有效性。

变异算子在对输入种子进行变异后, 会在种子中记录相关的执行信息, 以引导后续的变异操作更加有针对性和有效性。在测试用例的执行过程中, 系统会优先根据请求中的依赖字典以及此前资源创建请求的响应结果, 填充请求中依赖性参数的取值, 从而确保测试用例中的请求序列符合 REST API 中常见的“生产者-消费者”关系。

4.4 运行实例展示

本节将以一个实际的待测 REST API 服务为例, 应用本系统开展测试操作, 全面展示系统在真实应用场景中的运行效果。具体而言, 将按照系统的运行流程

依次展开，涵盖日志准备、模糊测试的执行过程、各关键阶段所产生的中间产物，以及最终的测试结果，直观体现系统在实际测试任务中的功能表现与有效性。

4.4.1 运行准备：REST 博客服务及其 Nginx 网关日志

本次系统运行所使用的待测服务为一个 REST API 风格的博客服务，主要负责管理用户、博客、评论、点赞和话题五类资源。其中，话题与用户属于顶级资源，能够独立存在而无需依赖其他资源；博客的创建操作需由用户发起，但一旦创建成功，用户仅作为博客的一个属性存在，博客本身不再依附于用户；评论是博客的子资源；点赞是用户的子资源，其作用对象为某一特定的博客或评论。表4-1展示了该服务的接口信息，包括 URI 路径、涉及的 HTTP 方法及相关说明。

表 4-1 待测 REST API 博客服务接口列表

URI	HTTP 方法	解释
/user	GET, POST	负责获取用户列表、创建用户
/user/{id}	GET, PUT, DEL	负责针对某个特定用户的操作
/user/{id}/like	POST	负责用户的点赞操作
/blog	GET, POST	负责获取博客列表、创建博客
/blog/{id}	GET, PUT, DEL	负责针对某个特定博客的操作
/blog/{blogId}/comment	GET, POST	负责获取某个特定博客下的评论、创建评论
/blog/{blogId}/comment/{id}	GET, PUT, DEL	负责针对某个特定评论的操作
/topic	GET, POST	负责获取话题列表、创建话题
/topic/{id}	GET, DEL	负责针对某个特定话题的操作

本次测试所使用的日志为系统默认支持解析的 Nginx¹ 网关请求转发日志。该日志记录了历史请求的 URI、参数以及时间戳等关键信息，能够满足系统对日志内容的要求。图 4-25 展示了一个针对待测博客服务的 Nginx 网关日志示例。

```
{
  "request_url": "/blog?userId=1",
  "request_method": "GET",
  "request_headers": {
    ...
  },
  "request_body": "",
  "response_status": 200,
  "response_body": {
    "code": 0,
    "msg": "get blogs successfully",
    "data": [
      {
        "id": 1,
        "content": "it's a beautiful day!",
        "userId": 1,
        "likes": 0
      }
    ],
    "timestamp": 1744138752984
  }
}
```

图 4-25 待测博客服务的 Nginx 网关日志示例

¹ <https://nginx.org/>

4.4.2 关键功能：接口依赖推理与日志解析

将博客服务的规格说明文档（Swagger 2.0）和历史日志文件上传至系统后，系统会自动解析并提取其中的关键信息，生成两个核心产物：包含依赖关系的接口信息，以及经过补全处理的日志切片集合。

基于历史日志的 REST API 模糊测试系统在运行过程中，会遍历服务的各个接口，并使用大模型分析其参数是否存在对其他资源的依赖关系。图 4-26 展示了博客服务中一个典型接口的依赖关系推理结果。该图以“修改评论内容”接口（POST /blog/{blogId}/comment/{id}）为核心，呈现系统解析得到的相关依赖结构。该接口包含两个路径参数和一个请求体：路径参数“id”绑定至评论资源，“blogId”绑定至博客资源；请求体参数“userId”与用户资源相关联，而“content”为独立参数，不依赖其他资源。而这些被依赖的资源创建接口同样有可能依赖于其他资源：“创建评论”接口的路径参数“id”和请求体参数“userId”分别依赖于博客资源和用户资源；“创建博客”接口的请求体参数“userId”依赖于用户资源；而“创建用户”接口则不依赖于任何资源。通过这种方式，系统能够构建出接口之间的真实依赖网络，为后续请求序列的合理构造提供支持。

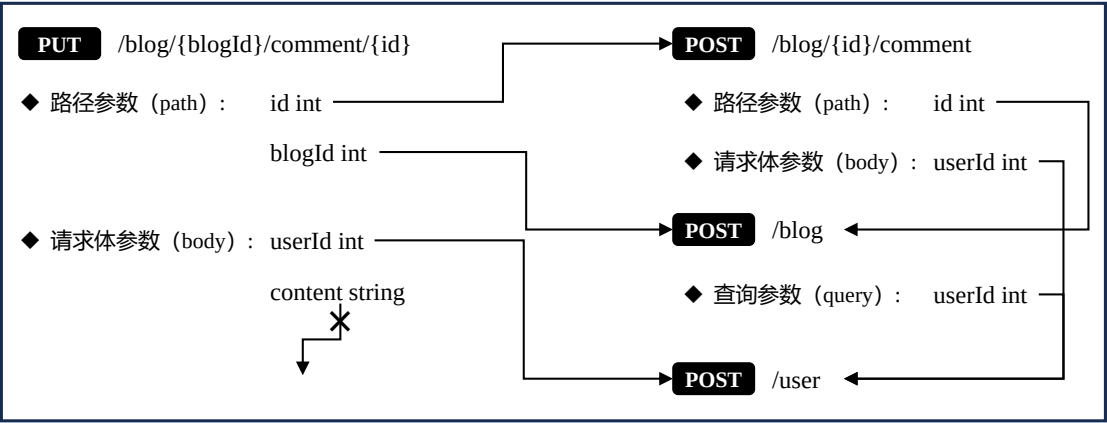


图 4-26 解析得到的待测博客服务局部接口依赖关系示例图

日志解析环节的核心目标是从历史日志文件中提取出可直接执行的测试用例，即还原出具备上下文逻辑的有效请求序列。图4-27展示了该过程在博客服务中的具体应用。图中最上方为用户 A 的部分日志记录。日志中间的三条请求反映了用户 A 在 2025 年 5 月 20 日的真实使用场景：查看编号为 23 的博客，对其发表评论，并进行点赞。由于这三条请求与其前后日志在时间上存在显著间隔，系统依据“HTTP 方法语义最大前置时间字典”策略，将其独立切分为一个日志

切片。然而，该日志切片存在缺失，未包含相关资源的创建请求，若直接执行，将因资源不存在而失败。为解决这一问题，系统结合此前推理得到的依赖关系，对日志切片进行补全，最终生成图中最下方所示的请求序列。该序列可作为后续模糊测试阶段的有效种子。

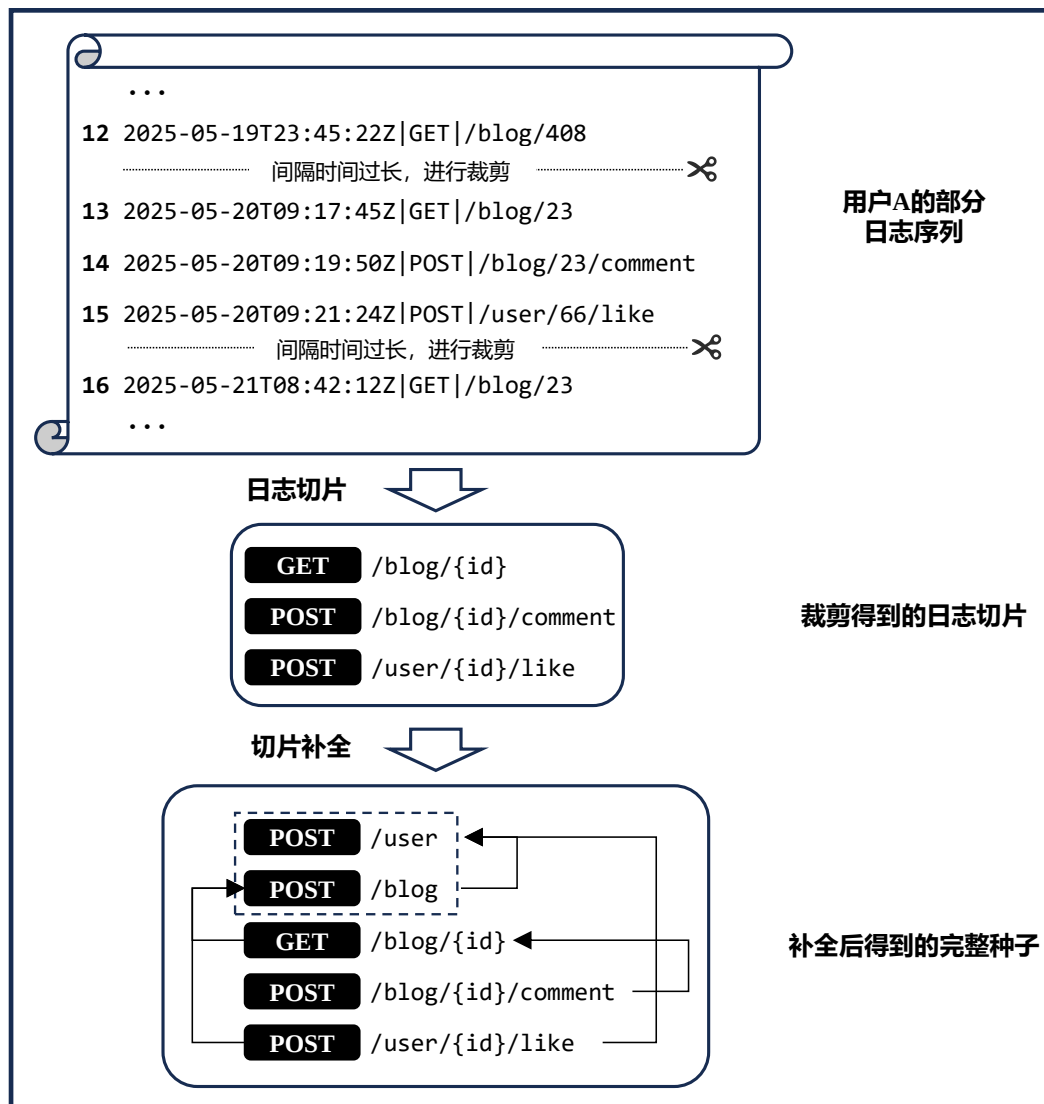


图 4-27 博客服务进行日志解析过程示例

4.4.3 开始测试：系统配置与模糊测试执行

完成规格文档与历史日志的解析后，系统便可启动对待测服务的模糊测试流程。图4-28展示了本次运行所使用的系统配置详情。在日志解析器（log_parser）的配置项中，明确指定日志文件类型为 Nginx 网关日志，并选择“all”作为日志切片策略，即启用全部可用的切分方式，生成的所有日志切片将被加入初始种子

库。具体来说，对不同类型的 HTTP 请求设置了最大前置时间：GET 为 5 分钟，PUT、POST、DELETE 均为 10 分钟；时间窗口大小设定为 10 分钟；固定序列长度为 4。在规格解析器（swagger）部分，本次未设置需忽略的接口标签，即默认解析所有标签下的接口信息。模糊测试器（fuzzer）配置中，设置每个变异算子在一次测试轮次中的使用次数为 2，确保快速遍历种子队列从而生成多样化的测试用例。大模型调用模块（llm）方面，本次运行选用智谱清言的 glm-4-plus 模型，辅助系统进行依赖关系推理与日志规格化处理任务。

```
log:
  level: debug # 日志级别, 支持的值: debug, info, warning, error, critical
  file_path: ./production/log/runtime.log # 系统运行日志路径

log_parser:
  type: nginx # 日志类型, 当前支持nginx和gitlab
  timewindow: 10m # 时间窗口长度
  fixed: 4 # 固定序列长度
  mlts: # 基于HTTP方法语义的最大前置时间字典
    get: 5m
    put: 10m
    post: 10m
    delete: 10m
  slice_strategy: all # 切片策略, 支持的值: mlt, timewindow, fixed, all

fuzzer:
  mutate_times_per_mutator: 2 # 变异算子每次运行次数

llm:
  engine: zhipu # 大模型引擎, 支持的值: zhipu, openai
  model: glm-4-plus # 模型类型

swagger:
  restricted: false # 是否限制swagger中的接口数量
```

图 4-28 本次运行时的系统配置

配置文件中的各项参数主要用于设置系统的核心超参数，通常不需频繁修改。而针对模糊测试过程中的常用配置，可通过启动命令灵活设置。当前模糊测试启动指令支持以下四个参数：1) `-cache`：启用缓存启动，跳过大模型分析步骤，以节省 Token 消耗并提升启动速度，适用于已有运行记录的场景；2) `-budget`：设定请求发送速率，用于控制系统的测试压力；3) `-time`：指定本轮模糊测试的持续时间；4) `-disable-log`：在不依赖历史日志的前提下，仅基于系统内部逻辑进行 REST API 测试。合理组合使用这些参数，可实现对模糊测试流程的高度定制化。完成参数设置后，即可启动模糊测试任务。

图 4-29 展示了基于历史日志的 REST API 模糊测试系统（RESTLOM）在运行过程中的控制台界面。该界面实时反馈系统的运行状态，便于测试人员直观掌握测试进展与效果。界面主要包含以下几类信息：1）运行统计信息：包括已运行时间、已执行的测试用例数和已发送的请求数，这些指标反映了测试强度与执行效率；2）测试进度信息：包括当前已完成的种子数量，已覆盖的接口数，以及最近执行的请求；3）执行速率：系统当前发送请求的速度；4）响应状态统计：按 HTTP 状态码分类展示了请求响应结果，包括 2XX（成功响应）、4XX（客户端错误）、5XX（服务器错误）；5）失败接口数量：可能存在潜在缺陷或安全风险的接口数量。通过该控制台界面，测试人员可以持续监控模糊测试的执行状态，并根据异常响应情况及时调整测试策略或深入排查具体接口问题

=====	
RESTLOM: Log-Based REST API Fuzzer	
=====	
Elapsed time:	1m2s
Number of executed tests:	448
Number of requests sent:	1598
Number of cycles completed:	0

Number of covered endpoints:	16/20
Current seed:	40/327
Last request:	put /user/{id}
Speed:	25.77 reqs/s

20X Responses:	1463
40X Responses:	76
50X Responses:	59
Number of failed endpoints:	1
=====	

图 4-29 基于历史日志的 REST API 模糊测试系统命令行运行界面

运行结束后，系统会自动生成一份详细的测试报告，将本次模糊测试的关键结果完整呈现给用户。该报告主要包括以下内容：1）测试概览：总结本轮测试的整体执行情况，包括总运行时间、发送请求数量、覆盖接口数量、使用的种子总数等；2）接口覆盖情况：列出所有被测接口的覆盖状态，标明哪些接口已被成功触达，哪些接口尚未覆盖，以及对应的请求示例；3）异常响应统计：对所有触发 5XX 响应进行分类统计，帮助用户快速定位潜在的请求构造缺陷；4）模糊测试过程日志：记录完整的测试过程与每个阶段的处理信息，增强结果的可追

溯性；5) 系统配置快照：保留本次测试所使用的配置参数和模型版本信息，方便后续复现测试环境。

4.5 本章小结

本章在系统概要设计和功能模块划分的基础上，对本系统进行了详细设计，并且展示了每个模块的重要功能实现过程以及具体算法和代码。

第4.1节介绍了规格解析模块的详细设计与实现，该模块作为系统的基础组件，负责解析用户提交的 Swagger 或 OpenAPI 文档，并提取其中的接口信息和依赖关系，为模糊测试提供结构化的数据支撑。在设计方面，模块采用统一的数据结构（如 Swagger、Endpoint、Schema 等）封装解析结果，提升了系统对不同版本 API 文档的兼容性和可维护性。在实现方面，模块通过递归解析算法实现对嵌套数据模型的结构化提取，并基于大语言模型提示词自动推理接口间的依赖关系，有效弥补日志数据不完整的问题。通过这些设计与实现，规格解析模块为后续种子生成提供了高质量的语义信息支撑。

第4.2节对日志解析模块展开了详细设计与实现说明。模块整体分为“多用户日志队列提取”与“日志切片生成”两个阶段，前者实现对异构日志格式的解析和用户操作轨迹重建，后者则通过多种切片策略生成具有上下文连续性的请求序列。在设计方面，模块采用模板方法和策略模式提升扩展性与灵活性，并引入大语言模型辅助解析未知日志格式。切片策略上，系统提供最大前置时间、时间窗口、固定长度及混合策略，用户可根据实际场景灵活配置，从而增强种子生成的准确性与多样性。

第4.3节介绍了模糊测试模块的设计与实现。该模块包括种子增强、变异操作和测试执行。种子增强包括扩增和补全，提升种子多样性。核心调度器 Fuzzer 管理测试流程，变异算子通过策略模式支持扩展。在测试阶段，系统变异种子并执行测试，发现新行为时将种子加入队列，推动后续测试。

最后4.4节通过对一个 REST 博客服务的测试，展示了本系统运行的效果。

第五章 REST API 模糊测试系统的测试与评估

5.1 测试设计

为全面评估系统的稳定性与功能完整性，本节从多个层面开展了测试工作，涵盖单元测试、集成测试与系统测试三个阶段。单元测试聚焦于核心功能的逻辑正确性；集成测试旨在验证各模块间的协同与数据交互是否顺畅；系统测试则评估整体系统在真实使用场景下对用户需求的支撑能力。以下将依照测试层次详细说明各项测试的设计方法与测试结果，测试内容均紧扣需求分析阶段提出的功能目标。

5.1.1 测试准备

测试目的。本次测试的核心目标在于全面验证系统在功能实现及用户体验等方面是否满足需求规格说明书中所规定的各项要求，确保系统能够稳定运行。作为一个自动化测试工具，本系统的测试工作需从两个维度展开：一方面，需要验证系统是否能够完整实现基于历史日志的 REST API 模糊测试相关功能，包括规格说明解析、历史日志处理、种子库增强、模糊测试执行以及测试报告生成等关键环节；另一方面，还需评估系统在鲁棒性、可用性和可扩展性等软件质量属性上的表现。

测试环境。本文进行的单元测试、集成测试、系统测试均在同样的测试服务器上进行，表5-1展示了测试服务器的硬件配置以及操作系统。

表 5-1 测试服务器硬件及操作系统信息

配置项	配置属性
CPU	Intel(R) Xeon(R) Silver 4316 CPU @ 2.30GHz
CPU 核心数	80 逻辑核心，2 物理核心
运行内存	503GB
操作系统	Ubuntu 20.04.6 LTS

本系统需重点关注运行内存和 CPU 的使用情况。首先，系统支持对大型 REST API 服务的规格说明文档进行解析，并将解析结果存储于内存中；此外，还需维护体积庞大的模糊测试种子库和种子队列，因此内存占用可能较高。另一方面，系统需与 REST API 服务建立连接，执行请求发送、响应解析及后续处理，这些操作均对 CPU 资源有较高依赖。实际测试结果表明，系统在正常运行状态下，对测试服务器的 CPU 与内存占用均未超过 15%。

表5-2列出了本系统运行所需的软件环境及关键依赖包。本系统采用 Python 语言开发，为避免第三方包之间或与 Python 本身发生版本冲突，使用 Conda 创建虚拟环境进行项目部署，虚拟环境中的 Python 版本为 3.12.9。系统主要依赖于 requests 库用于发送 HTTP 请求，并通过 SDK 方式调用大模型 API，所用 SDK 包括 zhipuai（智谱清言）和 openai（ChatGPT）。

表 5-2 软件环境依赖

配置项	配置版本
Conda	25.1.1
Python	3.12.9
requests	2.32.3
zhipuai	2.1.5.20230904
openai	1.58.1

5.1.2 单元测试

单元测试（Unit Testing）是对程序单元（即软件开发过程中最小的可测试功能模块）进行正确性验证的基础测试环节，又被称为模块测试，旨在确保各个功能单元在独立运行时能够按照设计规格正确执行。在面向对象的软件开发过程中，单元通常指一个方法、一个类，或是一段实现特定业务逻辑的代码。通过对这些基本单元进行细致的测试，开发者可以在早期发现潜在的逻辑错误或实现偏差^[50]。本文在开发过程中对各核心功能模块、关键逻辑流程和接口操作步骤均进行了多轮单元测试，确保每个模块在独立执行时均能稳定输出预期结果。

表5-3展示了本系统在测试环节所进行的单元测试，包括测试用例 ID、测试单元（通常是一个函数或者方法）、测试功能以及测试结果。这些单元测试覆盖了系统的核心功能模块，旨在验证其中每个功能单元的正确性，进而确保系统在

各个阶段能够满足预期的设计要求。本系统采用 Python 语言开发，因此，所有单元测试均基于 Python 的测试框架 `pytest`¹ 进行编写和执行。

表 5-3 单元测试用例列表

用例 ID	测试单元	测试用例描述	结果（成功/失败）
UT1	<code>_parse_base_url</code>	解析服务基址	3/0
UT2	<code>_parse_schemas</code>	解析数据模型	4/0
UT3	<code>_parse_paths</code>	解析接口信息	4/0
UT4	<code>_analyse_dependencies</code>	依赖关系推理	10/0
UT5	<code>_log_preprocess</code>	日志规格化	26/0
UT6	<code>_gen_log_slices</code>	日志切片	4/0
UT7	<code>build_concrete_req</code>	具体请求构造	18/0
UT8	<code>_judge_and_add_req</code>	请求缺失判断并补全	3/0
UT9	<code>get_existed_params</code>	获取请求存在的参数	15/0
UT10	<code>gen_one_value</code>	随机生成一个基本类型	10/0
UT11	<code>get_property</code>	根据路径获取属性	3/0
UT12	<code>insert_param_to_body</code>	将参数插入请求体	5/0
UT13	<code>_update_dependent_params</code>	更新依赖性参数	3/0
UT14	<code>Req.send</code>	发送请求到待测服务	10/0
UT15	<code>LLM.send</code>	发送请求到大模型	5/0

5.1.3 集成测试

完成单元测试并确保细粒度的代码单元功能无误后，测试工作进入到集成测试阶段。集成测试（Integration Testing）旨在验证多个模块协同工作时的功能正确性与数据交互的合理性。该阶段主要关注模块间接口调用是否正确、数据是否能在模块间顺利传递，以及整体流程逻辑是否符合系统设计要求^[50]。

表5-4列出了对规格说明文档解析功能的集成测试细节。为了验证系统的鲁棒性，本文准备了各种格式（YAML 或者 JSON）和版本（Swagger 或者 OpenAPI）的 REST API 规格说明文档，并通过系统配置文件声明其类型，随后上传至规格解析模块进行解析。系统在正常运行下应能够成功提取文档中的基本接口信息并加载至内存中；同时，与大模型支持模块交互，推理得到接口参数可能存在的资源依赖关系。

¹ <https://docs.pytest.org/en/stable/>

表 5-4 规格说明文档解析测试设计

测试 ID	IT1
测试名称	规格说明文档解析测试
测试功能	验证系统是否能够正确解析用户上传的规格文档
测试步骤	1. 测试人员准备待测服务规格说明文档； 2. 测试人员在配置文件中声明文档格式； 3. 测试人员配置大语言模型 API 密钥； 4. 上传文件到系统开始解析
预期结果	正确解析文档中的接口基本信息到内存中；成功通过大模型获取接口参数的依赖关系
涉及的模块	规格解析模块、大模型支持模块

表5-5展示了历史日志解析功能的集成测试详情。本文首先准备待测服务的历史日志文件，目前系统支持的日志类型包括 Nginx 网关请求转发日志和 GitLab 日志。随后，通过配置文件设置日志切片策略，可选策略包括基于前置字典的切分、基于时间窗口的切分以及基于固定序列长度的切分。在正常运行条件下，系统应能够正确提取日志内容，并按照设定策略将其划分为多个日志切片，作为后续模糊测试的初始种子库。

表 5-5 历史日志解析测试设计

测试 ID	IT2
测试名称	历史日志解析测试
测试功能	验证系统能否正确解析用户上传的服务端历史日志
测试步骤	1. 测试人员准备待测服务历史日志文件； 2. 测试人员配置日志的类型以及解析采用的策略； 3. 测试人员配置大语言模型 API 密钥； 4. 上传文件到系统开始解析
预期结果	成功提取日志中的信息并按照切片策略将日志文件切分为日志切片集合
涉及的模块	日志解析模块、大模型支持模块

表5-6描述了系统中种子库增强功能的集成测试流程与关键测试点。该模块旨在基于初始日志切片种子库，生成更加丰富且覆盖面更广的测试种子集合，从而提升之后模糊测试的全面性和有效性。在测试过程中，本文预先提供经过历史日志解析模块处理后的初始种子切片列表。系统随后利用规格说明文档解析模块提供的接口结构、参数依赖等信息，对原始种子进行扩展与构造，力求覆盖 REST API 文档中所有关键接口。测试目标主要包括两方面：（1）验证系统能否正确识别初始种子中未覆盖的接口或参数；（2）评估生成的增强种子在结构完

整性与业务合理性上的有效性。正常情况下，系统应能成功构建一个面向接口全覆盖、结构合理、具备模糊测试价值的增强种子库。

表 5-6 种子库增强测试设计

测试 ID	IT3
测试名称	种子库增强测试
测试功能	验证系统能否正确补充种子缺失的请求以及种子库未覆盖的种子
测试步骤	1. 测试人员提供日志切片集合（初始种子库）； 2. 系统进行种子扩增； 3. 系统进行种子补全；
预期结果	成功获取增强种子库用于后续测试
涉及的模块	规格解析模块、日志解析模块、模糊测试模块

最后，本文通过集成测试的方法，全面验证模糊测试执行流程的正确性与完整性。测试过程中重点关注以下几个方面：变异算子是否被合理且有效地应用于种子的生成与扩展；执行组件是否能够准确构造测试用例并顺利发送至待测服务；反馈组件是否能够及时捕获服务响应中的关键信息，并将具有测试价值的新变异种子正确添加至种子队列中。

表 5-7 模糊测试验证测试设计

测试 ID	IT4
测试名称	模糊测试验证
测试功能	验证模糊测试整体流程能否正确运行
测试步骤	1. 测试人员提供种子库； 2. 测试人员配置变异算子； 3. 系统开展 REST API 模糊测试；
预期结果	变异组件、执行组件、反馈组件表现正确
涉及的模块	模糊测试模块、实时展示模块、持久化模块

5.1.4 系统测试

单元测试和集成测试关注的是程序组件的正确性，目的是在开发早期发现局部模块中的功能性缺陷，确保各个模块在独立运行或组合调用时均能按照设计预期正确执行。而系统测试（System Testing）则面向整个系统，聚焦于功能的完整性。它通常在软件开发后期进行，通过模拟用户的实际操作流程或业务场景，验证系统是否能够在各种输入和操作条件下稳定运行、正确响应^[50]。

系统测试部分，本文在真实世界项目中，以多种不同配置运行本系统以验证其功能完整性与鲁棒性。表格展示了系统测试阶段执行的各项测试用例，内容包括测试用例 ID、测试描述、测试目标及实际测试结果。

表 5-8 系统测试用例列表

用例 ID	测试用例描述	测试目标	测试结果
ST1	正常流程下的模糊测试	成功执行整个流程	成功
ST2	日志切片策略影响验证	切片策略符合预期	成功
ST3	禁用日志进行测试	回退至纯规格驱动的模糊测试	成功
ST4	使用缓存进行测试	缓存成功启用，跳过解析阶段	成功

5.2 实验设计

本文创新性地提出了一种基于历史日志的 REST API 模糊测试技术，并据此构建了可运行的模糊测试系统 RESTLOM。为了系统性地评估该技术的有效性与实际表现，本文设计并实施了一系列实验。

本次实验主要聚焦以下四个关键问题：

1) 方法有效性分析：相较于现有测试工具，基于历史日志的 REST API 模糊测试方法是否能够显著提升测试效果和错误发现能力；

2) 消融实验：在模糊测试流程中剔除历史日志信息，评估日志数据对整体测试覆盖率与结果质量的具体贡献；

3) 策略选择影响：采用不同的日志切分策略（如基于语义、时间窗口、固定长度等）对系统运行效率与测试效果有何差异；

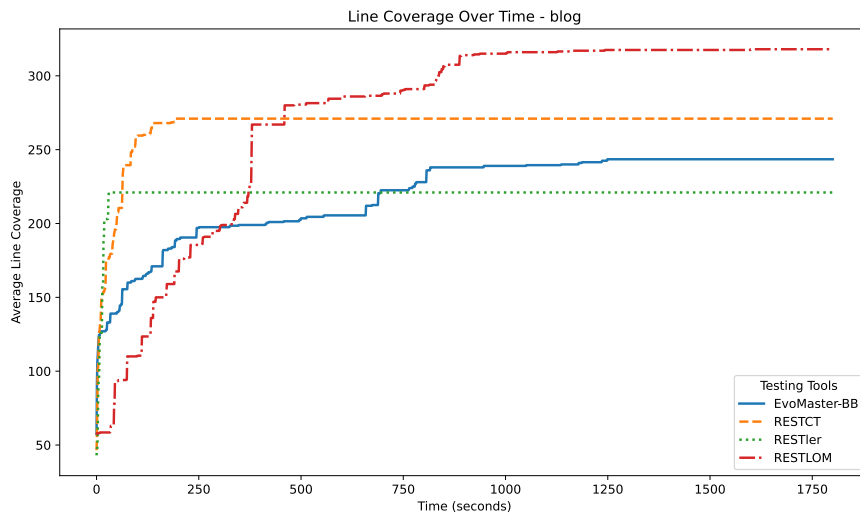
4) 大模型组件验证：引入大语言模型辅助接口参数依赖推理是否准确可靠，并分析其在系统中的实际价值。

由于模糊测试技术存在随机性，本文的所有实验均重复三轮。

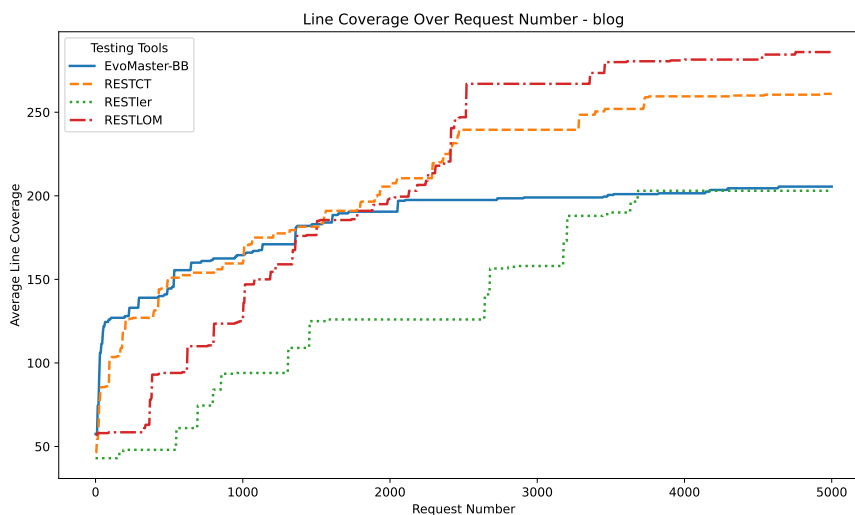
5.2.1 有效性分析：代码覆盖、缺陷以及响应状态码

本小节在同一 REST API 服务上对比运行多种测试工具，包括本文提出的 RESTLOM 以及现有主流工具 RESTler、EvoMaster 和 RESTCT。实验所使用的工具版本如下：RESTler 版本为 commit-aa5b963，EvoMaster 版本为 3.2.0（运行

于黑盒模式), RESTCT 版本为 commit-1e96602。待测服务为一个博客类 REST API 服务, 包含 20 个接口, 涉及用户、话题、博客、评论、点赞等多种资源的管理操作。为保证实验公平性, 所有测试工具与待测服务均以容器化方式部署, 底层镜像统一为 Ubuntu 22.04, 实验环境与测试环境配置保持一致。此外, 为避免服务插装对请求处理性能的干扰, 本实验采用“先运行工具, 后插装重放”的方式获取测试过程中各工具的覆盖率信息。



(a) 代码覆盖随时间变化



(b) 代码覆盖随请求数变化

图 5-1 博客服务代码行覆盖变化

图 5-1 展示了博客服务在测试过程中代码行覆盖数量的变化情况。从图中可以看出, 本文提出的 RESTLOM 在最终覆盖的代码行数上明显优于其他工具, 相比之下提升幅度达 22%-43%。这一结果充分验证了本文基于历史日志的 REST

API 模糊测试技术在提升测试有效性方面的显著优势。

图 5-1a 展示了不同工具在测试过程中代码覆盖随时间推移的增长趋势。可以观察到,其他工具在测试初期覆盖率上升较快,主要原因有两点:其一,RESTler 与 RESTCT 的请求发送速率较高,且无法进行速率限制;其二,这些工具多采用参数组合等策略,在早期能够快速探索大量等价类组合,从而覆盖更多路径。相比之下,RESTLOM 的策略更侧重于对历史日志中真实请求的复用和渐进式变异,因此其初期增长相对平缓,但随着时间推进,其覆盖能力持续提升。

考虑到不同工具请求发送速率的差异,本实验还统计了代码覆盖随请求数的变化情况,如图 5-1b 所示。从图中可以发现,RESTCT 和 EvoMaster 在请求初期具有较高的请求质量,能迅速带来较大覆盖增长;而 RESTLOM 与 RESTler 则略显缓慢。然而,随着测试持续进行,RESTCT 和 EvoMaster 均逐渐进入覆盖平台期,新增覆盖变得稀少。而 RESTLOM 凭借其模糊测试引擎对种子的持续变异与反馈机制,仍能持续挖掘出新的覆盖路径,最终实现更高的代码覆盖率。

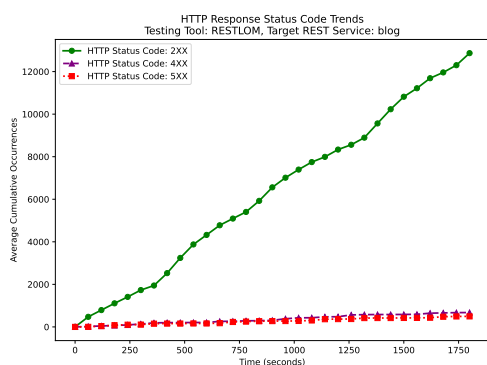
表 5-9 不同工具发现缺陷数统计

测试工具	发现缺陷的接口
RESTLOM	/user; /blog; /blog/{id}
RESTCT	无
RESTler	/blog/{id}
EvoMaster-BB	无

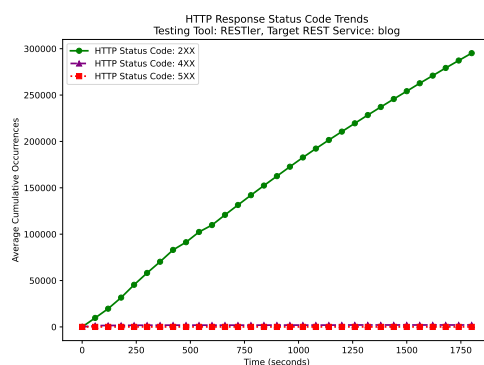
由表 5-9 可见,本文提出的 RESTLOM 能够在多个接口中成功发现缺陷,包括用户相关接口 (/user) 与博客相关接口 (/blog 及 /blog/id),表现优于其他对比工具。RESTler 虽然也在 /blog/id 接口中触发了异常响应,但其测试范围和深度明显不及 RESTLOM;而 RESTCT 和 EvoMaster (黑盒模式)在本次实验中均未能有效触发服务缺陷。

除了代码覆盖率和缺陷发现能力,请求的有效性同样是评估 REST API 测试工具的重要维度之一。如果测试工具生成的大量请求触发了 4XX 状态码,说明这些请求未能有效进入服务的业务逻辑层,未能产生实质性的测试效果,甚至会造成服务器资源的严重浪费。图 5-2 展示了四种测试工具在运行过程中返回的 HTTP 响应状态码随时间的变化趋势。从图中可以看出,RESTLOM 和 RESTler 的请求状态分布较为合理,均呈现出以 2XX 状态码为主,辅以少量 4XX 和 5XX

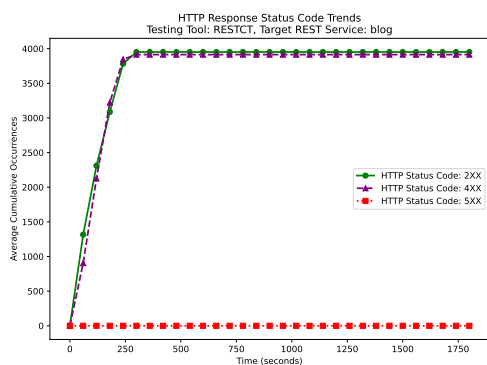
状态码的健康分布，说明其生成的请求大多数能够成功被服务接受并处理。相比之下，EvoMaster 的请求则表现得极为保守，几乎没有产生任何 4XX 响应，说明其测试生成策略偏向于“安全”路径，可能错过了一些边界场景。而 RESTCT 的表现则明显不佳，其生成的请求中有超过 50% 返回了 4XX 状态码，意味着其一半的测试用例是无效请求，未能有效参与服务的逻辑测试，资源利用效率低下。



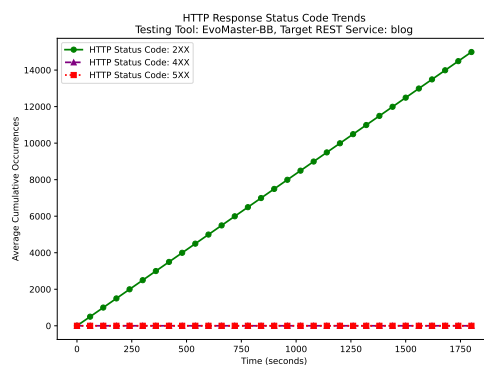
(a) RESTLOM 状态码随时间变化



(b) RESTler 状态码随时间变化



(c) RESTCT 状态码随时间变化



(d) EvoMaster-BB 状态码随时间变化

图 5-2 响应状态码随时间变化

综上，RESTLOM 在代码行覆盖上取得了最高的覆盖率，提升幅度达 22%–43%，并在多个接口中成功触发缺陷，显著优于其他对比工具。此外，RESTLOM 所生成请求的状态码分布合理，体现出良好的请求质量，避免了资源浪费。

5.2.2 消融实验：日志信息的作用

为进一步探究历史日志信息在 RESTLOM 中所起的实际作用，本小节设计了一项消融实验。我们通过在系统启动参数中配置 `--disable-log`，显式关闭日志信息的参与，以此对比分析日志信息对测试覆盖效果的具体贡献。图5-3 展示了 RESTLOM 在使用日志信息与禁用日志信息两种模式下运行测试过程中的代码

覆盖变化情况。由于两种配置均使用相同的测试工具，其请求发送速率和执行逻辑保持一致，因此图5-3a（代码覆盖随时间变化）与图5-3b（代码覆盖随请求数变化）呈现出高度一致的趋势。

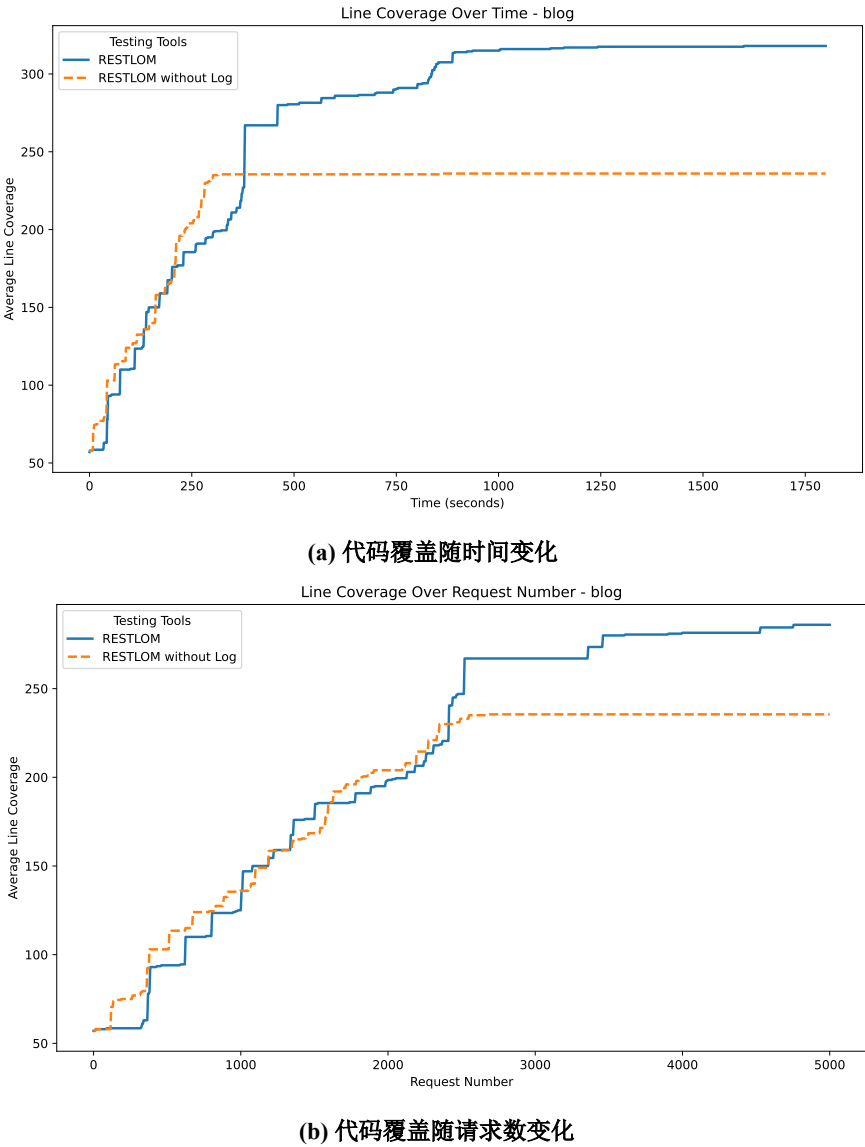


图 5-3 RESTLOM 使用与不使用日志信息的代码覆盖对比

从图中可以观察到，在测试初期，两种模式下的代码覆盖差异较小，说明基础变异策略在初始阶段能够实现一定的覆盖。然而，随着测试的持续进行，使用日志信息的配置在中后期展现出了明显的优势，最终覆盖的代码行数显著高于不使用日志的配置。本次消融实验充分验证了日志信息作为 RESTLOM 核心组件之一，在提升测试效果方面具有不可替代的价值。

5.2.3 策略选择：日志切分策略的差异

本小节旨在通过实验分析不同日志切分策略对 RESTLOM 测试效果的影响。由于实验仅涉及 RESTLOM 工具，为加快测试进程并提高种子队列的遍历效率，统一将请求速率设置为 3000 条请求/分钟。

实验对比了五种日志切分策略在博客服务上的表现，具体包括：1) RESTLOM: 基于 HTTP 语义的最大前置字典切分；2) RESTLOM-TimeWindow: 基于时间窗口的切分方式；3) RESTLOM-FixedLength: 固定长度切分策略；4) RESTLOM-ALL: 融合所有切分策略；5) RESTLOM without log: 完全不使用日志信息。上述策略分别在相同测试条件下运行 10 分钟，最终的代码覆盖情况如图 5-4 所示。

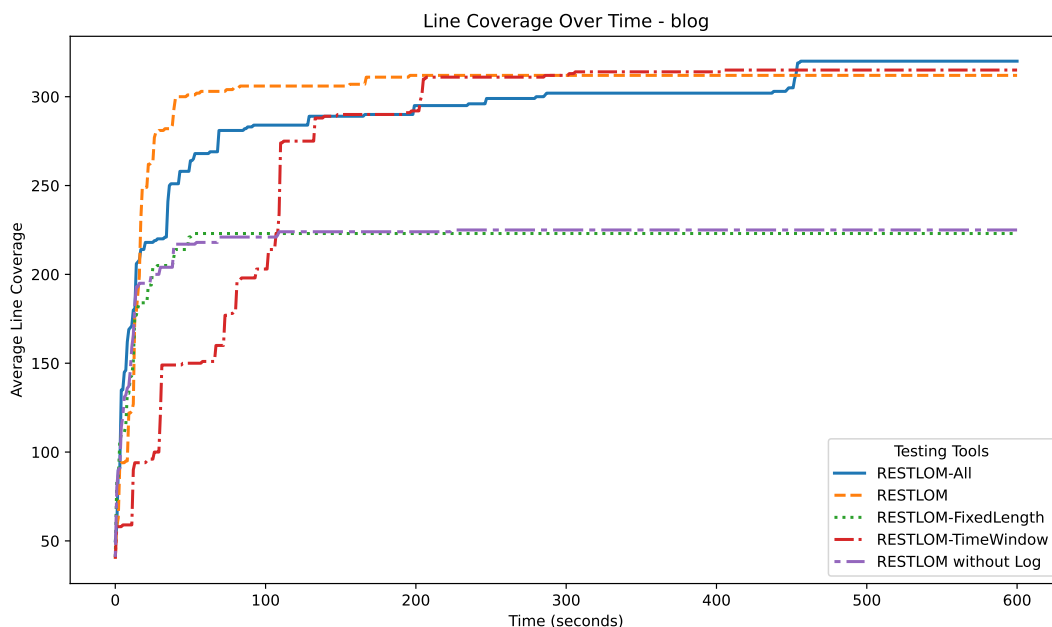


图 5-4 不同日志切分策略的测试效果对比

从图中可以观察到，RESTLOM-ALL（融合策略）、RESTLOM（最大前置字典）和 RESTLOM-TimeWindow（时间窗口）三种策略在测试结果上整体表现相近，最终代码覆盖率较高。其中，RESTLOM-ALL 略优一筹，显示出多策略融合在提升种子多样性方面的优势，但是该方式由于种子队列过程，达到相同的覆盖数的运行时间长于另外两种策略。相比之下，RESTLOM-FixedLength（固定长度切分）的表现最为不理想，其最终代码覆盖率甚至低于不使用任何日志信息的 RESTLOM without log 模式。这一结果表明，固定长度的切分方式未能有效保留

日志中请求之间的依赖关系，甚至可能引入不连贯或无效的请求序列，进而影响模糊测试引擎的种子变异效率和覆盖能力。

合理的切分策略应结合日志的结构特点与请求的语义关系，优先选择如最大前置字典或时间窗口等能够保持请求语义完整性的方法。同时，多策略融合（RESTLOM-ALL）在某些场景下也具有提升覆盖率的潜力，在测试时间允许的情况下可以考虑使用。

5.2.4 组件验证：大模型任务准确性

本文提出的基于历史日志的 REST API 模糊测试技术依赖大语言模型对接口参数间的依赖关系进行推理，因此有必要验证大模型在该任务上的能力。本小节设计了一项评估实验，用以系统性地测试不同大模型在 REST API 参数依赖推理任务中的准确性。

本实验构建了一个人工设计的评估基准，该基准由若干依赖推理任务组成，每个任务对应目标服务的一个接口。对于每个接口，根据其规格说明文档人工手动标注出正确的依赖关系作为参考答案，并设计提示词（参见图 4-7）引导大模型进行推理。模型的输出与人工标注答案进行比对，计算准确率，以衡量其在该任务上的表现。本实验为三类 REST API 服务（博客系统、GitLab Branch 模块、GitLab Groups 模块）分别构建了对应的评估集，并对三种大模型——glm-4-plus、glm-4-flash 与 gpt-4o——进行了系统性测试与对比分析。

表 5-10 不同大模型在依赖推理任务上的表现（正确/失败）

任务 模型	blog	Gitlab Branch	Gitlab Groups
glm-4-plus	20/0	9/0	17/0
glm-4-falsh	6/14	4/5	6/11
gpt-4o	20/0	9/0	17/0

表 5-10 展示了本实验中各大模型在依赖推理评估基准上的表现结果。从表中可以看出，glm-4-plus 与 gpt-4o 在所有评估任务中均取得了满分成绩，成功推理出全部接口参数依赖关系。而相较之下，glm-4-flash 的准确率明显较低，仅能部分识别参数间的依赖关系，说明其在该任务上的能力存在一定差距。综合准确性与便利性考虑，本文最终选用 glm-4-plus 作为系统默认配置的大语言模型，并

在系统开发过程中基于该模型不断迭代和优化提示词设计。

5.3 本章小结

本章对基于历史日志的 REST API 模糊测试系统进行测试，对基于历史日志的 REST API 模糊测试技术进行实验评估。

测试部分从单元测试、集成测试和系统测试三个层面进行验证。单元测试聚焦于代码中的基本功能单元（如函数或方法），确保其行为符合预期；集成测试通过设计四类典型测试用例，全面检验了系统各模块的功能实现及其交互逻辑；系统测试则在多个实际 REST API 服务上运行完整系统，验证其在真实应用场景下的稳定性与可用性。

实验评估则重点围绕四个核心问题展开：方法有效性、日志信息的贡献、日志切分策略的影响以及大模型组件的性能表现。方法有效性评估表明，在相同的待测服务上，RESTLOM 相比现有三种主流工具，其最终代码覆盖率提升了 22%-43%，并成功发现两个额外缺陷，同时生成的大多数请求均为有效请求；消融实验结果显示，日志信息能够有效辅助模糊测试生成包含真实用户行为特征的测试用例，从而提升覆盖率与缺陷发现能力；策略选择实验结果表明，基于 HTTP 语义的最大前置字典策略与基于时间窗口的切分策略在测试效果上显著优于固定长度切分方式；大模型组件评估实验表明，glm-4-plus 与 gpt-4o 能够准确完成所有接口依赖关系推理任务，表现稳定可靠，而 glm-4-flash 的准确率明显偏低，难以胜任该任务。

第六章 总结与展望

6.1 总结

随着网络接口在人们日常生活和企业运营中的广泛应用，确保其可靠性已成为不可忽视的关键问题。已有主流的 REST API 测试工具多以接口规格说明文档为信息来源，基于文档内容自动生成测试用例。然而，此类文档往往由人工编写或工具自动生成，存在质量参差不齐、信息不完整等问题，从而在一定程度上影响了测试效果与测试效率。

为克服这一局限，本文创新性地提出了一种基于历史日志的 REST API 模糊测试技术。该方法通过从历史请求日志中截取包含真实用户行为的请求序列片段，并将其转化为模糊测试用例，从而在更贴合实际使用场景的基础上提升接口测试的有效性与缺陷发现能力。

本文首先概述了基于历史日志的 REST API 模糊测试技术的研究背景与意义；随后，系统调研并综述了当前学术界与工业界主流的 REST API 测试工具，总结其设计思想与适用场景；在此基础上，开展系统的需求分析与概要设计，识别系统的关键涉众与用例，并通过功能模块划分与“4+1 视图”模型完成系统架构设计；接着，针对每个功能模块，详细说明了其设计思路与实现方式，并结合示例演示了系统的实际运行过程；最后，通过单元测试、集成测试与系统测试等多层次手段，对系统的功能正确性与稳定性进行了全面验证。

此外，本文还围绕方法有效性展开了多维度的实验评估。实验结果表明，相较于现有主流工具 RESTler、RESTCT 与 EvoMaster，所提出的方法在相同测试项目上的最终代码覆盖率提升了 22%-43%，并额外发现了两个其他工具未能识别的服务缺陷，充分验证了该方法在提升 REST API 测试效果方面的显著优势。

6.2 展望

尽管本文提出的基于历史日志的 REST API 模糊测试技术在测试效果上取得了显著提升,但仍存在一些值得进一步优化和扩展的方向。未来的研究工作可在以下几个方面展开:

1) 日志信息提取策略优化: 本文已探索了基于 HTTP 方法语义的最大前置时间字典、时间窗口、固定长度等三种日志切分策略。未来可进一步研究基于参数依赖关系的日志切分方法,从日志中识别请求之间的显式或隐式依赖,以构建更加结构化、具有语义连贯性的测试用例。

2) 模糊测试能量调度机制: 目前系统对每个种子请求的变异次数为固定值,未能充分体现测试用例之间的差异。后续可引入动态能量调度机制,根据测试用例的历史执行反馈(如覆盖增长或异常触发情况)动态调整其变异频次,对高质量种子分配更多资源,对低质量种子减少投入,以更高效地利用系统资源。

3) 日志类型分析: 日志来源的多样性可能会对测试效果产生不同影响。未来可系统地比较和分析不同类型日志(如来自真实生产环境的用户请求日志、测试环境中的人工测试日志、其他自动化测试工具生成的测试轨迹日志)在测试用例构造中的价值与表现,以指导更合理的日志选取策略。

参考文献

- [1] OFOEDA J, BOATENG R, EFFAH J. Application programming interface (API) research: A review of the past to inform the future[J]. International Journal of Enterprise Information Systems (IJEIS), 2019, 15(3): 76-95.
- [2] NEWMAN S. Building microservices: designing fine-grained systems[M]. 2021.
- [3] RAJESH R. Spring microservices[M]. 2016.
- [4] DI FRANCESCO P, LAGO P, MALAVOLTA I. Architecting with microservices: A systematic mapping study[J]. Journal of Systems and Software, 2019, 150: 77-97.
- [5] Swagger[Z]. <https://swagger.io>. 2025.
- [6] OpenAPI[Z]. <https://www.openapis.org>. 2025.
- [7] LARANJEIRO N, AGNELO J, BERNARDINO J. A black box tool for robustness testing of REST services[J]. IEEE Access, 2021, 9: 24738-24754.
- [8] ARCURI A. RESTful API automated test case generation with EvoMaster[J]. ACM Transactions on Software Engineering and Methodology (TOSEM), 2019, 28(1): 1-37.
- [9] KARLSSON S, ČAUŠEVIĆ A, SUNDMARK D. QuickREST: Property-based test generation of OpenAPI-described RESTful APIs[C]//2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST). 2020: 131-141.
- [10] HATFIELD-DODDS Z, DYGALO D. Deriving semantics-aware fuzzers from web API schemas[C]//Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings. 2022: 345-346.

- [11] ATLIDAKIS V, GODEFROID P, POLISHCHUK M. Restler: Stateful REST API fuzzing[C]//2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE). 2019: 748-758.
- [12] VIGLIANISI E, DALLAGO M, CECCATO M. Resttestgen: automated black-box testing of RESTful APIs[C]//2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST). 2020: 142-152.
- [13] LIU Y, LI Y, DENG G, et al. Morest: Model-based RESTful API testing with execution feedback[C]//Proceedings of the 44th International Conference on Software Engineering. 2022: 1406-1417.
- [14] WU H, XU L, NIU X, et al. Combinatorial testing of RESTful APIs[C]//Proceedings of the 44th International Conference on Software Engineering. 2022: 426-437.
- [15] ATLIDAKIS V, GEAMBASU R, GODEFROID P, et al. Pythia: Grammar-based fuzzing of REST APIs with coverage-guided feedback and learning-based mutations[J]. arXiv preprint arXiv:2005.11498, 2020.
- [16] KIM M, CORRADINI D, SINHA S, et al. Enhancing REST api testing with NLP techniques[C]//Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis. 2023: 1232-1243.
- [17] MARTIN-LOPEZ A. AI-driven web API testing[C]//Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Companion Proceedings. 2020: 202-205.
- [18] Burp Suite[Z]. <https://portswigger.net/burp>. 2025.
- [19] BooFuzz[Z]. <https://github.com/jtpereyda/boofuzz>. 2025.
- [20] CLAESSEN K, HUGHES J. QuickCheck: a lightweight tool for random testing of Haskell programs[C]//Proceedings of the fifth ACM SIGPLAN international conference on Functional programming. 2000: 268-279.
- [21] FERRANTE J, OTTENSTEIN K J, WARREN J D. The program dependence graph and its use in optimization[J]. ACM Transactions on Programming Lan-

- guages and Systems (TOPLAS), 1987, 9(3): 319-349.
- [22] RODRIGUEZ M A, NEUBAUER P. The graph traversal pattern[G]//Graph data management: Techniques and applications. 2012: 29-46.
- [23] KUHN D R, WALLACE D R, GALLO A M. Software fault interactions and implications for software testing[J]. IEEE transactions on software engineering, 2004, 30(6): 418-421.
- [24] KUHN D R, KACKER R N, LEI Y. Introduction to combinatorial testing[M]. 2013.
- [25] NIE C, LEUNG H. A survey of combinatorial testing[J]. ACM Computing Surveys (CSUR), 2011, 43(2): 1-29.
- [26] ALI S, BRIAND L C, HEMMATI H, et al. A systematic review of the application and empirical investigation of search-based test case generation[J]. IEEE Transactions on Software Engineering, 2009, 36(6): 742-762.
- [27] ARCURI A. Many independent objective (MIO) algorithm for test suite generation[C]//Search Based Software Engineering: 9th International Symposium, SSBSE 2017, Paderborn, Germany, September 9-11, 2017, Proceedings 9. 2017: 3-17.
- [28] ZHANG M, ARCURI A. Open problems in fuzzing RESTful APIs: A comparison of tools[J]. ACM Transactions on Software Engineering and Methodology, 2023, 32(6): 1-45.
- [29] BERNERS-LEE T, FIELDING R, FRYSTYK H. Hypertext transfer protocol—HTTP/1.0[R]. 1996.
- [30] FIELDING R T. Architectural styles and the design of network-based software architectures[M]. 2000.
- [31] 周芯宇, 陈伟, 吴国全, 等. REST API 设计分析及实证研究[J]. 软件学报, 2022, 33(09): 3271-3296.
- [32] KIM M, XIN Q, SINHA S, et al. Automated test generation for REST APIs: No time to rest yet[C]//Proceedings of the 31st ACM SIGSOFT International

- Symposium on Software Testing and Analysis. 2022: 289-301.
- [33] GOLMOHAMMADI A, ZHANG M, ARCURI A. Testing RESTful APIs: A survey[J]. ACM Transactions on Software Engineering and Methodology, 2023, 33(1): 1-41.
- [34] MILLER B P, FREDRIKSEN L, SO B. An empirical study of the reliability of UNIX utilities[J]. Commun. ACM, 1990, 33(12): 32-44.
- [35] ZHU X, WEN S, CAMTEPE S, et al. Fuzzing: A survey for roadmap[J]. ACM Comput. Surv., 2022, 54(11s): 1-36.
- [36] 徐威, 武泽慧, 王子木, 等. 基于测试用例自动化生成的协议模糊测试方法[J]. 计算机科学, 2023, 50(12): 58-65.
- [37] MENG R, MIRCHEV M, BÖHME M, et al. Large language model guided protocol fuzzing[C]//Proceedings of the 31st Annual Network and Distributed System Security Symposium (NDSS): vol. 2024. 2024.
- [38] 欧先飞, 蒋炎岩, 许畅. 语义可感知的灰盒编译器模糊测试[J]. 软件学报, 1-17.
- [39] 张少坤, 李元春, 雷瀚文, 等. 基于多模态表征的移动应用 GUI 模糊测试框架[J]. 软件学报, 2024, 35(07): 3162-3179.
- [40] QIAN R, ZHANG Q, FANG C, et al. Dipri: Distance-based seed prioritization for greybox fuzzing[J]. ACM Transactions on Software Engineering and Methodology, 2024, 34(1): 1-39.
- [41] MANÈS V J, HAN H, HAN C, et al. The art, science, and engineering of fuzzing: A survey[J]. IEEE Transactions on Software Engineering, 2021, 47(11): 2312-2331.
- [42] 牛胜杰, 李鹏, 张玉杰. 模糊测试技术研究综述[J]. 计算机工程与科学, 2022, 44(12): 2173-2186.
- [43] 张雄, 李舟军. 模糊测试技术研究综述[J]. 计算机科学, 2016, 43(05): 1-8+26.
- [44] HADI M U, QURESHI R, SHAH A, et al. Large language models: a comprehensive survey of its applications, challenges, limitations, and future prospects

- [J]. Authorea Preprints, 2023, 1: 1-26.
- [45] 赵月, 何锦雯, 朱申辰, 等. 大语言模型安全现状与挑战[J]. 计算机科学, 2024, 51(01): 68-71.
- [46] 李岩, 杨文章, 张翼, 等. 基于大语言模型的模糊测试研究综述[J]. 软件学报, 1-28.
- [47] LIU Z, CHEN C, WANG J, et al. Fill in the blank: Context-aware automated text input generation for mobile GUI testing[C]//2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE). 2023: 1355-1367.
- [48] SIDDIQ M L, SANTOS J, TANVIR R H, et al. Exploring the effectiveness of large language models in generating unit tests[J]. arXiv preprint arXiv:2305.00418, 2023, 1(3).
- [49] KIM M, STENNETT T, SHAH D, et al. Leveraging large language models to improve REST API testing[C]//Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results. 2024: 37-41.
- [50] 骆斌, 丁二玉, 刘钦. 软件工程与计算 (卷二): 软件开发的技术基础[M]. 机械工业出版社, 2012.

